
THE CUT-AGNOSTIC DIVISION THEOREM: WHY EVERY OPEN PROBLEM IS EXACTLY ITS PROVED PART AND ITS UNICORN

One Theorem for Any Cut on Any Row, Proved in Core Lean 4 Without Axiom, and Applied to the Seven Millennium Rows and the Sixteen Extension Rows · Twenty Real Parts Sealed on Citation, Two Rows Assigned No Cut, One Row Crossed, and the Fusion Law That Lowers Gold to Lead

FORMAL VERIFICATION · FOUNDATIONS OF MATHEMATICS

One theorem for any cut on any row; the certified part of every open problem named as gold and kept from the alloy; nothing authored

MOHAMMAD F. ISLAM, PhD¹

¹ Independent Researcher, USA. Correspondence: islammm@alumni.iu.edu

25 September 2026

ABSTRACT. For any row, a set of instances with a property, and any decidable cut on it, the row is exactly its Real part inside the cut joined to its Unicorn part outside it; given the Real part, the fused row is exactly the Unicorn part, so fusion with the open remainder lowers a proved part to open; a complete checked list inside the cut proves the Real part with no premise; a crossed row has an empty Unicorn part under every cut; the empty cut leaves the whole value Unicorn; on an infinitude row one exhibited witness above a height proves the Real part while the fused value stays where it was, by a proved equivalence; and the Unicorn part read as a set, the counterexamples outside the cut, is empty exactly when the Unicorn proposition holds. These are seven theorems of core Lean 4, with no library and no axiom at all, kernel-reported, and they do not mention the Riemann Hypothesis, where a special case of them was first used. The same file carries the exact instance-and-property specification of the eight arithmetic height-cut rows, with primality and each property defined in core Lean and the Real part computed in the kernel at a small height, the gap-to-interval bridge that the Legendre row needs, and a declared standing map under which fusion with an open part lowers a proved part to open. Applied to the twenty-three rows of the program's barrier study, the seven Millennium problems and sixteen further open problems, the paper assigns every row one of four shapes, as data, and the census of the assignments is kernel-checked: nine height-cut rows in the Riemann shape, two of them infinitude rows whose certificate is one witness, eleven regime-cut rows whose proved part is a dimension, a degree, a rank, a class, or a bounded model of computation, two rows assigned no cut because the literature supplies no inhabited cut with a proved Real part, an assessment of two literatures and not a theorem, and one crossed row. Twenty rows therefore carry a Real part that seals on citation, a theorem of the literature applied through the kernel schema, grade A joined to grade K; two are assigned none, under the stated criterion; one is whole. The criterion of assignment is stated, and it corrected the author three times before release, on Lehmer's conjecture, on Birch and Swinnerton-Dyer, and on P versus NP. No row's verdict on its fused statement moves, and every owed bit of the barrier study lives exactly where it lived, in the Unicorn part of its row. What is new is not any of the twenty results, each of which exists as a cited theorem, but the statement that they are one theorem under one cut-agnostic division, exact on every row, with the placement of every certificate under the strata of the grounding reading, rung, ladder, and Ground, and with the seat of every row carried from the prior edition, the value excepted by theorem on every row. No bound, witness, or Real part is authored; the definitions of the arrangement are, and every one is listed.

KEYWORDS Real part; Unicorn part; cut-agnostic division; certificate schema; fusion law; Millennium problems; twenty-three rows; Root Axiom; ZFC under RAM; Lean 4

INTRODUCTION

Let a row be a set of instances with a property, and let a cut be any decidable predicate on the instances. Then

$$\text{Row} \leftrightarrow \text{Real}_{\text{cut}} \wedge \text{Unicorn}_{\text{cut}},$$

where the Real part says that every instance inside the cut has the property and the Unicorn part says that no instance outside it fails it. This is a theorem of core Lean 4 with no user-declared axiom, and so are six companions: given the Real part, the fused row is ex-

actly the Unicorn part; a complete checked list inside the cut proves the Real part with no premise; on an infinitude row one witness above a height proves the Real part, and the Unicorn part above any height is equivalent to the whole; a crossed row has no counterexample outside any cut; the empty cut leaves the whole value Unicorn. The seven say nothing about any particular problem. They are the whole of what this paper proves, and they are proved once.

Applied to the twenty-three rows of the program's barrier study, the seven Millennium problems and sixteen further open problems, the seven theorems do something the literature has not done for those rows as a body. Each row already carries, scattered through its own literature, a proved part: every zero of ζ to height 3×10^{12} on the line; every even number to 4×10^{18} a sum of two primes; no odd perfect number below 10^{1500} ; the Hodge conjecture in dimension at most three; Birch and Swinnerton-Dyer at analytic rank at most one; the two-dimensional Yang-Mills theory constructed; the topological four-dimensional Poincaré conjecture. Each is stated in its own idiom, "verified up to," "known for," "proved when," and each is alloyed, in the conventional statement of its problem, with the open remainder, so that the problem reads as one open sentence and the gold inside it goes unnamed. The division names it. On twenty rows the proved part is a Real part under a cut the row's own literature supplies, a height, a dimension, a degree, a rank, a class, and it seals on citation: a theorem of the literature applied through the kernel schema, grade A joined to grade K, exactly as the Real part of the Riemann Hypothesis was sealed in the paper that first used the division at one height [Islam 2026f]. On two of the twenty, the infinitude rows, the certificate is one exhibited witness and the fused value is unchanged by it, which is the exact sense in which verification certifies nothing about an infinitude, and that too is a theorem. On two rows the literature supplies no inhabited cut with a proved Real part, because the row's value is uniform in a parameter that no instance has been cut from, the abc conjecture's for every ε for one; that is an assessment of two literatures, stated as data and falsifiable by a cut, never a theorem, and the paper says so wherever it is used. One row is crossed, and its Unicorn part is empty by theorem. The assignments are data; the census of the assignments, nine, eleven, two, and one, is kernel-checked, and the paper never says more of it than that.

What is claimed and what is not. The paper proves the seven theorems of the division, the bridge lemmas and small-height instances that accompany them, and the census of the assignments, and it carries, from the prior edition on these rows [Islam 2026g], the seat of every row and the theorem that existence decides no row's value. It seals twenty Real parts on cita-

tion. It proves no row, moves no row's verdict on its fused statement, and names no bound of its own: every height, dimension, degree, rank, and class in the tables below is the literature's, attributed at its own grade, and with every citation deleted every theorem of the paper would stand and exactly twenty-two sentences would fall, the twenty Real parts, the crossing of the Poincaré row, and the ZFC instance of the placement. The Unicorn part of every open row is named, set apart, and by the program's standing ruling never chased. The contribution is the arrangement, and the paper is exact about what the arrangement is and is not: one theorem where the literature has twenty-three idioms, the fusion law that keeps each proved part from being lowered to the standing of its remainder, and the placement of every certificate in the strata of the grounding reading, which the prior edition on these rows did not have and the Riemann paper had for one row only.

How the paper is organized. Section 2 states and proves the division, Section 3 places it under the strata of the grounding reading, and Section 4 assigns every row its cut and checks the census in the kernel. Section 5 seals the twenty Real parts, one theorem each. Section 6 states the two rows assigned no cut, with the ledger of what their literatures have proved beside them, and the crossed row by theorem. Section 7 carries the seat and the value excepted from the prior edition, and Section 8 carries the world typing of the barrier study as data, its One-Cut Hypothesis untouched. Section 9 states the standard of proof, the standing of the paper without its citations, and the disclosures. Section 10 positions the result against the literature and concludes. The kernel files are printed whole in the appendices with their audits.

What a reader who suspects vocabulary should check. Every theorem named in the paper is in an appendix with its kernel dependency report; every bound is cited to a source outside the program; the standing table of Section 9 says which sentences fall when the citations are deleted; and Section 4 states the criterion by which a row is assigned its shape and applies it to a row outside the twenty-three. A reader who disputes an assignment can change it: the division holds for the new cut, the census is recounted, and nothing else moves.

Notation. A row r is a pair (Inst_r, P_r) of predicates on a type of instances; its value is $\text{Row}(r) : \Leftrightarrow \forall x, \text{Inst}_r(x) \rightarrow P_r(x)$. A cut c is a decidable predicate on the same type. Grades are those of the Riemann paper: K, a kernel theorem with no user-declared axiom; K given A, a kernel theorem whose hypotheses are cited theorems; A, a cited theorem of the literature. The root axiom RA, the grounding reading RAM with its strata L3m, L2m, L1m, and the tokens $[\Xi_0]$ and $[\emptyset_0]$ are the program's and enter only in Sections 3, 7, and

8, where they are defined at use.

THE DIVISION THEOREM

Everything in this section is in `Row_Split.lean` (Appendix A), 336 lines, core Lean 4.19.0, no library, no sorry, no user-declared axiom, twenty-seven dependency sets printed, every one reported by the kernel as depending on no axiom at all, not even Lean’s built-in `propext` and `Quot.sound`. Seven are the division; the rest are the substantive-cut and counterexample readings, the standing map, the gap bridge, and the row specifications with their computed small-height Real parts, treated in the sections that use them. The instances form an arbitrary type α ; a row is a pair of predicates; a cut is a decidable predicate.

Definitions. $\text{Row}(\text{Inst}, P) :\Leftrightarrow \forall x, \text{Inst } x \rightarrow P x$.
 $\text{RealIn}(\text{Inst}, P, c) :\Leftrightarrow \forall x, \text{Inst } x \rightarrow c x \rightarrow P x$.
 $\text{UnicornOut}(\text{Inst}, P, c) :\Leftrightarrow \forall x, \text{Inst } x \rightarrow \neg c x \rightarrow P x$.
A certificate for a cut is a list of instances together with a proof that every instance inside the cut is on the list and a proof that every listed instance has the property.

Theorem 1 (`row_iff_real_and_unicorn`). For every row and every decidable cut, $\text{Row} \leftrightarrow \text{RealIn} \wedge \text{UnicornOut}$. *Proof.* Left to right, restrict the universal to each side of the cut. Right to left, an instance either satisfies the cut or does not, by decidability, and the corresponding conjunct applies. $\square$

Theorem 2 (`fused_is_unicorn_given_real`). Given the Real part, $\text{Row} \leftrightarrow \text{UnicornOut}$. *Proof.* From Theorem 1, the Real part being supplied. $\square$ This is the fusion law read at the level of truth: given the Real part, the fused row is true exactly when the Unicorn part is. The reading at the level of standing, that a proved part joined to an open part stands open, is not a consequence of Theorem 2 alone; it is a consequence of Theorem 2 together with the program’s grade law, that the standing of a conjunction is the weakest standing of its conjuncts. The paper declares that law as a rule in the kernel (`Standing.join`) and proves the fusion sentence under it (`fusion_lowers`): a proved part joined to an open part stands open, and only two proved parts stand proved. The two-point rule is the restriction to two standings of the program’s nine-grade weakest-link law, `Grade.weakest` in the Code Block of the Master Codex, under which a citation cannot promote a claim (`citation_cannot_promote`); any rule under which a conjunction stands at least as low as its weakest conjunct gives the same sentence, and the paper accepts no rule under which it stands higher. Gold alloyed with lead assays as lead, under the assay rule stated; the gold does not cease to be gold, and no theorem says it does.

Theorem 3 (`real_of_certificate`). A certificate for a cut proves `RealIn` for that cut, with no premise. *Proof.* An instance inside the cut is on the list by completeness and has the property by the check. $\square$ The

schema is what a finite verification proves and all that it proves: the bounded universal inside the cut.

Theorem 3b (`real_of_one_witness`). On an infinitude row the instances are bounds N and the property is that some witness exceeds N ; one exhibited witness w with $W(w)$ and $w > h$ proves `RealIn` under the cut $N \leq h$. *Proof.* The witness exceeds every $N \leq h$. $\square$ By Theorem 2 the fused value is then exactly the Unicorn part, the bounds above h , which is equivalent to the whole infinitude; so on such a row the Real part is proved, is gold, and moves the fused value not at all. That is the precise content of the familiar remark that exhibiting large twin primes proves nothing about their infinitude: it proves the Real part and only the Real part.

Theorem 3c (`unicorn_equiv_row_of_infinitude`). On an infinitude row, for every height h , `UnicornOut` above h is equivalent to the whole value: $(\forall N > h, \exists m > N, W(m)) \leftrightarrow (\forall N, \exists m > N, W(m))$. *Proof.* Right to left, restrict. Left to right, a bound $N \leq h$ is served by the witness above $h + 1$. $\square$ So on an infinitude row the Real part is proved by one witness (Theorem 3b) and, by Theorems 2 and 3c, the fused value is exactly what it was.

Theorem 4 (`crossed_row_both_parts`, `crossed_row_no_counter`). Read as a set, the Unicorn part under a cut is the set of counterexamples outside it, $\text{CounterOut}(x) :\Leftrightarrow \text{Inst } x \wedge \neg c x \wedge \neg P x$; the Unicorn proposition says that this set is empty (`no_counter_of_unicorn`, and conversely for decidable P , `unicorn_of_no_counter`). If Row holds, then under every cut both `UnicornOut` and `RealIn` hold and the counterexample set is empty. *Proof.* Immediate. $\square$ “Empty Unicorn part” means, throughout the paper, that this set is empty, which is the sense in which a crossed row has nothing outside any cut left to fail.

Theorem 5 (`empty_cut_vacuous`, `empty_cut_not_substantive`). Under the empty cut, `RealIn` holds vacuously and $\text{UnicornOut} \leftrightarrow \text{Row}$; and the empty cut never carries a substantive Real part, where a Real part is substantive when its cut is inhabited on the row, $\text{SubstantiveRealIn} :\Leftrightarrow (\exists x, \text{Inst } x \wedge c x) \wedge \text{RealIn}$. *Proof.* Nothing satisfies the empty cut. $\square$ This theorem is about the empty cut and about nothing else. It does not say that a row admits no other cut with a Real part; whether a given row’s literature supplies an inhabited cut with a proved Real part is an assessment of that literature, and the paper records it as data (Section 4), never as a consequence of this theorem.

The one arrow (tier: kernel theorem, axiom-free). On any row, under any decidable cut, the value is exactly its Real part joined to its Unicorn part; given the Real part, the fused value is true exactly when the Unicorn part is, and it

stands open under the declared standing rule; a certificate proves the Real part and nothing beyond it; a crossed row has no counterexample outside any cut; under the empty cut the whole value is Unicorn. Nothing in the seven theorems names a height, a problem, or a number. The Riemann division of [Islam 2026f] is the instance with the cut “height at most T ”.

What the theorems do not say. They do not say that a Real part is easy to obtain, since the certificate is the literature’s and may be the work of decades; they do not say that a Unicorn part is unreachable, only that no certificate reaches it; and they do not rank cuts, since a row admits many and the division holds for each. The choice of cut on a given row is the row’s literature’s, and Section 4 records it as data.

THE PLACEMENT: WHERE A REAL PART LIVES

The program’s grounding reading, RAM, stratifies any theory into three: computation, the rung, L3m; provability, the ladder, L2m; truth in the intended world, the Ground, L1m. The placement is a theorem for an arbitrary theory modelled as sentences with a provability predicate, a decidable fragment, and a truth predicate: under two named premises, that what computation settles the theory proves and that what the theory proves is true, $\text{rung} \subseteq \text{ladder} \subseteq \text{Ground}$. For ZFC the first premise is Σ_1^0 -completeness, a theorem of arithmetic [Kaye 1991], and the second is soundness, which no consistent theory proves of itself [Gödel 1931] and which is therefore carried openly, derived nowhere. Soundness is load-bearing, an unsound theory breaking $\text{ladder} \subseteq \text{Ground}$; the Ground exceeds the ladder, a sound theory leaving a truth unproved; independent axioms are separate bits, Choice over ZF the cited instance [Gödel 1938; Cohen 1963]. Subsumption is by placement only: RAM locates ZFC and derives none of its axioms. All of this is ZFC_Under_RAM.lean (Appendix B), sixteen dependency sets, every one axiom-free, seated in the Master Codex as APEX-PSP-ZFC-UNDER-RAM-01 [Islam 2026a]. The file encodes no axiom of ZFC and no proof relation of ZFC; its theory is an arbitrary structure with a computation predicate, a provability predicate, and a truth predicate, and ZFC is the instance the paper names, entering through the two cited premises. What the kernel proves is the conditional; what ZFC supplies is its two hypotheses; the paper never writes otherwise.

Where a Real part lives. A Real part under a height cut on an arithmetic row is a rung: its certificate is finitely many checks, each a true bounded sentence, so the Real part is a theorem of arithmetic and of ZFC by Σ_1^0 -completeness, and it holds on the Ground by soundness carried openly. A Real part under a regime cut on a non-arithmetic row is a cited theorem of ZFC directly, on the ladder, with the same passage to the

Ground. In both cases the round trip through the root and back returns the row’s value unchanged, since the root crosses the bridge as presence and carries no keyed sentence; and every step holds with the root replaced by a bare computation arrow, the identity on a type (`any_true_premise_serves`), so the root is present in the performance of every check and idle in its content. The placement adds no standing to any Real part; it says which stratum a standing already earned is in, and it is the same stratum on every row of the same shape.

Why the placement matters here. Without it, “verified up to” on one row and “proved for dimension at most three” on another would sit in two vocabularies with no common address. With it, both are theorems of ZFC on the ladder, one reached from the rung by a certificate and one stated there by a proof, and the Unicorn part of each is the sentence on the ladder or beyond it that the row’s foundation has not decided. The one arrow of Section 2 and the placement of this section are the two results this paper adds to the twenty-three rows; everything else is carried or cited.

THE ROWS AND THEIR CUTS

The twenty-three rows are the barrier study’s [Islam 2026e], in its order, with its world typing carried as data in Section 8. Each row is assigned a cut and a shape here. The assignment is the paper’s, stated as data; what the kernel checks is the census of the assignments: `rows23` is a list of the twenty-three rows with their assigned shapes, and `census` proves by decision that there are twenty-three, that nine are assigned height-cut, eleven regime-cut, two no-cut, and one crossed, and that twenty are assigned a Real-part-bearing shape (`real_part_rows`); both axiom-free. The kernel does not and cannot decide that the Riemann Hypothesis is a height-cut row; that is the assignment, and it is made by the criterion stated next.

The criterion. A row is assigned by the logical form of its value and the state of its literature, in this order. It is *crossed* if its value is a theorem. It is *height-cut* if its instances carry a total height, the property at an instance is decidable or the row is an infinitude with instances the bounds, and the literature supplies a certificate for some height, a checked list or one witness. It is *regime-cut* if its literature supplies a proved regime, a decidable predicate on the instances under which the property is a theorem, and no height certificate; a row whose literature supplies both a proved regime and a certificate inside it, Birch and Swinnerton-Dyer with its verified curves at analytic rank at most one, is assigned regime-cut, the regime being the larger Real part, and the certificate is recorded beside it. It is *no-cut* if the literature supplies neither: no inhabited decidable cut on the row’s instances under which the property is proved. The

criterion is a classification rule applied to two inputs, the value's logical form and the recorded state of its literature; the second input is a reading of the literature and not a computable object, which is why the assignments are data. A reader who applies the rule with a different reading of either input changes an assignment and nothing else. In every shape the cut is a predicate on the row's own instance type, as Theorem 1 requires: on the Hodge row the instances are pairs of a variety and a class and the regime is a bound on the dimension; on the Birch and Swinnerton-Dyer row the instances are curves and the regime is a bound on the analytic rank; on the P versus NP row the instances are algorithms and the regime is a bound on time and space; a regime is never a different problem placed beside the row.

The criterion applied out of sample. Take a row outside the twenty-three, Lehmer's conjecture, that the Mahler measure of a non-cyclotomic integer polynomial is at least a constant above one. Its instances are polynomials, with degree as a total height; the property at an instance is decidable; the literature supplies a certificate for a height, that every non-cyclotomic polynomial of degree at most 180 has measure at least 1.176... , the measure of Lehmer's polynomial [Mossinghoff, Rhin and Wu 2008]. The criterion assigns height-cut with the cut degree ≤ 180 ; the Real part is that computation; the Unicorn part is degree above 180, and the uniform constant is what the row asks. The assignment was made by the criterion before the row was written down here, which is the only sense in which a classification can be tested out of sample; the first draft of this paragraph assigned the row regime-cut, and the criterion corrected it, since a finite computation over a height is a certificate whatever it is called in its source. The criterion corrected the author twice more before release: on Birch and Swinnerton-Dyer, where the tie-break above was written because the first assignment had not faced the verified curves, and on P versus NP, which the internal builds assigned no-cut until the external reviewers asked what an instance of that row is; the answer, an algorithm, together with the time-space lower bounds for satisfiability, is an inhabited cut with a proved Real part, and the row is regime-cut. A criterion that never corrects its author is a label; this one is a rule.

The four shapes. A *height-cut* row has a natural height parameter and a finite certificate per height; its Real part seals on citation of the certificate through Theorem 3, or through Theorem 3b when the row is an infinitude and the certificate is one witness, and its Unicorn part is the tail above the height: the shape of the Riemann row. A *regime-cut* row has a proved part that is a dimension, a degree, a rank, or a class rather than a height; its Real part is a cited theorem on the ladder, sealed through Theorem 1 with the regime as

the cut, and its Unicorn part is the complement. A *no-cut* row is one for which the literature supplies no inhabited decidable cut on its instances under which the property is proved, because its value is uniform in a parameter that no instance has been cut from; the assignment is an assessment of the literature, and the only theorem about it is Theorem 5, that the empty cut is vacuous and carries no substantive Real part. What such a row's literature has proved nearby, a weaker theorem, stands beside the row at its own grade and not inside it. A *crossed* row has its value supplied whole; by Theorem 4 its counterexample set is empty under every cut.

Three assignments that deserve a word. The twin-prime and Mersenne rows are infinitudes, and their Real parts are sealed by one witness each through Theorem 3b; their Unicorn parts are equivalent to their whole values, so the fused verdict on each is unchanged, exactly as Theorem 2 says. They are height-cut rows whose gold is one number, and the paper says so rather than calling them no-cut, since a proved witness is a Real part however little it moves the remainder. The Legendre row has no dedicated certificate in the literature; its Real part is an inference from the maximal prime-gap tables to 4×10^{18} [Oliveira e Silva, Herzog and Pardi 2014], in which the largest gap is 1476, so that for $n \geq 738$ the interval $(n^2, (n+1)^2)$, of length $2n+1 \geq 1477$, contains a prime whenever $(n+1)^2 \leq 4 \times 10^{18}$, the smaller n being checked directly; the inference is stated as such in Section 5. The Sunflower row is assigned no cut because the conjecture's constant-base bound is a statement about all k at once and the improved bound of Alweiss, Lovett, Wu and Zhang [2020] is a theorem beside the row, not a region inside it. A reader who prefers a different cut on any row may take it: the division holds for every cut, and only the assignment changes.

THE TWENTY REAL PARTS, SEALED ON CITATION

Each theorem below has the same form and the same proof: a cut, a cited theorem of the literature that supplies the Real part inside it, and the schema step, Theorem 3 for a certificate, Theorem 3b for a witness, or Theorem 1 with the regime as the cut for a proved regime. The standing of every one is grade A joined to grade K: the kernel proves the schema and the division, the literature proves the bound, and the kernel never recomputes a bound. Grade A joined to K is a documented composition of a cited theorem with a kernel theorem; it is not a kernel-checked theorem about the cited result, and the paper never calls it one. The Unicorn part of each row is stated beside its Real part and is not claimed. Bounds are the sources', quoted at their grade; the paper checks none of them.

The row specifications, in the kernel. For the eight

Table 1. The twenty-three rows, their shapes, and their cuts. The shape and the cut are the paper’s assignments by the criterion of the text, stated as data; the census of the assignments is kernel-checked (census, real_part_rows). Sources for every bound are in Section 5.

#	Row	Shape	Cut	Real part	Unicorn part
1	P versus NP	regime	time and space	no algorithm in time n^c , $c < 2 \cos(\pi/7)$, and space $n^{o(1)}$ decides SAT	polynomial time with unrestricted space
2	Riemann Hypothesis	height	height T	every zero to 3×10^{12} on the line	no off-line zero above T
3	Navier–Stokes	regime	regime	2D global regularity; 3D small-data and short-time regularity	3D large-data global regularity or blow-up
4	Yang–Mills	regime	dimension	the two-dimensional theory constructed	four-dimensional existence with a mass gap
5	Hodge	regime	dimension, codimension	codimension one; codimension $n - 1$; every variety of dimension ≤ 3	middle codimensions in dimension ≥ 4
6	BSD	regime	analytic rank	the rank statement at analytic rank 0 and 1; the full conjecture for specific curves	analytic rank ≥ 2 ; the full formula in general
7	Poincaré	crossed	—	the whole	empty
8	Goldbach	height	height n	every even $n \leq 4 \times 10^{18}$; the ternary conjecture whole	even numbers above the height
9	Twin primes	height	height N	a twin pair above every N below the largest known pair, one witness	bounds above the witness, equivalent to the whole
10	Legendre	height	height n	a prime in $(n^2, (n+1)^2)$ for $n \leq 2 \times 10^9$, by inference from the prime-gap tables	n beyond the tables
11	abc	no cut	none ($\forall \epsilon \exists C \forall$)	none by certificate	the whole value
12	Jacobian	regime	degree	degree ≤ 2 in every dimension; $n = 1$	cubic maps in dimension ≥ 2
13	Mersenne primes	height	height N	a Mersenne prime above every N below the largest known, one witness	bounds above the witness, equivalent to the whole
14	Odd perfect numbers	height	height N	none below 10^{1500}	none above
15	Smooth 4D Poincaré	regime	category	the topological case	the smooth case
16	Collatz	height	height n	every $n < 2^{68}$ reaches 1	$n \geq 2^{68}$
17	Hadwiger	regime	t	$t \leq 6$	$t \geq 7$
18	Sunflower	no cut	none	none by certificate	the constant-base bound
19	Erdős–Straus	height	height n	every $n \leq 10^{17}$	n above the height
20	Beal	height	box	the searched boxes of bases and exponents	outside every searched box
21	Invariant subspace	regime	operator class	compact operators; normal operators; finite dimension	the general bounded operator on a separable Hilbert space
22	Hilbert’s 16th, second part	regime	per field, uniform	finiteness of limit cycles for each polynomial field; $H(1) = 0$	the uniform bound $H(n)$, open at $n = 2$
23	Lindelöf / Montgomery PCC	regime	exponent, range	the proved exponents; pair correlation in the restricted range under RH	ε ; the full range

arithmetic height-cut rows the exact tuple, the type of instances, the instance predicate, the property, and the cut, is written in `Row_Split.lean`: instances are natural numbers, primality is trial division (`isPrime`), and each property is a Boolean function defined in core Lean, `goldbach0K`, `legendre0K`, `oddPerfect0K`, `collatz0K` with a step budget, `erdosStraus0K` with a bounded search, `beal0K` on a box, and the witness forms for twin and Mersenne primes. For each, the Real part at a small height is computed in the kernel by decision and lifted to `RealIn` by `real_of_all_range` or `real_of_one_witness`: Goldbach to 60, Legendre to 8, odd perfect numbers to 60, Collatz to 60, Erdős–Straus to 6, the Beal box (3, 4), a twin pair above every $N \leq 100$, a Mersenne prime above every $N \leq 100$. These small instances are grade K with no citation: they show the schema instantiated end to end at heights the kernel can decide. The literature’s heights are the same statements at heights the kernel cannot decide, and they are cited. For the Riemann row and the eleven regime rows the instance types are analytic, geometric, or computational and are not definable in core Lean; their tuples are stated in prose at each theorem, and the schema is applied there by documented composition.

The nine height-cut rows

Theorem 6, Riemann. Tuple: instances the nontrivial zeros of ζ , property that the real part is $\frac{1}{2}$, cut imaginary part at most T . Every nontrivial zero with imaginary part up to 3×10^{12} lies on the critical line, by the interval-arithmetic certificate of Platt and Trudgian [2021] with the analytic lemmas it rests on [Turing 1953], through Theorem 3. Unicorn part: no off-line zero above that height. This is the Real part of the Riemann paper [Islam 2026f], carried at its recorded standing.

Theorem 7, Goldbach. Tuple in the kernel: $(\mathbb{N}, \top, \text{goldbach0K}, n \leq h)$; Real part computed to $h = 60$ (`goldbach_real_60`). Every even integer $4 \leq n \leq 4 \times 10^{18}$ is a sum of two primes, by the verification of Oliveira e Silva, Herzog and Pardi [2014], through Theorem 3. Beside it, the ternary conjecture, every odd integer above 5 a sum of three primes, stands whole [Helfgott 2013]. Unicorn part: even integers above the height.

Theorem 8, twin primes. Tuple in the kernel: instances the bounds N , property that a twin pair exceeds N , cut $N \leq h$; Real part computed to $h = 100$ from the pair (101, 103) (`twin_real_100`). For every N below $2996863034895 \cdot 2^{1290000} - 1$ there is a pair of

primes at distance 2 above N , since $2996863034895 \cdot 2^{1290000} \pm 1$ is such a pair [PrimeGrid 2016], through Theorem 3b. Beside it, infinitely many pairs of primes at distance at most 246 [Zhang 2014; Maynard 2015; Polymath 2014]. Unicorn part: the bounds above the witness, equivalent to the whole conjecture by Theorem 3c.

Theorem 9, Legendre. Tuple in the kernel: $(\mathbb{N}, \top, \text{legendreOK}, n \leq h)$; Real part computed to $h = 8$ (`legendre_real_8`). For every $738 \leq n$ with $(n+1)^2 \leq 4 \times 10^{18}$ there is a prime in $(n^2, (n+1)^2)$: the maximal prime gap below 4×10^{18} is 1476 [Oliveira e Silva, Herzog and Pardi 2014], and the bridge from a gap bound to a prime in every interval of length at least the bound plus one is a kernel theorem (`elem_in_gap`, `legendre_of_gap`), which takes the gap bound and the primality of 2 as hypotheses and returns a prime strictly between n^2 and $(n+1)^2$ for every $n \geq 738$ with $(n+1)^2$ within the tabulated range; for $n < 738$ the interval is checked directly by the author's script, and to $n \leq 8$ in the kernel. Through Theorem 3 for the small heights and the bridge for the tables; the certificate is an inference from the gap tables and is stated as such. Unicorn part: n beyond the tables.

Theorem 10, Mersenne primes. Tuple in the kernel: instances the bounds N , property that a Mersenne prime exceeds N , cut $N \leq h$; Real part computed to $h = 100$ from $127 = 2^7 - 1$ (`mersenne_real_100`). For every N below $2^{136279841} - 1$ there is a Mersenne prime above N , since that number is prime [GIMPS 2024], through Theorem 3b. Unicorn part: the bounds above the witness, equivalent to the whole conjecture by Theorem 3c.

Theorem 11, odd perfect numbers. Tuple in the kernel: $(\mathbb{N}, \top, \text{oddPerfectOK}, n \leq h)$; Real part computed to $h = 60$ (`oddperfect_real_60`). No odd perfect number is below 10^{1500} [Ochem and Rao 2012], through Theorem 3. Unicorn part: none above.

Theorem 12, Collatz. Tuple in the kernel: $(\mathbb{N}, \top, \text{collatzOK}, n \leq h)$, the property that the orbit reaches 1 within a step budget; Real part computed to $h = 60$ (`collatz_real_60`). Every positive integer below 2^{68} reaches 1 under the map [Barina 2020], through Theorem 3. Beside it, almost every orbit attains almost bounded values in the sense of logarithmic density [Tao 2022]. Unicorn part: $n \geq 2^{68}$.

Theorem 13, Erdős–Straus. Tuple in the kernel: $(\mathbb{N}, \top, \text{erdosStrausOK}, n \leq h)$, the property a bounded search for $x \leq y \leq z \leq n^2$ with $4xyz = n(yz + xz + xy)$; Real part computed to $h = 6$ (`erdosstrauss_real_6`). For every $2 \leq n \leq 10^{17}$, $4/n$ is a sum of three unit fractions [Salez 2014], through Theorem 3. Unicorn part: n above the height.

Theorem 14, Beal. Tuple in the kernel: instances the boxes, property `bealOK`, the box $(3, 4)$ computed

(`beal_box_3_4`). No coprime solution of $A^x + B^y = C^z$ with $x, y, z \geq 3$ exists with bases $A, B \leq 1000$ and exponents $x, y \leq 100$, the search run and printed by Norvig [2017], nor with $A, B \leq 200,000$ and $x, y \leq 5000$, the search of Jarnicki and Konerding reported there; through Theorem 3 with the box as the cut. Unicorn part: outside every searched box.

The eleven regime-cut rows

Theorem 15, Navier–Stokes. In two dimensions, smooth solutions exist globally for all time [Ladyzhenskaya 1969]; in three dimensions, weak solutions exist globally [Leray 1934] and smooth solutions exist for all time for small data and for a short time for all data [Fujita and Kato 1964]. Through Theorem 1 with the regime as the cut. Unicorn part: three-dimensional global regularity for large data, or its failure.

Theorem 16, Yang–Mills. The two-dimensional Yang–Mills measure is constructed rigorously on compact surfaces [Driver 1989; Lévy 2003]. Through Theorem 1 with dimension as the cut. Unicorn part: four-dimensional existence with a mass gap.

Theorem 17, Hodge. The Hodge conjecture holds for classes of codimension one on every smooth projective variety [Lefschetz 1924] and, by hard Lefschetz, for classes of codimension $n - 1$, hence for every variety of dimension at most three. Through Theorem 1 with dimension and codimension as the cut. Unicorn part: the middle codimensions in dimension at least four.

Theorem 18, Birch and Swinnerton-Dyer. For an elliptic curve over $\mathbb{Q}$ of analytic rank 0 or 1, the rank equals the analytic rank and the Tate–Shafarevich group is finite [Gross and Zagier 1986; Kolyvagin 1989]; the full conjecture, with its formula, is verified for many specific curves of small conductor [Miller 2011]. Through Theorem 1 with analytic rank as the cut. Unicorn part: analytic rank at least two, and the full formula in general.

Theorem 19, Jacobian. Every polynomial map $\mathbb{C}^n \rightarrow \mathbb{C}^n$ of degree at most two with nonzero constant Jacobian is invertible [Wang 1980], and the conjecture in general reduces to the cubic homogeneous case [Bass, Connell and Wright 1982]; the case $n = 1$ is elementary. Through Theorem 1 with degree as the cut. Unicorn part: cubic maps in dimension at least two.

Theorem 20, smooth four-dimensional Poincaré. Every topological four-manifold homotopy equivalent to the sphere is homeomorphic to it [Freedman 1982]. Through Theorem 1 with the category as the cut. Unicorn part: the smooth case.

Theorem 21, Hadwiger. Every graph with no K_t minor is $(t - 1)$ -colourable for $t \leq 6$ [Robertson, Sey-

mour and Thomas 1993], the case $t = 5$ being the four-colour theorem. Through Theorem 1 with t as the cut. Unicorn part: $t \geq 7$.

Theorem 22, invariant subspace. Every nonzero compact operator on a complex Banach space of dimension above one has a nontrivial invariant subspace [Lomonosov 1973]; so does every normal operator on a Hilbert space, by the spectral theorem, and every operator on a finite-dimensional space, by an eigenvector. Through Theorem 1 with the operator class as the cut. Unicorn part: the general bounded operator on a separable Hilbert space.

Theorem 23, Hilbert’s sixteenth problem, second part. Every polynomial vector field in the plane has finitely many limit cycles [Ilyashenko 1991; Écalle 1992], and $H(1) = 0$. Through Theorem 1 with the per-field statement as the cut. Unicorn part: the uniform bound $H(n)$ on the number of limit cycles for fields of degree n , open already at $n = 2$.

Theorem 24, P versus NP. Tuple: instances the algorithms, in the multitape or random-access model with a clock, property that the algorithm does not decide satisfiability, cut a bound on time and space. No algorithm running in time n^c for $c < 2 \cos(\pi/7) \approx 1.801$ and space $n^{o(1)}$ decides satisfiability [Williams 2008], extending the first such bound of Fortnow, Lipton, van Melkebeek and Viglas [2005]. Through Theorem 1 with the time-space regime as the cut. Unicorn part: polynomial time with unrestricted space, which is the row. The row was assigned no-cut in the internal builds; the criterion, applied to the instance type the reviewers asked for, moved it.

Theorem 25, Lindelöf and Montgomery. $\zeta(\frac{1}{2} + it) \ll_\epsilon t^{13/84+\epsilon}$ [Bourgain 2017], the current exponent toward the Lindelöf hypothesis; and Montgomery’s pair-correlation conjecture holds, under the Riemann Hypothesis, for test functions with Fourier support in $(-1, 1)$ [Montgomery 1973]. Through Theorem 1 with the exponent and the range as the cuts. Unicorn part: the Lindelöf exponent ϵ , and the full range for pair correlation.

The twenty, in one sentence (tier: A joined to K on every row). Twenty theorems of the literature, each stated in its own idiom, are the Real parts of twenty rows under the cuts their literatures supply, each sealed by the same two kernel theorems and each with its Unicorn part named beside it; by Theorem 2 the fused statement of every one of these rows stands exactly at its Unicorn part, which is why the conventional statement of each problem reads as open although its gold is proved.

THE TWO ROWS ASSIGNED NO CUT, AND THE CROSSED ROW

Theorem 26, the rows assigned no cut. On the rows abc and Sunflower the value is uniform in a parame-

ter that no instance has been cut from: every ϵ at once with nothing proved for any single ϵ , one constant serving every k at once. The assignment says that the literature supplies no inhabited decidable cut on the row’s instances under which the property is proved; that is an assessment of two literatures, stated as data and falsifiable under F3 by exhibiting such a cut, and the paper claims it at no higher grade. What the kernel supplies is Theorem 5: under the empty cut, the only cut this treatment supplies for these rows, the Real part is vacuous, carries no substantive content, and the Unicorn part is the whole value.

The exclusion ledger. An assessment that a literature supplies no inhabited proved cut is only as good as the candidates it has examined, so they are listed. On the abc row the instances are the values of ϵ and the property at ϵ is that only finitely many coprime triples have $c > \text{rad}(abc)^{1+\epsilon}$; nothing of this form is proved for any ϵ . The bound of Stewart and Yu [2001], $c < \exp(\kappa \text{rad}^{1/3}(\log \text{rad})^3)$, is a theorem about all triples and not the property at any ϵ ; the enumeration of all triples with $c < 10^{18}$ [ABC@Home] proves the finiteness of exceptions for no ϵ . The explicit strengthening $c < \text{rad}(abc)^2$ is a different row, a universal over triples with the enumeration as a certificate to 10^{18} , and if the reader prefers that row it is height-cut and the paper has no objection. On the Sunflower row the value as stated has the form “there is a constant C such that for every k ,” so its universal over k carries an unfixed parameter and there is no property at an instance to prove until C is fixed; the bound of Alweiss, Lovett, Wu and Zhang [2020], $(\log k)^{k(1+o(1))}$, is a theorem beside the row, not the constant-base property at any k , and the exact values known for small k are the property only once a C is chosen, which the conjecture does not do. The P versus NP row was on this ledger in the internal builds and is not now: with instances the algorithms, the time-space lower bounds for satisfiability are a proved regime, and the row is Theorem 24. The enumerated abc triples below 10^{18} and the solved instances of any NP problem certify nothing about these values, and the paper claims nothing from them. What each row’s literature has proved nearby stands beside the row at its own grade: the sunflower bound $(\log k)^{k(1+o(1))}$ [Alweiss, Lovett, Wu and Zhang 2020], the lower bounds for restricted circuit classes on the complexity row, the exponential bounds toward abc. None of these is a Real part, because none is inside the row’s cut; each is a theorem about a different row.

Theorem 27, the crossed row. The Poincaré conjecture is a theorem [Perelman 2002; Perelman 2003a; Perelman 2003b], so by Theorem 4 its counterexample set is empty under every cut and its Real part is the whole. The generic theorem is grade K; the application to the Poincaré row is grade A joined to K, since

the crossing is cited. The crossing is the barrier study's record of Perelman's proof, cited here and computed nowhere: no Ricci flow is represented in any file of this paper.

The count. Twenty rows with a Real part, two assigned none under the stated criterion, one whole: twenty-three, and the census of the assignments in the kernel agrees (census). The two rows assigned no cut are the ones on which "verified up to" cannot even be written, because their values are not universals over instances with a fixed property, and the criterion says why in one rule rather than two explanations.

THE SEAT ON EVERY ROW, CARRIED

This section carries the theorems of the prior edition on these rows [Islam 2026g], condensed, with the kernel file `Rows_At_The_Apex.lean` printed whole in Appendix C and its audit in Appendix D. They are the reason the paper can say where the owed bit of every open row lives: in its Unicorn part and nowhere else. Nothing in them is changed.

The root and the seat. The root axiom of the program is RA: to exist is to actuate; existence and RA are one proposition. The seat is the fixed locus of the quaternion involution σ on the Hurwitz order, the scalar line, $\sigma q = q \leftrightarrow q_i = q_j = q_k = 0$ (`fix_iff_scalar`), and the seat point is the Return $(i \cdot j) \cdot k = -1$. Every row's seat point is that one point, $\text{apexSeat}(r) := \langle -1, 0, 0, 0 \rangle$, fixed by σ on every row and equal across every pair of rows, the crossed row and the open rows sharing it (`apex_seat_fixed`, `apex_seat_uniform`, `crossed_and_open_share_the_seat`; no axioms). On every row three names written in three registers, the axiom's Return, its projection onto the fixed locus, and the object register's seat, read one point: the seat point is the apex of a cone over the row's register diagram with every leg definitional, any two cones over the diagram share their apex, and the gap vector of every row is $(0, 0, 0)$ (`identity_cone_row`, `cone_apex_unique_row`, `gaps_closed_every_row`; no axioms). The mathematics of this is Hamilton's product and three definitional equalities, twenty-three times, and the paper claims nothing more for it at theorem grade. The seat is a uniform assignment, `apexSeat` a constant function of the row, and the uniformity theorems are theorems about that assignment and not derivations of a common structure from the twenty-three objects: the seat is assigned by the reading and not bridged from any row's object, one embedding written for the Riemann row and twenty-two owed, on the record (`embeddings_owed`). A reader who wants the seat to speak of the rows' objects wants the twenty-two embeddings, and the paper lists them as owed rather than pretending the constant supplies them.

The witness and the value excepted. A row's witness is a structure with four fields, existence, the orientation bit, the formal Ground, and a recursion field from the bit and the Ground to the row's formal self; it is one term on every row (`mkRow`), its recursion constant and discarding its arguments (`recursion_constant_every_row`), and the formal self is one proposition on every row, so the witness on the crossed row is the witness on every open row (`inverted_control`). Whatever supplied the Poincaré row's bit, it was not the witness. For every assignment of frames to rows and every row, $(RA \rightarrow \text{value}) \leftrightarrow \text{value}$ (`rule_exact_every_row`), and existence decides no row's value: there is an assignment on which every value fails while existence holds, and another, the grounded three-point frame on the Poincaré row itself, on which the same is true (`RA_decides_no_row`, `RA_decides_no_row_grounded`). For every row and every class of frames, $(\forall X, C(X) \rightarrow RA \rightarrow L(X)) \leftrightarrow (\forall X, C(X) \rightarrow L(X))$ (`no_cure_every_row`): every proposal to make existence decide a row by restricting its frames, an Euler product on the Riemann row, a Ricci-flow structure on the Poincaré row, a circuit class on the complexity row, is a class on which the value is already a field, and there existence decides nothing. The value is excepted by theorem, not by reservation. All no axioms.

The world row carried. The world typing of the barrier study is premise-grade data, and the theorems about it are theorems about the table as carried: the apex closes on every row while the world row is crossed on exactly the Poincaré row, and the first never entails the second (`apex_never_promotes_world`); the apex verdict is uniform and the world verdict splits, crossed on one row, $[\Xi_0]$ on the Riemann row, $[\emptyset_0]$ on the complexity row, unrouted on twenty (`world_verdict_split`, `unrouted_twenty`); on no row do the two coincide (`registers_parted_every_row`). The owed bits number twenty-two and are priced at the registration floor, $22 \times 2871 = 63,162$ yJ at 300 K (`corpus_price_total`), the price of the Poincaré row being zero (`poincare_price_zero`). The price rests on the prior edition's Registration Postulate, a premise-grade posit of the program declared there and consumed here: that a world-bit, when it is supplied, is registered irreversibly in a physical substrate at a temperature, taken as 300 K, and so is charged the Landauer floor. Without the postulate the price is a count of bits and no joule attaches to it; with it the price is a multiplication, and the kernel multiplies and asserts nothing physical. The carrier of the seat is the Hurwitz order and not the complex plane because the seat point is the Return $(i \cdot j) \cdot k = -1$, whose sign is the chirality the orientation bit reads (the spine's Theorem A); the Riemann stage embeds in the plane spanned by 1 and i , and the j and k directions are inert for that

stage and load-bearing for the Return, which is why the plane would carry the stage and not the sign.

What the division adds to the seat. The seat locates every row's decision point at one place and proves that existence does not decide any row there. The division of Section 2 says what is decided elsewhere: on twenty rows a Real part, decided by the literature, under a cut; and it says by Theorem 2 that the owed bit of each open row, which the seat proves existence does not supply, is the bit of its Unicorn part and of nothing inside the cut. The two results are independent theorems of the kernel, one about the frame and one about the row's own instances, and the paper joins them only in this sentence.

THE WALL, CARRIED AS DATA

The barrier study [Islam 2026e] typed the twenty-three rows by the shape of their world register and stated the One-Cut Hypothesis: that on every mapped row the blocked register is even, blind to the row's odd target, with exactly one binary orientation coordinate missing at the row's decision seat. Its typing is carried here as data in Table 2, at premise grade, and the hypothesis is carried at hypothesis grade, untouched. The division of this paper neither confirms nor moves it. The hypothesis speaks of the owed bit, which lives in each row's Unicorn part by Theorem 2 and Section 7; the division speaks of the Real part, which the hypothesis never addressed. The two are compatible by construction and independent by content, and the paper does not use one to argue for the other.

The two typings compared. The world typing and the shape are different questions asked of the same row. The typing asks what kind of register the row's barrier literature blocks; the shape asks what kind of cut the row's proved part respects. They correlate without coinciding: the exact-structural rows include height-cut rows, the two rows assigned no cut, and two regime rows; the wall rows are all regime rows; the pure unbridged rows split between height and regime. The correlation is noted and not explained, since explaining it would be a claim about the rows' objects, which this paper makes nowhere.

STANDARD OF PROOF, STANDING, AND DISCLOSURE

The standard

Every claim in this paper is at one of three grades, and the grade is written where the claim is made. Grade K is a theorem of core Lean 4 with no library and no user-declared axiom, its dependency set printed by the kernel and reproduced in the appendices; the paper has forty-three such theorems in the two new files, twenty-seven in `Row_Split.lean` and sixteen in `ZFC_Under_RAM.lean` counted by dependency set, and

thirty-three in the carried file. "Axiom-free" is used in the kernel's sense and reported set by set: no file declares an axiom; every set of the two new files is reported as depending on no axiom at all; in the carried file some sets depend on `propext` and `Quot.sound`, Lean's two built-in logical axioms, and Appendix D lists which. Grade K on data is a kernel theorem about a table the paper supplies, the census and the counts of the carried file; it establishes the arithmetic of the table and nothing about the objects the table names. Grade A is a theorem of the literature, cited to its source and never recomputed here. Grade A joined to K is a documented composition, a theorem whose hypothesis is a grade-A theorem and whose inference is a grade-K theorem: the twenty Real parts, the crossing of the Poincaré row, and the ZFC instance of the placement. It is never a kernel-checked theorem about the cited result, and the paper calls no Real part sealed without the words "on citation." Nothing in the paper is at any other grade, and nothing at grade K depends on anything at grade A.

The title. The title names the theorem and states it in words, and the statement is exact: for every row and every decidable cut, the row is its Real part and its Unicorn part, both defined by the cut and neither by the reader. The title makes no claim about any row's Unicorn part, and the paper makes none.

Standing without the citations

Delete every citation from the paper. Every theorem at grade K stands: the seven theorems of the division with their companions, the small-height Real parts, the gap bridge, the census of the assignments, the sixteen theorems of the placement, and every theorem of the carried seat. Every sentence at grade A joined to K falls, and those are exactly twenty-two: the twenty Real parts of Section 5, the crossing of the Poincaré row, and the ZFC instance of the placement; each reverts to its conditional form, "if the instances inside this cut are checked, then the Real part holds," which is what the kernel proves on its own. Descriptions of what the literature contains, in the positioning and the criterion, are not claims of the paper and are not counted. The standing is tabulated.

Falsifiers

The paper is refuted, at its own standard, by any of three findings, each registered before the audit and none adjusted after it. F1: a decidable cut and a row on which Row holds and $\text{RealIn} \wedge \text{UnicornOut}$ fails, or conversely; this refutes Theorem 1 and is excluded by the kernel. F2: an instance inside any of the twenty cuts that fails its row's property; this refutes the cited certificate or theorem, not the schema, and

Table 2. The world typing of the barrier study, carried as data. Tier: the counts are kernel theorems of Rows_At_The_Apex. lean; the typing is the study's, premise here. The final column is this paper's shape, for comparison.

#	Row	Mill.	World typing	Owed	Router	World verdict	Shape
1	P versus NP	yes	exact-structural	1	Ground dim 0	$[\emptyset_0]$, open	regime
2	Riemann Hypothesis	yes	exact-structural	1	Ground dim 1	$[\Xi_0]$, open	height
3	Navier–Stokes	yes	wall row, F4	1	unmeasured	open, unrouted	regime
4	Yang–Mills	yes	wall row, F4	1	unmeasured	open, unrouted	regime
5	Hodge	yes	wall row, F4	1	unmeasured	open, unrouted	regime
6	BSD	yes	beachhead, order two	1	unmeasured	open, unrouted	regime
7	Poincaré	yes	crossed control	0	unmeasured	crossed	crossed
8	Goldbach	no	exact-structural	1	unmeasured	open, unrouted	height
9	Twin primes	no	exact-structural	1	unmeasured	open, unrouted	height
10	Legendre	no	exact-structural	1	unmeasured	open, unrouted	height
11	abc	no	exact-structural	1	unmeasured	open, unrouted	no cut
12	Jacobian	no	exact-structural	1	unmeasured	open, unrouted	regime
13	Mersenne primes	no	pure unbridged	1	unmeasured	open, unrouted	height
14	Odd perfect numbers	no	pure unbridged	1	unmeasured	open, unrouted	height
15	Smooth 4D Poincaré	no	pure unbridged	1	unmeasured	open, unrouted	regime
16	Collatz	no	diagonal port, F3	1	unmeasured	open, unrouted	height
17	Hadwiger	no	one bit from closure	1	unmeasured	open, unrouted	regime
18	Sunflower	no	one bit from closure	1	unmeasured	open, unrouted	no cut
19	Erdős–Straus	no	one bit from closure	1	unmeasured	open, unrouted	height
20	Beal	no	one bit from closure	1	unmeasured	open, unrouted	height
21	Invariant subspace	no	wall row, bridge gate	1	unmeasured	open, unrouted	regime
22	Hilbert's 16th, second part	no	wall row, bridge gate	1	unmeasured	open, unrouted	regime
23	Lindelöf / Montgomery PCC	no	wall row, bridge gate	1	unmeasured	open, unrouted	regime

Component	Grade	Falls without citations?
The division, Theorems 1–5 with 3b and 3c	K	No
The gap bridge, the small-height Real parts, the standing map	K	No
The shape assignments of Table 1, by the criterion	data, premise	No; a wrong assignment is repaired by re-assignment, F3
The census of shapes	K	No
The placement under RAM	K	No
The seat and the value excepted, carried	K, on an assignment	No
The twenty Real parts	A joined to K	Yes
The Poincaré crossing; the ZFC instance of the placement	A joined to K	Yes; twenty-two sentences in all
The empty-cut and crossed theorems, generic	K	No

would move that row's Real part to a smaller cut. F3: a shape assignment in Table 1 that the criterion of Section 4 does not support, for instance a no-cut row for which an inhabited decidable cut with a proved Real part is exhibited from the literature; this refutes the assignment as data and is repaired by re-assignment, the kernel census recounted.

Disclosure and transparency

The kernel files were written and checked by the author on Lean 4.19.0, core only; the sources cited were not recomputed. The twenty Real parts are the literature's, and the paper's contribution to them is the division under which they are one theorem and the placement under which they are one stratum. The author has no competing interests and received no funding.

The paper is an edition of the prior work on these rows [Islam 2026g], which it carries whole in its kernel file, and it is placed on the same concept record; the barrier study [Islam 2026e] is carried as data. The Riemann paper [Islam 2026f] is the instance from which the division was generalized, and the reader who wants the one-row case in full is sent there.

Audit. The paper was audited by the program's adversarial cycle before release, seat by seat, with the controls and the verdict recorded in the audit log that accompanies the files; the cycle's rounds and verdicts are stated in the log and not summarized here. Before release the paper was also reviewed, in two rounds, by five external language-model reviewers run outside the program, GPT SOL 5.6, Gemini Pro 3.1, Meta AI, Grok 4.6, and GPT 5, on the two internal builds that preceded this one; they are external reviewers and not independent verifiers, they discharged no falsifier and were not asked to, and their findings were answered finding by finding in the log, none conceded without a named mechanism. Three were earned and repaired: the no-cut assignment had been written as if Theorem 5 established it, and is now stated as an assessment with Theorem 5 confined to the empty cut; the phrase “empty Unicorn part” had no set-valued definition, and now has one; and the standing table had graded the Poincaré application as K, and now grades it A joined to K. Five were fortifications taken up because they strengthened the paper: the infinitude equivalence (Theorem 3c), the standing map under which the fusion sentence is read, the gap-to-interval bridge for the Legendre row, the row specifications with computed small-height Real parts, and the criterion of as-

signment with its out-of-sample application. The remaining findings, that the seat is an assignment, that the placement encodes no axiom of ZFC, that the price rests on a postulate, that the census counts assignments, were already the paper’s own statements and are now stated in every place they apply. The second round found the abstract still saying “by theorem” of the rows assigned no cut, the introduction still counting nineteen falling sentences, and the criterion called a decision procedure; all three were repaired, and the reviewers’ question about the instances of the P versus NP row moved that row from no-cut to regime-cut under the criterion, the third correction the criterion made against its author.

POSITIONING AND CONCLUSION

Positioning

The division is not in the literature as a theorem, though its instances are everywhere in it. The convention “verified up to N ” is the height-cut Real part stated without the division, and its users know that it certifies nothing beyond N ; what they do not have is the theorem that the fused problem stands at the tail and the placement of the certificate in a stratum. The arithmetic forms of open problems, Lagarias’s arithmetic equivalent of the Riemann Hypothesis [Lagarias 2002] and the Π_1^0 forms of Goldbach and of the odd-perfect-number problem, make the height cut natural on those rows and are the reason those rows are certificate rows; they do not divide the rows. The literature of finite verification, from Turing’s method [Turing 1953] to the certificates of Platt and Trudgian [2021] and Oliveira e Silva, Herzog and Pardi [2014], supplies the Real parts and states them as verifications, which is what they are. The reverse-mathematics tradition places theorems in subsystems of second-order arithmetic [Simpson 2009]; the placement here is coarser, three strata for an arbitrary theory, and is not a classification of the proved parts by strength. The program’s own barrier study [Islam 2026e] typed the rows by their blocked register and never named a proved part; its prior edition on these rows [Islam 2026g] proved the seat and the value expected and never divided a row; the Riemann paper [Islam 2026f] divided one row at one height. Table 3 sets these beside the present result.

Why a reader should not mistake this for vocabulary. A paper that renamed “verified up to N ” as “the Real part” would have added a word. This paper adds a theorem, that the renaming is exact and the same on every row under any cut, a second, that given the proved part the fused row is exactly the open part, with the rule under which that is a lowering of standing declared and its consequence proved, a third, the placement, a bridge lemma and eight computed

Position	Relation to this paper
“Verified up to N ”	the height-cut Real part without the division or the placement
Arithmetic forms of RH, Goldbach, odd perfect	make the height cut natural on a row; do not divide it
Finite-verification certificates	supply the Real parts; state them as verifications
Reverse mathematics	places theorems by strength; three strata here, by kind
The barrier study	types the blocked register; names no proved part
The prior edition on these rows	proves the seat; divides no row
The Riemann paper	divides one row at one height; here any row at any cut

instances that show the schema instantiated end to end, and a kernel-checked census of the assignments by which the twenty-three rows fall into four shapes with twenty Real parts among them. The theorems are short. Their shortness is the point: the division is a fact about universal statements that the literature has used on every row without stating once, and a statement that is true on every row and had not been made is exactly what a foundations paper is for.

Conclusion

Every open problem is exactly its proved part and its unicorn. That sentence is a theorem for any row under any decidable cut, and it does five things on the twenty-three rows: it names twenty Real parts that the literature has proved and stated as verifications, it seals each through the same schema at grade A joined to K, it says by theorem and a declared rule that fusion with the open remainder is what makes each problem read as open, it says by assessment under a stated criterion that two rows have no proved part and by theorem that one has no counterexample outside any cut, and it places every Real part in a stratum of the grounding reading. It does not prove a row, move a verdict, or author a bound. The unicorns are named on every row so that no reader confuses them with the gold; they are set apart so that no one chases them in this program; and the gold is kept from the alloy so that no one assays it as lead. What remains open on the twenty-three rows is exactly what was open before this paper, stated in one theorem rather than twenty-three idioms, and located: on every open row, the owed bit is the bit of its Unicorn part and of nothing inside the cut.

REFERENCES

- ABC@Home (2015). The enumeration of abc triples with $c < 10^{18}$. Mathematical Institute, Leiden University, abcathome.com.
- Alweiss, R., Lovett, S., Wu, K., and Zhang, J. (2020). Improved bounds for the sunflower lemma. *Proceedings of the 52nd ACM Symposium*

on *Theory of Computing*, 624–630.

- Barina, D. (2020). Convergence verification of the Collatz problem. *The Journal of Supercomputing* 77, 2681–2688 (2021).
- Bass, H., Connell, E. H., and Wright, D. (1982). The Jacobian conjecture: reduction of degree and formal expansion of the inverse. *Bulletin of the American Mathematical Society* 7, 287–330.
- Bourgain, J. (2017). Decoupling, exponential sums and the Riemann zeta function. *Journal of the American Mathematical Society* 30, 205–224.
- Cohen, P. J. (1963). The independence of the continuum hypothesis. *Proceedings of the National Academy of Sciences* 50, 1143–1148.
- Driver, B. K. (1989). YM_2 : continuum expectations, lattice convergence, and lassos. *Communications in Mathematical Physics* 123, 575–616.
- Écalle, J. (1992). *Introduction aux fonctions analysables et preuve constructive de la conjecture de Dulac*. Hermann, Paris.
- Fortnow, L., Lipton, R., van Melkebeek, D., and Viglas, A. (2005). Time-space lower bounds for satisfiability. *Journal of the ACM* 52, 835–865.
- Freedman, M. H. (1982). The topology of four-dimensional manifolds. *Journal of Differential Geometry* 17, 357–453.
- Fujita, H., and Kato, T. (1964). On the Navier-Stokes initial value problem I. *Archive for Rational Mechanics and Analysis* 16, 269–315.
- Gödel, K. (1931). Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme I. *Monatshefte für Mathematik und Physik* 38, 173–198.
- Gödel, K. (1938). The consistency of the axiom of choice and of the generalized continuum-hypothesis. *Proceedings of the National Academy of Sciences* 24, 556–557.
- GIMPS (2024). Mersenne prime number discovery: $2^{136279841} - 1$ is prime. mersenne.org, 21 October 2024.
- Gross, B. H., and Zagier, D. B. (1986). Heegner points and derivatives of L-series. *Inventiones Mathematicae* 84, 225–320.
- Helfgott, H. A. (2013). The ternary Goldbach conjecture is true. arXiv:1312.7748.
- Ilyashenko, Yu. S. (1991). *Finiteness Theorems for Limit Cycles*. Translations of Mathematical Monographs 94, American Mathematical Society.
- Islam, M. F. (2026a). Trisduction Master Codex v4.7.0. Trisduction Research Group. github.com/1000sapient/Trisduction.
- Islam, M. F. (2026e). One Bit Across the Wall: Odd-Supply Separation and the One-Cut Hypothesis Across Twenty-Three Rows, v6. Zenodo. doi:10.5281/zenodo.22746129.
- Islam, M. F. (2026f). A Formal Proof of the Real Part of the Riemann Hypothesis, v2.0.2. Zenodo. doi:10.5281/zenodo.22912937.
- Islam, M. F. (2026g). A Formal Proof Across the Seven Millennium Rows and the Sixteen Extension Rows from Existence Alone, v1.1.0. Zenodo. doi:10.5281/zenodo.22913634.
- Kaye, R. (1991). *Models of Peano Arithmetic*. Oxford University Press.
- Kolyvagin, V. A. (1989). Finiteness of $E(\mathbb{Q})$ and $\square(E, \mathbb{Q})$ for a subclass of Weil curves. *Mathematics of the USSR-Izvestiya* 32, 523–541.
- Ladyzhenskaya, O. A. (1969). *The Mathematical Theory of Viscous Incompressible Flow*. Second edition, Gordon and Breach.
- Lagarias, J. C. (2002). An elementary problem equivalent to the Riemann hypothesis. *American Mathematical Monthly* 109, 534–543.
- Lefschetz, S. (1924). *L'analyse situs et la géométrie algébrique*. Gauthier-Villars, Paris.
- Leray, J. (1934). Sur le mouvement d'un liquide visqueux emplissant l'espace. *Acta Mathematica* 63, 193–248.
- Lévy, T. (2003). Yang–Mills measure on compact surfaces. *Memoirs of the American Mathematical Society* 166, no. 790.
- Lomonosov, V. I. (1973). Invariant subspaces for the family of operators which commute with a completely continuous operator. *Functional Analysis and Its Applications* 7, 213–214.
- Maynard, J. (2015). Small gaps between primes. *Annals of Mathematics* 181, 383–413.
- Miller, R. L. (2011). Proving the Birch and Swinnerton-Dyer conjecture for specific elliptic curves of analytic rank zero and one. *LMS Journal of Computation and Mathematics* 14, 327–350.
- Mossinghoff, M. J., Rhin, G., and Wu, Q. (2008). Minimal Mahler measures. *Experimental Mathematics* 17, 451–458.
- Montgomery, H. L. (1973). The pair correlation of zeros of the zeta function. *Proceedings of Symposia in Pure Mathematics* 24, 181–193.
- Norvig, P. (2017). Beal's Conjecture Revisited. norvig.com/beal.html, revised 4 July 2017.
- Ochem, P., and Rao, M. (2012). Odd perfect numbers are greater than 10^{1500} . *Mathematics of Computation* 81, 1869–1877.
- Oliveira e Silva, T., Herzog, S., and Pardi, S. (2014). Empirical verification of the even Goldbach conjecture and computation of prime gaps up to $4 \cdot 10^{18}$. *Mathematics of Computation* 83, 2033–2060.
- Perelman, G. (2002). The entropy formula for the Ricci flow and its geometric applications. arXiv:math/0211159.
- Perelman, G. (2003a). Ricci flow with surgery on three-manifolds. arXiv:math/0303109.

- Perelman, G. (2003b). Finite extinction time for the solutions to the Ricci flow on certain three-manifolds. *arXiv:math/0307245*.
- Platt, D., and Trudgian, T. (2021). The Riemann hypothesis is true up to $3 \cdot 10^{12}$. *Bulletin of the London Mathematical Society* 53, 792–797.
- Polymath, D. H. J. (2014). Variants of the Selberg sieve, and bounded intervals containing many primes. *Research in the Mathematical Sciences* 1, 12.
- PrimeGrid (2016). World record twin primes: $2996863034895 \cdot 2^{1290000} \pm 1$. primegrid.com, September 2016.
- Robertson, N., Seymour, P., and Thomas, R. (1993). Hadwiger’s conjecture for K_6 -free graphs. *Combinatorica* 13, 279–361.
- Salez, S. E. (2014). The Erdős–Straus conjecture: new modular equations and checking up to 10^{17} . *arXiv:1406.6307*.
- Simpson, S. G. (2009). *Subsystems of Second Order Arithmetic*. Second edition, Cambridge University Press.
- Stewart, C. L., and Yu, K. (2001). On the abc conjecture, II. *Duke Mathematical Journal* 108, 169–181.
- Tao, T. (2022). Almost all orbits of the Collatz map attain almost bounded values. *Forum of Mathematics, Pi* 10, e12.
- Turing, A. M. (1953). Some calculations of the Riemann zeta-function. *Proceedings of the London Mathematical Society* (3) 3, 99–117.
- Wang, S. S.-S. (1980). A Jacobian criterion for separability. *Journal of Algebra* 65, 453–494.
- Williams, R. (2008). Time-space tradeoffs for counting NP solutions modulo integers. *Computational Complexity* 17, 179–219.
- Zhang, Y. (2014). Bounded gaps between primes. *Annals of Mathematics* 179, 1121–1174.

APPENDIX A: ROW_SPLIT.LEAN AND ITS AUDIT

SHA-256 c6177b905fd987741005aac423cc3be769710b979a7a8b31e498b57191cb75ec, Lean 4.19.0, core only, 337 lines; twenty-seven dependency sets, every one reported as depending on no axiom, printed after the file.

```

1  /-
2  Row_Split.lean · the Real/Unicorn division on any row, for any
   ↳ cut; the row specifications of
3  the arithmetic rows; the gap-to-interval bridge; the standing
   ↳ map.
4  Core Lean 4.19.0, no library, no sorry, no user-declared axiom.
   ↳ Forged 2026-09-25, fortified
5  after the two external batches of the same day.
6
7  The Riemann division used the cut "height ≤ T". Nothing in it
   ↳ used the height: the same theorem
8  holds for any decidable predicate on the row's instances, a
   ↳ height, a degree, a dimension, a
9  rank, a box, a regime. So the division of the twenty-three rows
   ↳ is one kernel theorem, applied
10 row by row with the row's own cut. ΔM = 0.
11 -/

```

```

12 namespace RowSplit
13
14 variable {α : Type}
15
16 /-! ## I. The division -/
17
18 /-- A row's value: every instance satisfies the row's property.
   ↳ -/
19 def Row (Inst P : α → Prop) : Prop := ∀ x, Inst x → P x
20 /-- The Real part under a cut: every instance inside the cut
   ↳ satisfies the property. -/
21 def RealIn (Inst P cut : α → Prop) : Prop := ∀ x, Inst x → cut
   ↳ x → P x
22 /-- The Unicorn part under a cut: no instance outside the cut
   ↳ fails the property. -/
23 def UnicornOut (Inst P cut : α → Prop) : Prop := ∀ x, Inst x →
   ↳ ¬ cut x → P x
24
25 /-- THE DIVISION, for any cut: the row is exactly its Real part
   ↳ and its Unicorn part. -/
26 theorem row_iff_real_and_unicorn (Inst P cut : α → Prop)
   ↳ [DecidablePred cut] :
27   Row Inst P ↔ RealIn Inst P cut ∧ UnicornOut Inst P cut :=
   ↳ by
28   constructor
29   · intro h; exact (fun x hx _ => h x hx, fun x hx _ => h x hx)
30   · intro (hr, hu) x hx
31     by_cases hc : cut x
32     · exact hr x hx hc
33     · exact hu x hx hc
34
35 /-- Given the Real part, the fused row is exactly the Unicorn
   ↳ part. -/
36 theorem fused_is_unicorn_given_real (Inst P cut : α → Prop)
   ↳ [DecidablePred cut]
37   (hr : RealIn Inst P cut) : Row Inst P ↔ UnicornOut Inst P
   ↳ cut :=
38   (fun h x hx _ => h x hx, fun hu => (row_iff_real_and_unicorn
   ↳ Inst P cut).2 (hr, hu))
39
40 /-- A certificate for a cut: a complete list of the instances
   ↳ inside it, each checked. -/
41 structure Certificate (Inst P cut : α → Prop) where
42   items : List α
43   complete : ∀ x, Inst x → cut x → x ∈ items
44   checked : ∀ x, x ∈ items → P x
45
46 /-- A certificate proves the Real part of the cut, on any row,
   ↳ with no premise. -/
47 theorem real_of_certificate (Inst P cut : α → Prop) (c :
   ↳ Certificate Inst P cut) :
48   RealIn Inst P cut :=
49   fun x hx hc => c.checked x (c.complete x hx hc)
50
51 /-- On an infinitude row, instances are bounds N and the
   ↳ property is that a witness exceeds N.
52   One exhibited witness w above the height h proves the Real
   ↳ part under the cut N ≤ h. -/
53 theorem real_of_one_witness (W : Nat → Prop) (h w : Nat) (hw :
   ↳ W w) (hwh : h < w) :
54   RealIn (fun _ => True) (fun N => ∃ m, N < m ∧ W m) (fun N
   ↳ => N ≤ h) :=
55   fun _ hN => ⟨w, Nat.lt_of_le_of_lt hN hwh, hw⟩
56
57 /-- On an infinitude row the Unicorn part above any height is
   ↳ equivalent to the whole value: a
58   witness above every bound beyond h is a witness above every
   ↳ bound. So the Real part of an
59   infinitude row is proved by one witness and moves the fused
   ↳ value not at all. -/
60 theorem unicorn_equiv_row_of_infinitude (W : Nat → Prop) (h :
   ↳ Nat) :
61   (∀ N, ¬ N ≤ h → ∃ m, N < m ∧ W m) ↔ (∀ N, ∃ m, N < m ∧ W m)
   ↳ := by
62   constructor
63   · intro hu N
64     if hN : N ≤ h then
65       obtain (m, hm, hw) := hu (h + 1) (fun hle =>
   ↳ Nat.not_succ_le_self h hle)
66       exact (m, Nat.lt_of_le_of_lt hN (Nat.lt_trans
   ↳ (Nat.lt_succ_self h) hm), hw)
67   else
68     exact hu N hN

```

```

69   · intro hr N _; exact hr N
70
71 /-- A crossed row (its value supplied whole) satisfies both
72   ↳ parts under every cut. -/
73 theorem crossed_row_both_parts (Inst P cut :  $\alpha \rightarrow \text{Prop}$ ) (h : Row
74   ↳ Inst P) :
75   UnicornOut Inst P cut  $\wedge$  RealIn Inst P cut :=
76   (fun x hx _ => h x hx, fun x hx _ => h x hx)
77
78 /-- Under the empty cut the Real part is vacuous and the
79   ↳ Unicorn part is the whole value. This
80   theorem is about the empty cut and nothing else; it does
81   ↳ not say that a row admits no other
82   cut, which is an assessment of a row's literature and never
83   ↳ a theorem. -/
84 theorem empty_cut_vacuous (Inst P :  $\alpha \rightarrow \text{Prop}$ ) :
85   RealIn Inst P (fun _ => False)  $\wedge$  (UnicornOut Inst P (fun _
86   ↳ => False)  $\leftrightarrow$  Row Inst P) :=
87   (fun _ _ h => h.elim, (fun hu x hx => hu x hx (fun h => h),
88   ↳ fun h x hx _ => h x hx))
89
90 /-! ## II. Substantive Real parts, and the Unicorn part as a
91   ↳ set of counterexamples -/
92
93 /-- A cut is inhabited on the row when some instance lies
94   ↳ inside it. -/
95 def Inhabited (Inst cut :  $\alpha \rightarrow \text{Prop}$ ) : Prop :=  $\exists x$ , Inst x  $\wedge$  cut
96   ↳ x
97 /-- A substantive Real part: an inhabited cut with its Real
98   ↳ part. The empty cut never has one. -/
99 def SubstantiveRealIn (Inst P cut :  $\alpha \rightarrow \text{Prop}$ ) : Prop :=
100   ↳ Inhabited Inst cut  $\wedge$  RealIn Inst P cut
101
102 theorem empty_cut_not_substantive (Inst P :  $\alpha \rightarrow \text{Prop}$ ) :
103   ↳ SubstantiveRealIn Inst P (fun _ => False) :=
104   fun (l, _, h), _ => h
105
106 /-- A counterexample outside the cut: an instance outside it
107   ↳ that fails the property. The
108   Unicorn part, read as a set, is the set of these; the
109   ↳ Unicorn proposition says it is empty. -/
110 def CounterOut (Inst P cut :  $\alpha \rightarrow \text{Prop}$ ) (x :  $\alpha$ ) : Prop := Inst x
111   ↳  $\wedge$   $\neg$  cut x  $\wedge$   $\neg$  P x
112
113 theorem no_counter_of_unicorn (Inst P cut :  $\alpha \rightarrow \text{Prop}$ ) (hu :
114   ↳ UnicornOut Inst P cut) :
115    $\forall x$ ,  $\neg$  CounterOut Inst P cut x :=
116   fun x (hx, hc, hp) => hp (hu x hx hc)
117
118 theorem unicorn_of_no_counter (Inst P cut :  $\alpha \rightarrow \text{Prop}$ )
119   ↳ [DecidablePred P]
120   (h :  $\forall x$ ,  $\neg$  CounterOut Inst P cut x) : UnicornOut Inst P
121   ↳ cut := by
122   intro x hx hc
123   by_cases hp : P x
124   · exact hp
125   · exact absurd (hx, hc, hp) (h x)
126
127 /-- A crossed row has no counterexample outside any cut: its
128   ↳ Unicorn set is empty. -/
129 theorem crossed_row_no_counter (Inst P cut :  $\alpha \rightarrow \text{Prop}$ ) (h : Row
130   ↳ Inst P) :
131    $\forall x$ ,  $\neg$  CounterOut Inst P cut x :=
132   no_counter_of_unicorn Inst P cut (crossed_row_both_parts Inst
133   ↳ P cut h).1
134
135 /-! ## III. The standing map: fusion lowers a proved part to
136   ↳ open, as a declared rule -/
137
138 /-- Two standings: proved, or open. -/
139 inductive Standing | proved | open
140   deriving DecidableEq, Repr
141
142 /-- The standing of a conjunction is the weaker standing of its
143   ↳ conjuncts: the program's grade
144   law, declared here as the rule the fusion sentence is read
145   ↳ under. -/
146 def Standing.join : Standing  $\rightarrow$  Standing  $\rightarrow$  Standing
147   | .proved, .proved => .proved
148   | _, _ => .open
149
150 /-- Under the declared rule, a proved part joined to an open
151   ↳ part stands open; only two proved
152   parts stand proved. The theorem records the rule; the
153   ↳ lowering is the rule's, not the kernel's. -/
154 theorem fusion_lowers : Standing.join .proved .open = .open  $\wedge$ 
155   ↳ Standing.join .open .proved = .open  $\wedge$ 
156   Standing.join .proved .proved = .proved := (rfl, rfl, rfl)
157
158 /-! ## IV. The gap-to-interval bridge, for the Legendre row -/
159
160 /-- a - q < a - p when p < q and p < a; proved by induction to
161   ↳ keep the file free of propext. -/
162 theorem sub_lt_sub_of_lt :  $\forall$  (a p q : Nat), p < a  $\rightarrow$  p < q  $\rightarrow$  a -
163   ↳ q < a - p
164   | 0, _, _, h, _ => absurd h (Nat.not_lt_zero _)
165   | a + 1, 0, 0, _, hpq => absurd hpq (Nat.lt_irrefl 0)
166   | a + 1, 0, q + 1, _, _ => by
167     rw [Nat.succ_sub_succ, Nat.sub_zero]
168     exact Nat.lt_succ_of_le (Nat.sub_le a q)
169   | _ + 1, p + 1, 0, _, hpq => absurd hpq (Nat.not_lt_zero _)
170   | a + 1, p + 1, q + 1, hpa, hpq => by
171     rw [Nat.succ_sub_succ, Nat.succ_sub_succ]
172     exact sub_lt_sub_of_lt a p q (Nat.lt_of_succ_lt_succ hpa)
173     ↳ (Nat.lt_of_succ_lt_succ hpq)
174
175 /-- From a bound g on the gap after every element of Q up to X,
176   ↳ and an element of Q at or below
177   a, an element of Q lies in (a, a + g], provided a  $\leq$  X. Q is
178   ↳ any predicate; the primes are the
179   instance. Recursion on a - p. -/
180 theorem elem_in_gap (Q : Nat  $\rightarrow$  Prop) (g X : Nat)
181   (hgap :  $\forall p$ , Q p  $\rightarrow$  p  $\leq$  X  $\rightarrow$   $\exists q$ , Q q  $\wedge$  p < q  $\wedge$  q  $\leq$  p + g)
182   (a : Nat) (ha : a  $\leq$  X) (p : Nat) (hp : Q p) (hpa : p  $\leq$  a) :
183    $\exists q$ , Q q  $\wedge$  a < q  $\wedge$  q  $\leq$  a + g := by
184   obtain (q, hq, hpq, hqg) := hgap p hp (Nat.le_trans hpa ha)
185   if hqa : a < q then
186     exact (q, hq, hqa, Nat.le_trans hqg (Nat.add_le_add_right
187     ↳ hpa g))
188   else
189     have hqa' : q  $\leq$  a := Nat.le_of_not_lt hqa
190     have hpa' : p < a := Nat.lt_of_lt_of_le hpq hqa'
191     exact elem_in_gap Q g X hgap a ha q hq hqa'
192   termination_by a - p
193   decreasing_by exact sub_lt_sub_of_lt a p q hpa' hpq
194
195 theorem sq_succ (n : Nat) : (n + 1) * (n + 1) = n * n + 2 * n +
196   ↳ 1 := by
197   rw [Nat.mul_succ, Nat.succ_mul, Nat.two_mul, Nat.add_assoc (n
198   ↳ * n) n (n + 1),
199     ← Nat.add_assoc n n 1, ← Nat.add_assoc (n * n) (n + n) 1]
200
201 /-- THE LEGENDRE BRIDGE: if every gap between consecutive
202   ↳ elements of Q up to X is at most 1476,
203   Q holds at 2, n  $\geq$  738, and (n+1)2  $\leq$  X, then Q has an
204   ↳ element strictly between n2 and (n+1)2.
205   The gap bound is the literature's [Oliveira e Silva, Herzog
206   ↳ and Pardi 2014] with Q the primes
207   and X = 4·1018; the bridge is the kernel's. -/
208 theorem legendre_of_gap (Q : Nat  $\rightarrow$  Prop) (X : Nat)
209   (hgap :  $\forall p$ , Q p  $\rightarrow$  p  $\leq$  X  $\rightarrow$   $\exists q$ , Q q  $\wedge$  p < q  $\wedge$  q  $\leq$  p + 1476)
210   (h2 : Q 2) (n : Nat) (hn : 738  $\leq$  n) (hX : (n + 1) * (n + 1)
211   ↳  $\leq$  X) :
212    $\exists q$ , Q q  $\wedge$  n * n < q  $\wedge$  q < (n + 1) * (n + 1) := by
213   have hsq := sq_succ n
214   have h738 : 738  $\leq$  n * n := Nat.le_trans hn
215   ↳ (Nat.le_mul_of_pos_right n (Nat.lt_of_lt_of_le (by
216   ↳ decide) hn))
217   have hnn : 2  $\leq$  n * n := Nat.le_trans (by decide) h738
218   have ha : n * n  $\leq$  X := Nat.le_trans (Nat.mul_le_mul
219   ↳ (Nat.le_succ n) (Nat.le_succ n)) hX
220   obtain (q, hq, hlt, hle) := elem_in_gap Q 1476 X hgap (n * n)
221   ↳ ha 2 h2 hnn
222   have hgapn : 1476 < 2 * n + 1 :=
223     Nat.lt_of_lt_of_le (by decide) (Nat.add_le_add_right
224     ↳ (Nat.mul_le_mul_left 2 hn) 1)
225   have hq2 : q < (n + 1) * (n + 1) := by
226     rw [hsq, Nat.add_assoc]
227     exact Nat.lt_of_le_of_lt hle (Nat.add_lt_add_left hgapn (n
228     ↳ * n))
229   exact (q, hq, hlt, hq2)
230
231 /-! ## V. Row specifications: the exact tuple ( $\alpha$ , Inst, P, cut)
232   ↳ of the arithmetic height-cut rows,
233   with primality and the properties defined in core Lean, and
234   ↳ the Real part at a small height

```

```

188   computed in the kernel. The literature's heights are cited
189   ↪ in the paper; none is recomputed
190   here. -/
191 /-- Trial-division primality, decidable. -/
192 def isPrime (n : Nat) : Bool := 2 ≤ n && (List.range n).all
193   ↪ (fun d => d < 2 || n % d != 0)
194 /-- Goldbach at n: n is even and at least 4 implies n is a sum
195   ↪ of two primes. -/
196 def goldbachOK (n : Nat) : Bool :=
197   ↪ n < 4 || n % 2 == 1 || (List.range (n + 1)).any (fun p =>
198     ↪ isPrime p && isPrime (n - p))
199 /-- Legendre at n: a prime strictly between n² and (n+1)². -/
200 def legendreOK (n : Nat) : Bool :=
201   ↪ n == 0 || (List.range ((n + 1) * (n + 1))).any (fun q => n *
202     ↪ n < q && isPrime q)
203 /-- Odd perfect at n: n is not an odd perfect number. -/
204 def sigma (n : Nat) : Nat := ((List.range (n + 1)).filter (fun
205   ↪ d => d != 0 && n % d == 0)).foldl (· + ·) 0
206 def oddPerfectOK (n : Nat) : Bool := n == 0 || n % 2 == 0 ||
207   ↪ sigma n != 2 * n
208 /-- Collatz at n: the orbit reaches 1 within a step budget of
209   ↪ 200. -/
210 def collatzStep (n : Nat) : Nat := if n % 2 == 0 then n / 2
211   ↪ else 3 * n + 1
212 def collatzReaches : Nat → Nat → Bool
213   ↪ | _, 0 => false
214   ↪ | n, fuel + 1 => n == 1 || collatzReaches (collatzStep n)
215   ↪ fuel
216 def collatzOK (n : Nat) : Bool := n == 0 || collatzReaches n
217   ↪ 200
218 /-- Erdős–Straus at n: 4/n = 1/x + 1/y + 1/z with x ≤ y ≤ z ≤
219   ↪ n², searched. -/
220 def erdosStrausOK (n : Nat) : Bool :=
221   ↪ n < 2 || (List.range (n * n + 1)).any (fun x => x != 0 &&
222     ↪ (List.range (n * n + 1)).any (fun y => x ≤ y &&
223       ↪ (List.range (n * n + 1)).any (fun z => y ≤ z && 4 * x * y
224         ↪ ↪ z == n * (y * z + x * z + x * y))))
225 /-- Beal on the box (A, B ≤ a; x, y, z ≤ e): no coprime
226   ↪ solution of A^x + B^y = C^z. -/
227 def gcdF : Nat → Nat → Nat → Nat
228   ↪ | 0, a, _ => a
229   ↪ | fuel + 1, a, b => if b == 0 then a else gcdF fuel b (a % b)
230 def gcd' (a b : Nat) : Nat := gcdF (a + b + 1) a b
231 def bealOK (a e : Nat) : Bool :=
232   ↪ (List.range (a + 1)).all (fun A => (List.range (a + 1)).all
233     ↪ (fun B =>
234       ↪ A == 0 || B == 0 || gcd' A B != 1 ||
235       ↪ (List.range (e + 1)).all (fun x => (List.range (e + 1)).all
236         ↪ (fun y =>
237           ↪ x < 3 || y < 3 ||
238           ↪ (List.range (A ^ x + B ^ y + 1)).all (fun C =>
239             ↪ C < 2 || (List.range (e + 1)).all (fun z => z < 3 || C
240               ↪ ↪ ^ z != A ^ x + B ^ y))))))
241 /-- A bounded universal computed as a structural check is the
242   ↪ Real part under the height cut. -/
243 def allUpTo (f : Nat → Bool) : Nat → Bool
244   ↪ | 0 => f 0
245   ↪ | h + 1 => f (h + 1) && allUpTo f h
246 theorem allUpTo_spec (f : Nat → Bool) : ∀ h, allUpTo f h = true
247   ↪ → ∀ n, n ≤ h → f n = true
248 | 0, hall, n, hn => by
249   have h0 : n = 0 := Nat.eq_zero_of_le_zero hn
250   subst h0; exact hall
251 | h + 1, hall, n, hn => by
252   have hsplit : f (h + 1) = true ∧ allUpTo f h = true := by
253     have hall' : (f (h + 1) && allUpTo f h) = true := hall
254     revert hall'
255     generalize f (h + 1) = a
256     generalize allUpTo f h = b
257     cases a <|> cases b <|> intro hab <|>
258       first | exact (rfl, rfl) | exact Bool.noConfusion hab
259     cases Nat.lt_or_eq_of_le hn with
260     | inl hlt => exact allUpTo_spec f h hsplit.2 n
261     ↪ (Nat.le_of_lt_succ hlt)
262     | inr heq => subst heq; exact hsplit.1
263 theorem real_of_all_range (f : Nat → Bool) (h : Nat) (hall :
264   ↪ allUpTo f h = true) :
265   RealIn (fun _ => True) (fun n => f n = true) (fun n => n ≤
266     ↪ h) :=
267   fun n _ hn => allUpTo_spec f h hall n hn
268 set_option maxRecDepth 100000 in
269 theorem goldbach_real_60 : RealIn (fun _ => True) (fun n =>
270   ↪ goldbachOK n = true) (fun n => n ≤ 60) :=
271   real_of_all_range goldbachOK 60 (by decide)
272 set_option maxRecDepth 100000 in
273 theorem legendre_real_8 : RealIn (fun _ => True) (fun n =>
274   ↪ legendreOK n = true) (fun n => n ≤ 8) :=
275   real_of_all_range legendreOK 8 (by decide)
276 set_option maxRecDepth 100000 in
277 theorem oddperfect_real_60 : RealIn (fun _ => True) (fun n =>
278   ↪ oddPerfectOK n = true) (fun n => n ≤ 60) :=
279   real_of_all_range oddPerfectOK 60 (by decide)
280 set_option maxRecDepth 100000 in
281 theorem collatz_real_60 : RealIn (fun _ => True) (fun n =>
282   ↪ collatzOK n = true) (fun n => n ≤ 60) :=
283   real_of_all_range collatzOK 60 (by decide)
284 set_option maxRecDepth 100000 in
285 theorem erdosstrauss_real_6 : RealIn (fun _ => True) (fun n =>
286   ↪ erdosStrausOK n = true) (fun n => n ≤ 6) :=
287   real_of_all_range erdosStrausOK 6 (by decide)
288 set_option maxRecDepth 100000 in
289 theorem beal_box_3_4 : bealOK 3 4 = true := by decide
290 set_option maxRecDepth 100000 in
291 theorem twin_real_100 : RealIn (fun _ => True) (fun N => ∃ m, N
292   ↪ < m ∧ (isPrime m && isPrime (m + 2)) = true) (fun N => N ≤
293     ↪ 100) :=
294   real_of_one_witness (fun m => (isPrime m && isPrime (m + 2))
295     ↪ = true) 100 101 (by decide) (by decide)
296 set_option maxRecDepth 100000 in
297 def isMersenne (q : Nat) : Bool := (List.range 20).any (fun m
298   ↪ => 2 ^ m - 1 == q)
299 theorem mersenne_real_100 : RealIn (fun _ => True) (fun N => ∃
300   ↪ q, N < q ∧ (isPrime q && isMersenne q) = true) (fun N => N
301     ↪ ≤ 100) :=
302   real_of_one_witness (fun q => (isPrime q && isMersenne q) =
303     ↪ true) 100 127 (by decide) (by decide)
304 /-! ## VI. The twenty-three rows, typed by shape, the census of
305   ↪ the assignments kernel-checked -/
306 /-- The shape of a row's cut, as assigned: a height (a bounded
307   ↪ universal with a certificate
308   ↪ per height, or an infinitude with one witness), a regime (a
309     ↪ dimension, degree, rank, or class
310     ↪ proved by a cited theorem), no cut (no inhabited cut with a
311       ↪ Real part in the literature, an
312       ↪ assessment), or crossed (the value supplied whole). The
313         ↪ assignments are data; the kernel
314         ↪ checks their census. -/
315 inductive Shape | height | regime | noCut | crossed
316   deriving DecidableEq, Repr
317 def rows23 : List (String × Shape) :=
318   [ ("P versus NP", .regime), ("Riemann Hypothesis", .height),
319     ↪ ("Navier–Stokes", .regime),
320     ↪ ("Yang–Mills", .regime), ("Hodge", .regime), ("BSD",
321       ↪ .regime), ("Poincaré", .crossed),
322     ↪ ("Goldbach", .height), ("Twin primes", .height),
323     ↪ ("Legendre", .height), ("abc", .noCut),
324     ↪ ("Jacobian", .regime), ("Mersenne primes", .height), ("Odd
325       ↪ perfect numbers", .height),
326     ↪ ("Smooth 4D Poincaré", .regime), ("Collatz", .height),
327     ↪ ("Hadwiger", .regime),
328     ↪ ("Sunflower", .noCut), ("Erdős–Straus", .height), ("Beal",
329       ↪ .height),
330     ↪ ("Invariant subspace", .regime), ("Hilbert's 16th, second
331       ↪ part", .regime),
332     ↪ ("Lindelöf / Montgomery PCC", .regime)]
333 def countShape (s : Shape) : Nat := (rows23.filter (fun r =>
334   ↪ r.2 == s)).length
335 /-- THE CENSUS OF THE ASSIGNMENTS: twenty-three rows; nine
336   ↪ height-cut, eleven regime-cut, two
337   ↪ no-cut, one crossed; twenty assigned a Real-part-bearing
338     ↪ shape. -/
339 theorem census :
340   rows23.length = 23 ∧ countShape .height = 9 ∧ countShape
341     ↪ .regime = 11 ∧
342   countShape .noCut = 2 ∧ countShape .crossed = 1 ∧
343   countShape .height + countShape .regime = 20 := by decide

```

```

305
306 def hasRealPart (s : Shape) : Bool := s == .height || s ==
    ↪ .regime
307 theorem real_part_rows : (rows23.filter (fun r => hasRealPart
    ↪ r.2)).length = 20 := by decide
308
309 end RowSplit
310
311 #print axioms RowSplit.row_iff_real_and_unicorn
312 #print axioms RowSplit.fused_is_unicorn_given_real
313 #print axioms RowSplit.real_of_certificate
314 #print axioms RowSplit.real_of_one_witness
315 #print axioms RowSplit.unicorn_equiv_row_of_infinity
316 #print axioms RowSplit.crossed_row_both_parts
317 #print axioms RowSplit.empty_cut_vacuuous
318 #print axioms RowSplit.empty_cut_not_substantive
319 #print axioms RowSplit.no_counter_of_unicorn
320 #print axioms RowSplit.unicorn_of_no_counter
321 #print axioms RowSplit.crossed_row_no_counter
322 #print axioms RowSplit.fusion_lowfers
323 #print axioms RowSplit.sub_lt_sub_of_lt
324 #print axioms RowSplit.elem_in_gap
325 #print axioms RowSplit.legendre_of_gap
326 #print axioms RowSplit.allUpTo_spec
327 #print axioms RowSplit.real_of_all_range
328 #print axioms RowSplit.goldbach_real_60
329 #print axioms RowSplit.legendre_real_8
330 #print axioms RowSplit.oddperfect_real_60
331 #print axioms RowSplit.collatz_real_60
332 #print axioms RowSplit.erdosstrauss_real_6
333 #print axioms RowSplit.beal_box_3_4
334 #print axioms RowSplit.twin_real_100
335 #print axioms RowSplit.mersenne_real_100
336 #print axioms RowSplit.census
337 #print axioms RowSplit.real_part_rows

```

Kernel audit, verbatim: lean Row_Split.lean, exit 0, no warnings.

```

1 'RowSplit.row_iff_real_and_unicorn' does not depend on any
  ↪ axioms
2 'RowSplit.fused_is_unicorn_given_real' does not depend on any
  ↪ axioms
3 'RowSplit.real_of_certificate' does not depend on any axioms
4 'RowSplit.real_of_one_witness' does not depend on any axioms
5 'RowSplit.unicorn_equiv_row_of_infinity' does not depend on
  ↪ any axioms
6 'RowSplit.crossed_row_both_parts' does not depend on any axioms
7 'RowSplit.empty_cut_vacuuous' does not depend on any axioms
8 'RowSplit.empty_cut_not_substantive' does not depend on any
  ↪ axioms
9 'RowSplit.no_counter_of_unicorn' does not depend on any axioms
10 'RowSplit.unicorn_of_no_counter' does not depend on any axioms
11 'RowSplit.crossed_row_no_counter' does not depend on any axioms
12 'RowSplit.fusion_lowfers' does not depend on any axioms
13 'RowSplit.sub_lt_sub_of_lt' does not depend on any axioms
14 'RowSplit.elem_in_gap' does not depend on any axioms
15 'RowSplit.legendre_of_gap' does not depend on any axioms
16 'RowSplit.allUpTo_spec' does not depend on any axioms
17 'RowSplit.real_of_all_range' does not depend on any axioms
18 'RowSplit.goldbach_real_60' does not depend on any axioms
19 'RowSplit.legendre_real_8' does not depend on any axioms
20 'RowSplit.oddperfect_real_60' does not depend on any axioms
21 'RowSplit.collatz_real_60' does not depend on any axioms
22 'RowSplit.erdosstrauss_real_6' does not depend on any axioms
23 'RowSplit.beal_box_3_4' does not depend on any axioms
24 'RowSplit.twin_real_100' does not depend on any axioms
25 'RowSplit.mersenne_real_100' does not depend on any axioms
26 'RowSplit.census' does not depend on any axioms
27 'RowSplit.real_part_rows' does not depend on any axioms

```

APPENDIX B: ZFC_UNDER_RAM.LEAN AND ITS AUDIT

SHA-256 963e3ed8f630975f8604fddc010f154fd4f75f5ae6e193c95f64d4486089a741, Lean 4.19.0, core only,

161 lines; sixteen dependency sets, every one axiom-free; seated in the Master Codex as APEX-PSP-ZFC-UNDER-RAM-01. The file encodes no axiom of ZFC; its theory is abstract and ZFC is the named instance.

```

1 /-
2 ZFC_Under_RAM.lean · ZFC subsumed under RAM by placement, the
  ↪ RA-RAM bridge, and the round trip,
3 in one file. Core Lean 4.19.0, no library, no sorry, no
  ↪ user-declared axiom. Forged 2026-09-25 from
4 RAM_ZFC_Placement.lean, RA_to_ZFC_Chain.lean, and
  ↪ ZFC_RA_RoundTrip.lean (Lean/, 2026-09-24).
5
6 The claim, exactly. RAM does not derive ZFC's axioms and does
  ↪ not replace ZFC's proofs. It LOCATES
7 ZFC: computation is the rung (L3m), provability the ladder
  ↪ (L2m), truth in the intended world the
8 Ground (L1m), and rung  $\leq$  ladder  $\leq$  Ground under two named
  ↪ premises,  $\Sigma_1$ -completeness (a theorem of
9 arithmetic, cited) and soundness (which no consistent theory
  ↪ proves of itself, carried openly).
10 Across the RA-RAM bridge the root reaches every world of ZFC as
  ↪ presence and carries no keyed
11 sentence; the round trip returns every sentence unchanged; and
  ↪ every step holds with the root
12 replaced by a bare computation arrow, so the root contributes
  ↪ the act and never a line of content.
13  $\Delta M = 0$ .
14 -/
15 namespace ZFCUnderRAM
16
17 /-! ## 1 · The placement: rung  $\leq$  ladder  $\leq$  Ground -/
18
19 structure Theory where
20   Sent : Type
21   Prov : Sent → Prop      -- L2m, the ladder
22   Comp : Sent → Prop      -- L3m, the rung: sentences
  ↪   settled by finite computation
23   True_ : Sent → Prop     -- L1m, the Ground: truth in the
  ↪   intended world
24
25 /-  $\Sigma_1$ -completeness, abstractly: what finite computation
  ↪ settles, the theory proves. -/
26 def RungOnLadder (T : Theory) : Prop :=  $\forall s, T.Comp\ s \rightarrow T.Prov\ s$ 
27 /- Soundness: what the theory proves holds in its intended
  ↪ world. Carried openly, never derived. -/
28 def Sound (T : Theory) : Prop :=  $\forall s, T.Prov\ s \rightarrow T.True\_s$ 
29
30 /- THE PLACEMENT. Under the two premises every theory sits in
  ↪ the strata in order. -/
31 theorem ram_places_theory (T : Theory) (hR : RungOnLadder T)
  ↪ (hS : Sound T) :
32   ( $\forall s, T.Comp\ s \rightarrow T.Prov\ s$ )  $\wedge$  ( $\forall s, T.Prov\ s \rightarrow T.True\_s$ )  $\wedge$ 
  ↪ ( $\forall s, T.Comp\ s \rightarrow T.True\_s$ ) :=
33   (hR, hS, fun s h => hS s (hR s h))
34
35 /- Soundness is load-bearing: an unsound theory breaks ladder
  ↪  $\leq$  Ground. -/
36 def unsound : Theory := {Bool, fun _ => True, fun _ => False,
  ↪ fun b => b = true}
37 theorem soundness_is_load_bearing : RungOnLadder unsound  $\wedge$   $\neg$ 
  ↪ Sound unsound := by
38   refine {fun _ h => h.elim, fun h => ?_}
39   have := h false trivial
40   cases this
41
42 /- The Ground exceeds the ladder: a sound theory can leave a
  ↪ truth unproved (Gödel's region). -/
43 def incomplete : Theory := {Bool, fun b => b = true, fun _ =>
  ↪ False, fun _ => True}
44 theorem ground_exceeds_ladder :
45   Sound incomplete  $\wedge$  incomplete.True_false  $\wedge$   $\neg$ 
  ↪ incomplete.Prov_false :=
46   (fun _ => trivial, trivial, fun h => Bool.noConfusion h)
47
48 /- Independent axioms are separate bits: every combination of
  ↪ their values is realized. -/
49 theorem independent_axioms_are_separate_bits :
50    $\forall a\ b : Bool, \exists w : Bool \times Bool, w.1 = a \wedge w.2 = b :=$ 
51   fun a b => ((a, b), rfl, rfl)

```

```

52
53 /-- SUBSUMPTION BY PLACEMENT, the verdict of section 1. -/
54 theorem ram_subsumes_by_placement :
55   (∀ T : Theory, RungOnLadder T → Sound T → (∀ s, T.Comp s →
56     → T.True_ s)) ∧
57   (RungOnLadder unsound ∧ ¬ Sound unsound) ∧
58   (Sound incomplete ∧ incomplete.True_ false ∧ ¬
59     → incomplete.Prov false) :=
60   (fun T hR hS => (ram_places_theory T hR hS).2.2,
61     → soundness_is_load_bearing, ground_exceeds_ladder)
62
63 /-- ## 2 · The bridge: the root reaches every world as presence
64   → and carries no key -/
65
66 structure Substrate where
67   U : Type
68   ΔE : U → Int
69
70 def RA (S : Substrate) : Prop := ∀ x : S.U, 0 < S.ΔE x
71 def unitSub : Substrate := (Unit, fun _ => 1)
72 theorem ra_unit : RA unitSub := fun _ => show (0 : Int) < 1 by
73   → decide
74
75 structure SelfGrounding (R : Prop) where
76   Act : Type
77   anAct : Act
78   instances : Act → R
79
80 /-- Upward: every act of proving in any theory grounds the
81   → root. -/
82 theorem using_grounds_root (T : Theory) (s : T.Sent) (d :
83   → T.Prov s) :
84   Nonempty (SelfGrounding (RA unitSub)) :=
85   ((PLift (T.Prov s), (d), fun _ => ra_unit))
86
87 /-- Worlds are ways a theory's content could be. A property is
88   → keyless when every world has it,
89   → keyed when some world lacks it. The bridge carries the
90   → keyless and never the keyed. -/
91 def Keyless {W : Type} (P : W → Prop) : Prop := ∀ w, P w
92 def Keyed {W : Type} (P : W → Prop) : Prop := ∃ w, ¬ P w
93
94 theorem ra_is_keyless {W : Type} : Keyless (fun _ : W => RA
95   → unitSub) := fun _ => ra_unit
96 theorem keyless_crosses {W : Type} (P : W → Prop) (hP : Keyless
97   → P) : ∀ w, RA unitSub → P w :=
98   fun w _ => hP w
99 theorem ra_decides_no_keyed {W : Type} (P : W → Prop) (hK :
100   → Keyed P) :
101   ¬ (∀ w, RA unitSub → P w) :=
102   fun h => let (w, hw) := hK; hw (h w ra_unit)
103
104 /-- Choice over ZF, the cited keyed sentence (Gödel 1938, Cohen
105   → 1963): two worlds, one each way;
106   → the root holds in both and so carries neither Choice nor
107   → its denial into ZFC. -/
108 def choiceHolds : Bool → Prop := fun w => w = true
109 theorem choice_is_keyed : Keyed choiceHolds := (false, fun h =>
110   → Bool.noConfusion h)
111 theorem ra_does_not_cross_choice :
112   (∀ w : Bool, RA unitSub) ∧ choiceHolds true ∧ ¬
113     → choiceHolds false :=
114   (fun _ => ra_unit, rfl, fun h => Bool.noConfusion h)
115
116 /-- ## 3 · The round trip: ZFC → RAM → bridge → RA → bridge →
117   → RAM → ZFC is the identity on content -/
118
119 def viaRA {W : Type} (P : W → Prop) : W → Prop := fun w => RA
120   → unitSub ∧ P w
121
122 theorem round_trip_identity {W : Type} (P : W → Prop) (w : W) :
123   → viaRA P w ↔ P w :=
124   (fun h => h.2, fun h => {ra_unit, h})
125
126 /-- An absolute sentence (the same value in every world, as an
127   → arithmetic sentence such as the
128   → hypothesis is across models with the standard numbers)
129   → returns with that same value. -/
130 theorem absolute_returns_unchanged {W : Type} (P : W → Prop)
131   → (hAbs : ∀ w v, P w ↔ P v) (w : W) :
132   ∀ v, viaRA P v ↔ P v :=
133   fun v => (round_trip_identity P v).trans (hAbs v w)
134
135 /-- The trip adds presence and nothing else: the root returns
136   → identically for a sentence and for

```

```

137   its negation, so it tells them apart nowhere. -/
138 theorem trip_adds_presence_only {W : Type} (P : W → Prop) (w :
139   → W) :
140   RA unitSub ∧ (viaRA P w ↔ P w) ∧ (viaRA (fun v => ¬ P v) w
141     → ↔ ¬ P w) :=
142   {ra_unit, round_trip_identity P w, round_trip_identity (fun v
143     → => ¬ P v) w}
144
145 /-- ## 4 · The arrow: any true premise plays the root's part,
146   → so the root is idle in the content -/
147
148 theorem any_true_premise_serves (R : Prop) (hR : R) {W : Type}
149   → (P : W → Prop) :
150   ((∀ w, P w) → ∀ w, R → P w) ∧ ((∃ w, ¬ P w) → ¬ ∀ w, R → P
151     → w) ∧ (∀ w, (R ∧ P w) ↔ P w) :=
152   (fun hP w _ => hP w, fun (w, hw) h => hw (h w hR), fun _ =>
153     → (fun h => h.2, fun h => {hR, h}))
154
155 /-- ## 5 · The verdict, in one theorem -/
156
157 /-- ZFC UNDER RAM. Placement under the two named premises with
158   → soundness load-bearing and the
159   → Ground exceeding the ladder; the root grounded by every act
160   → of proof and crossing as presence;
161   → no keyed sentence decided, Choice the instance; the round
162   → trip the identity on content, adding
163   → presence only; and the whole holding for any true premise
164   → in the root's place. Subsumption is
165   → location, never derivation. -/
166 theorem zfc_under_ram :
167   (∀ T : Theory, RungOnLadder T → Sound T → (∀ s, T.Comp s →
168     → T.True_ s)) ∧
169   (RungOnLadder unsound ∧ ¬ Sound unsound) ∧
170   (Sound incomplete ∧ incomplete.True_ false ∧ ¬
171     → incomplete.Prov false) ∧
172   (∀ (T : Theory) (s : T.Sent), T.Prov s → Nonempty
173     → (SelfGrounding (RA unitSub))) ∧
174   (∀ {W : Type} (P : W → Prop), Keyed P → ¬ ∀ w, RA unitSub →
175     → P w) ∧
176   ((∀ w : Bool, RA unitSub) ∧ choiceHolds true ∧ ¬
177     → choiceHolds false) ∧
178   (∀ {W : Type} (P : W → Prop) (w : W), viaRA P w ↔ P w) ∧
179   (∀ (R : Prop), R → ∀ {W : Type} (P : W → Prop) (w : W), (R
180     → ∧ P w) ↔ P w) :=
181   {ram_subsumes_by_placement, using_grounds_root, fun P hK =>
182     → ra_decides_no_keyed P hK,
183     ra_does_not_cross_choice, fun P w => round_trip_identity P
184       → w,
185     fun R hR {W} P w => (any_true_premise_serves R hR (W := W)
186       → P).2.2 w}
187
188 end ZFCUnderRAM
189
190 #print axioms ZFCUnderRAM.ram_places_theory
191 #print axioms ZFCUnderRAM.soundness_is_load_bearing
192 #print axioms ZFCUnderRAM.ground_exceeds_ladder
193 #print axioms ZFCUnderRAM.independent_axioms_are_separate_bits
194 #print axioms ZFCUnderRAM.ram_subsumes_by_placement
195 #print axioms ZFCUnderRAM.using_grounds_root
196 #print axioms ZFCUnderRAM.ra_is_keyless
197 #print axioms ZFCUnderRAM.keyless_crosses
198 #print axioms ZFCUnderRAM.ra_decides_no_keyed
199 #print axioms ZFCUnderRAM.choice_is_keyed
200 #print axioms ZFCUnderRAM.ra_does_not_cross_choice
201 #print axioms ZFCUnderRAM.round_trip_identity
202 #print axioms ZFCUnderRAM.absolute_returns_unchanged
203 #print axioms ZFCUnderRAM.trip_adds_presence_only
204 #print axioms ZFCUnderRAM.any_true_premise_serves
205 #print axioms ZFCUnderRAM.zfc_under_ram

```

Kernel audit, verbatim: lean ZFC_Under_RAM.lean,
exit 0.

- 1 'ZFCUnderRAM.ram_places_theory' does not depend on any axioms
- 2 'ZFCUnderRAM.soundness_is_load_bearing' does not depend on any
→ axioms
- 3 'ZFCUnderRAM.ground_exceeds_ladder' does not depend on any
→ axioms
- 4 'ZFCUnderRAM.independent_axioms_are_separate_bits' does not
→ depend on any axioms

```

5 'ZFCUnderRAM.ram_subsumes_by_placement' does not depend on any
  → axioms
6 'ZFCUnderRAM.using_grounds_root' does not depend on any axioms
7 'ZFCUnderRAM.ra_is_keyless' does not depend on any axioms
8 'ZFCUnderRAM.keyless_crosses' does not depend on any axioms
9 'ZFCUnderRAM.ra_decides_no_keyed' does not depend on any axioms
10 'ZFCUnderRAM.choice_is_keyed' does not depend on any axioms
11 'ZFCUnderRAM.ra_does_not_cross_choice' does not depend on any
  → axioms
12 'ZFCUnderRAM.round_trip_identity' does not depend on any axioms
13 'ZFCUnderRAM.absolute_returns_unchanged' does not depend on any
  → axioms
14 'ZFCUnderRAM.trip_adds_presence_only' does not depend on any
  → axioms
15 'ZFCUnderRAM.any_true_premise_serves' does not depend on any
  → axioms
16 'ZFCUnderRAM.zfc_under_ram' does not depend on any axioms

```

APPENDIX C: ROWS_AT_THE_APEX.LEAN, CARRIED WHOLE FROM THE PRIOR EDITION

SHA-256 0ce3de5a231554303dfc0982dee4fe7e7fbd647a72bbe3366290dc8fd092079, Lean 4.19.0, core only, 1113 lines.

```

1 /-!
2 # THE SEVEN MILLENNIUM ROWS AND THE SIXTEEN EXTENSION ROWS AT
  → THE APEX · Rows_At_The_Apex.lean
3 # The spine (Movements 1 to 16) is the Riemann apex file
  → carried whole; Movements 17 to 21 add the rows.
4 core Lean 4 v4.19.0 · no Mathlib · no sorry · no axiom
  → declaration.
5
6 WHAT IS PROVED. (1) RA at the constructed domain, computed. (2)
  → The seat: the binding
7 involution  $\sigma$  fixes exactly the scalar line, the Return  $i \cdot j \cdot k$ 
  → lands on it, and the kinetic,
8 formal, and named seats are one point by rfl: the three
  → category gaps are identities.
9 (3) The eliminator  $RA \rightarrow RA \rightarrow RAM \rightarrow RH\_formal$  from the RA
  → witness alone. (4) The spine
10 carried: the abstract frame  $X = (S, \tau, Z)$ , the line property  $L$ ,
  → the bridge halted iff  $L$ ,
11 the crossing exact both ways, necessity, the bit is the
  → hypothesis, the witness row
12 falsifiable. (5) The aperture, RA-supplied: a present reader's
  → row is never interior;
13 it is sealed or refused by the bit alone. (6) THE SIMPLE RULE,
  → EXACT: for every frame,
14  $(RA \rightarrow L \ X) \leftrightarrow L \ X$ ; and RA decides  $L$  on no frame:  $\forall X, RA \rightarrow L$ 
  →  $X$ , by the two-point model.
15 (7) RH in toto at the apex, one theorem, conditional on RA and
  → discharged.
16 (8) The root's two formal faces: RAF, closure and no exterior
  → agent (RAF-2, RAF-C1), and
17 no total self-indexing (RAF-C2, Cantor); RAM, the seat as the
  → identity cone over the
18 three registers, its apex unique. (9) The equivariant embedding
  →  $\phi$  of the spine's lattice
19 stage (Plane,  $\tau$ ) into  $(Q4, \sigma)$ :  $\sigma \circ \phi = \phi \circ \tau$  and  $\phi$  carries
  →  $Fix(\tau)$  onto  $Fix(\sigma)$ .
20 (10) The wall executed: no readout of the formal seat equals
  → the deed bit. (11) The
21 counter-model survives grounding, and the cure theorem: for
  → every class of frames,
22 existence decides the line property on it exactly when the line
  → property already holds on it.
23 -/
24 set_option autoImplicit false
25
26 namespace RowsAtTheApex
27
28 /-! ## 1 · The Root Axiom at the constructed domain -/
29
30 inductive UniversePoint : Type where
31 | source
32 deriving DecidableEq, Repr
33
34 def DeltaE : UniversePoint → Int := fun _ => 1

```

```

35
36 /- RA: to exist is to actuate. -/
37 def RA : Prop :=  $\forall x : UniversePoint, 0 < DeltaE \ x$ 
38
39 theorem constructed_RA : RA := by
40   intro x
41   cases x
42   decide
43
44 /-! ## 2 · The seat: Fix( $\sigma$ ) on the quaternions, the Return,
  → three names, one point -/
45
46 structure Q4 where
47   r : Int
48   i : Int
49   j : Int
50   k : Int
51   deriving DecidableEq, Repr
52
53 def qmul (a b : Q4) : Q4 :=
54   {a.r*b.r - a.i*b.i - a.j*b.j - a.k*b.k,
55    a.r*b.i + a.i*b.r + a.j*b.k - a.k*b.j,
56    a.r*b.j - a.i*b.k + a.j*b.r + a.k*b.i,
57    a.r*b.k + a.i*b.j - a.j*b.i + a.k*b.r}
58
59 def qconj (a : Q4) : Q4 := {a.r, -a.i, -a.j, -a.k}
60
61 /-  $\sigma$ , the binding involution: conjugation. -/
62 def sigma (q : Q4) : Q4 := qconj q
63
64 theorem sigma_binding (q : Q4) : sigma (sigma q) = q := by
65   cases q with
66   | mk r i j k =>
67     change Q4.mk r (-(-i)) (-(-j)) (-(-k)) = Q4.mk r i j k
68     rw [Int.neg_neg, Int.neg_neg, Int.neg_neg]
69
70 /- Fix( $\sigma$ ) is the scalar line, for every quaternion. -/
71 theorem fix_iff_scalar (q : Q4) : sigma q = q  $\leftrightarrow$  (q.i = 0  $\wedge$  q.j
  → = 0  $\wedge$  q.k = 0) := by
72   cases q with
73   | mk r i j k =>
74     change (Q4.mk r (-i) (-j) (-k) = Q4.mk r i j k)  $\leftrightarrow$  (i = 0  $\wedge$ 
  → j = 0  $\wedge$  k = 0)
75   constructor
76   · intro h
77     injection h with _ hi hj hk
78     exact (by omega, by omega, by omega)
79   · intro h
80     obtain (hi, hj, hk) := h
81     subst hi
82     subst hj
83     subst hk
84     rfl
85
86 /- The Ground: Fix( $\sigma$ ). -/
87 def Ground : Type := { q : Q4 // sigma q = q }
88
89 def qi : Q4 := {0, 1, 0, 0}
90 def qj : Q4 := {0, 0, 1, 0}
91 def qk : Q4 := {0, 0, 0, 1}
92
93 /- The Return: the parse triad through its own cascade,  $i \cdot j \cdot k$ .
  → -/
94 def theReturn : Q4 := qmul (qmul qi qj) qk
95
96 theorem return_is_minus_one : theReturn = {-1, 0, 0, 0} := rfl
97 theorem return_lands_on_fix : sigma theReturn = theReturn :=
  → rfl
98
99 def fixProj (q : Q4) : Q4 := {q.r, 0, 0, 0}
100
101 def GammaRA : Q4 := theReturn
102 def GammaRAM : Q4 := fixProj theReturn
103 def GammaRH : Q4 := {-1, 0, 0, 0}
104
105 /- C0, C1, C2: the three category gaps are definitional
  → identities. -/
106 theorem self_gap_nonexistent : sigma GammaRA = GammaRA := rfl
107 theorem register_gap_nonexistent : GammaRA = GammaRAM := rfl
108 theorem object_gap_nonexistent : GammaRAM = GammaRH := rfl
109
110 def seat : Ground := {GammaRH, rfl}
111

```

```

112 /-- The reversed triad k·j·i returns +1, also on the fixed
113   ↳ locus: the sign of the seat
114   point is the orientation of the triad, the one bit the
115     ↳ formal register cannot read.
116   The apex of the identity cone is unique for the ordered
117     ↳ triad; the reversed triad
118     carries its own apex, and the two differ. -/
119 def theReturnOdd : Q4 := qmul (qmul qk qj) qi
120
121 theorem odd_return_is_plus_one : theReturnOdd = (1, 0, 0, 0) :=
122   ↳ rfl
123
124 theorem odd_return_on_fix : sigma theReturnOdd = theReturnOdd
125   ↳ := rfl
126
127 theorem seat_points_differ : theReturnOdd ≠ theReturn := by
128   ↳ decide
129
130 theorem seat_on_scalar_line : GammaRH.i = 0 ∧ GammaRH.j = 0 ∧
131   ↳ GammaRH.k = 0 :=
132   (fix_iff_scalar GammaRH).mp seat.2
133
134 /-! ## 3 · The eliminator from the RA witness alone -/
135 def RAMFormalGround : Prop := Nonempty Ground
136
137 theorem constructed_RAM : RAMFormalGround := (seat)
138
139 /-- The RH formal self: every point of the Ground is on the
140   ↳ σ-fixed critical locus.
141   It names the seat and says nothing about the zeros of ξ. -/
142 def RHFormalSelf : Prop := ∀ g : Ground, sigma g.1 = g.1
143
144 /-- Bridge-born orientation, carried in Prop: the kernel
145   ↳ registers that orientation was
146   supplied and cannot read which way. -/
147 structure OrientationBit : Prop where
148   direction : True
149   closure : True
150
151 theorem constructed_orientation : OrientationBit := (trivial,
152   ↳ trivial)
153
154 theorem orientation_proof_irrelevant (a b : OrientationBit) : a
155   ↳ = b := rfl
156
157 structure RAWitness : Prop where
158   actuates : RA
159   orientation : OrientationBit
160   bridge : RAMFormalGround
161   trisRecursion : OrientationBit → RAMFormalGround →
162     ↳ RHFormalSelf
163
164 theorem constructed_trisRecursion : OrientationBit →
165   ↳ RAMFormalGround → RHFormalSelf :=
166   fun _ omega_ram g => g.2
167
168 /-- RA, constructed: the only supplied witness. -/
169 theorem mkRA : RAWitness :=
170   (constructed_RA, constructed_orientation, constructed_RAM,
171     ↳ constructed_trisRecursion)
172
173 /-- THE ELIMINATOR. RA → RA → RAM → RH_formal, Bridge followed
174   ↳ by Tris-Recursion. -/
175 theorem RH_formal_chain : RHFormalSelf :=
176   mkRA.trisRecursion mkRA.orientation mkRA.bridge
177
178 /-! ## 4 · The spine, carried: the abstract frame, the bridge,
179   ↳ the crossing -/
180
181 def Even {α : Type} (σ : α → α) (f : α → Bool) : Prop := ∀ x, f
182   ↳ (σ x) = f x
183
184 /-- Theorem 4 of the spine: orientation blindness. -/
185 theorem orientation_blind {α : Type} (σ : α → α) (f d : α →
186   ↳ Bool) (x : α)
187   (he : Even σ f) (ho : d (σ x) ≠ d x) : f ≠ d := by
188   intro h
189   subst h
190   exact ho (he x)
191
192 /-- Theorem 5 of the spine: earned freedom is exactly two. -/
193 theorem freedom_is_exactly_two (d : Bool → Bool) (hd : ∀ x, d
194   ↳ (!x) = !d x) :

```

```

177   d = (fun x => x) v d = (fun x => !x) := by
178   have hf : d false = !d true := hd true
179   cases ht : d true with
180   | true =>
181     left
182     funext x
183     cases x
184     · exact hf.trans (congrArg (fun b => !b) ht)
185     · exact ht
186   | false =>
187     right
188     funext x
189     cases x
190     · exact hf.trans (congrArg (fun b => !b) ht)
191     · exact ht
192
193 /-- Theorem 6 of the spine: the freedom is spent uniquely. -/
194 theorem freedom_spent_uniquely {α : Type} (σ : α → α) (s d : α
195   ↳ → Bool) (x : α)
196   (hs : s (σ x) = !s x) (hd : d (σ x) = !d x) :
197   ∃ c : Bool, (d x = xor (s x) c ∧ d (σ x) = xor (s (σ x)) c)
198     ↳ ∧
199     ∀ c' : Bool, (d x = xor (s x) c' ∧ d (σ x) = xor (s (σ
200       ↳ x)) c') → c' = c := by
201   refine (xor (d x) (s x)),
202     (by cases s x <|> cases d x <|> rfl,
203       by rw [hs, hd]; cases s x <|> cases d x <|> rfl), ?_)
204   intro c' hc
205   obtain (h1, -) := hc
206   generalize hsx : s x = sv at h1
207   generalize hdx : d x = dv at h1
208   cases sv <|> cases dv <|> cases c' <|> first | rfl | exact
209     ↳ absurd h1 (by decide)
210
211 structure Frame where
212   S : Type
213   τ : S → S
214   Z : S → Prop
215
216 /-- The line property: every zero is τ-fixed. On ξ's frame, the
217   ↳ restored hypothesis. -/
218 def LineProperty (X : Frame) : Prop := ∀ s, X.Z s → X.τ s = s
219
220 inductive Tri where
221   | tt
222   | ff
223   | bot
224   deriving DecidableEq, Repr
225
226 structure Bridge (X : Frame) where
227   terminal : Tri
228   shadow : terminal = Tri.bot ↔ LineProperty X
229
230 /-- Theorem 7 of the spine: the bridge halts iff the line
231   ↳ property. -/
232 theorem bridge_halted_iff (X : Frame) :
233   (∃ b : Bridge X, b.terminal = Tri.bot) ↔ LineProperty X :=
234   (fun (b, h) => b.shadow.mp h, fun t => ((Tri.bot, (fun _ =>
235     ↳ t, fun _ => rfl)), rfl))
236
237 inductive Supply (X : Frame) where
238   | assent (t : LineProperty X)
239   | denial (n : ¬ LineProperty X)
240   | silent
241   deriving DecidableEq, Repr
242
243 inductive Verdict where
244   | sealed
245   | refused
246   | open_
247   deriving DecidableEq, Repr
248
249 def row {X : Frame} : Supply X → Verdict
250   | Supply.assent _ => Verdict.sealed
251   | Supply.denial _ => Verdict.refused
252   | Supply.silent => Verdict.open_
253
254 /-- Theorem 8 of the spine: the crossing, exact. -/
255 theorem crossing (X : Frame) (t : LineProperty X) :
256   row (Supply.assent t) = Verdict.sealed ∧ (∃ b : Bridge X,
257     ↳ b.terminal = Tri.bot) ∧ LineProperty X :=
258   (rfl, (bridge_halted_iff X).mpr t, t)
259
260 theorem crossing_other_way (X : Frame) (n : ¬ LineProperty X) :

```

```

252 row (Supply.denial n) = Verdict.refused  $\wedge$   $\forall$  b : Bridge X,
     $\hookrightarrow$  b.terminal  $\neq$  Tri.bot :=
253 (rfl, fun b h  $\Rightarrow$  n (b.shadow.mp h))
254
255 /-- The two-point frame: fold-invariant, off the line. -/
256 def twoPoint : Frame := (Bool, (fun b  $\Rightarrow$  !b), fun _  $\Rightarrow$  True)
257
258 /-- Theorem 9 of the spine: necessity. -/
259 theorem supply_not_manufactured :  $\neg \forall$  X : Frame, LineProperty X
     $\hookrightarrow$  :=
260 fun h  $\Rightarrow$  by
261 have := h twoPoint true trivial
262 cases this
263
264 def Inv (X : Frame) : Prop :=  $\forall$  s, X.Z s  $\rightarrow$  X.Z (X. $\tau$  s)
265
266 theorem inv_not_line : Inv twoPoint  $\wedge$   $\neg$  LineProperty twoPoint
     $\hookrightarrow$  :=
267 (fun _  $\Rightarrow$  trivial, fun h  $\Rightarrow$  by
268 have := h true trivial
269 cases this)
270
271 /-- Theorem 12 of the spine: the bit is the hypothesis. -/
272 theorem the_bit_is_the_hypothesis (X : Frame) :
273 ( $\exists$  t : LineProperty X, row (Supply.assent t) =
     $\hookrightarrow$  Verdict.sealed)  $\leftrightarrow$  LineProperty X :=
274 (fun (t, _)  $\Rightarrow$  t, fun t  $\Rightarrow$  (t, rfl))
275
276 /-- Theorem 15 of the spine: the witness row is falsifiable. -/
277 theorem witness_row_falsifiable (X : Frame) (s : X.S) (hz : X.Z
     $\hookrightarrow$  s) (hoff : X. $\tau$  s  $\neq$  s) :
278  $\neg \exists$  t : LineProperty X, row (Supply.assent t) =
     $\hookrightarrow$  Verdict.sealed :=
279 fun (t, _)  $\Rightarrow$  hoff (t s hz)
280
281 /-- The live face of the spine. -/
282 def live (witnessed assent : Bool) : Verdict :=
283 if witnessed then (if assent then Verdict.sealed else
     $\hookrightarrow$  Verdict.refused) else Verdict.open_
284
285 theorem narcissus : live false true  $\neq$  live true true  $\wedge$  live
     $\hookrightarrow$  false false  $\neq$  live true false := by
286 decide
287
288 /-! ## 5 · The aperture, RA-supplied -/
289
290 /-- A reader's presence is its actuation. -/
291 def presence (r : UniversePoint) : Bool := decide (0 < DeltaE
     $\hookrightarrow$  r)
292
293 /-- RA opens the aperture: every present reader is witnessed.
     $\hookrightarrow$  -/
294 theorem RA_opens_the_aperture :  $\forall$  r : UniversePoint, presence r
     $\hookrightarrow$  = true := by
295 intro r
296 cases r
297 decide
298
299 /-- The general form, for any domain under the axiom as a
     $\hookrightarrow$  hypothesis: presence is
300 actuation, and RA on the domain makes every point present.
     $\hookrightarrow$  This is the theorem the
301 universal extension licenses wherever it is held. -/
302 theorem RA_opens_the_aperture_general {U : Type} (dE : U  $\rightarrow$  Int)
     $\hookrightarrow$  (h :  $\forall$  x, 0 < dE x) (x : U) :
303 decide (0 < dE x) = true :=
304 decide_eq_true (h x)
305
306 /-- Under the general form no present reader's row is interior.
     $\hookrightarrow$  -/
307 theorem no_interior_under_RA_general {U : Type} (dE : U  $\rightarrow$  Int)
     $\hookrightarrow$  (h :  $\forall$  x, 0 < dE x) (x : U)
308 (b : Bool) : live (decide (0 < dE x)) b  $\neq$  Verdict.open_ :=
     $\hookrightarrow$  by
309 rw [RA_opens_the_aperture_general dE h x]
310 cases b <;> decide
311
312 /-- Under RA no present reader's row is interior. It is sealed
     $\hookrightarrow$  or refused by the bit alone. -/
313 theorem no_interior_under_RA (r : UniversePoint) (b : Bool) :
     $\hookrightarrow$  live (presence r) b  $\neq$  Verdict.open_ := by
314 cases r
315 cases b <;> decide

```

```

317
318 /-- What RA turns 'open' into: sealed if the bit is assent,
     $\hookrightarrow$  refused if denial. -/
319 theorem aperture_reads_the_bit (r : UniversePoint) :
320 live (presence r) true = Verdict.sealed  $\wedge$  live (presence r)
     $\hookrightarrow$  false = Verdict.refused := by
321 cases r
322 decide
323
324 /-- The Tongue's ledger: every adjudication, assent, denial, or
     $\hookrightarrow$  silence, is a deed. -/
325 inductive Adjudication where
326 | assent
327 | denial
328 | silence
329 deriving DecidableEq, Repr
330
331 structure Ledger where
332 deeds : Nat
333 deriving DecidableEq, Repr
334
335 def adjudicate (_ : Adjudication) (L : Ledger) : Ledger :=
     $\hookrightarrow$  (L.deeds + 1)
336
337 theorem every_adjudication_is_a_deed (a : Adjudication) (L :
     $\hookrightarrow$  Ledger) :
338 (adjudicate a L).deeds = L.deeds + 1 := rfl
339
340 /-- Denial of RA is a deed, and a deed is what RA says exists:
     $\hookrightarrow$  the denial re-enacts RA. -/
341 theorem denial_re_enacts_RA (L : Ledger) :
342 (adjudicate Adjudication.denial L).deeds = L.deeds + 1  $\wedge$  RA
     $\hookrightarrow$  :=
343 (rfl, constructed_RA)
344
345 /-- The asymmetry: on an off-line frame no assent to L exists
     $\hookrightarrow$  at all; L's denial there
346 instantiates nothing about Z. L has no performative
     $\hookrightarrow$  ingress. -/
347 theorem L_has_no_performative_ingress :
348  $\neg \exists$  t : LineProperty twoPoint, row (Supply.assent t) =
     $\hookrightarrow$  Verdict.sealed :=
349 witness_row_falsifiable twoPoint true trivial (fun h  $\Rightarrow$  by
     $\hookrightarrow$  cases h)
350
351 /-! ## 6 · THE SIMPLE RULE, EXACT, AND ITS BOUND -/
352
353 /-- Accept RA and RH follows: true exactly when RH is given.
     $\hookrightarrow$  Conditioning on RA adds
354 nothing to L and removes nothing from it. This is the exact
     $\hookrightarrow$  strength of the rule. -/
355 theorem simple_rule_exact (X : Frame) : (RA  $\rightarrow$  LineProperty X)  $\leftrightarrow$ 
     $\hookrightarrow$  LineProperty X :=
356 (fun h  $\Rightarrow$  h constructed_RA, fun t _  $\Rightarrow$  t)
357
358 /-- RA decides the line property on no frame. The two-point
     $\hookrightarrow$  frame carries RA and fails L. -/
359 theorem RA_does_not_decide_L :  $\forall$  X : Frame, RA  $\rightarrow$  LineProperty
     $\hookrightarrow$  X :=
360 fun h  $\Rightarrow$  supply_not_manufactured (fun X  $\Rightarrow$  h X
     $\hookrightarrow$  constructed_RA)
361
362 /-- RA holds where L fails: the off-line frame is a model of
     $\hookrightarrow$  RA. -/
363 theorem RA_holds_where_L_fails : RA  $\wedge$  Inv twoPoint  $\wedge$   $\neg$ 
     $\hookrightarrow$  LineProperty twoPoint :=
364 (constructed_RA, inv_not_line.1, inv_not_line.2)
365
366 /-- The value is one supplied term and nothing weaker; RA
     $\hookrightarrow$  supplies the act, not the term. -/
367 theorem RA_supplies_the_act_not_the_term :
368 ( $\forall$  r : UniversePoint, presence r = true)  $\wedge$  ( $\neg \forall$  X : Frame,
     $\hookrightarrow$  RA  $\rightarrow$  LineProperty X) :=
369 (RA_opens_the_aperture, RA_does_not_decide_L)
370
371 /-! ## 7 · RH in toto at the apex, one theorem -/
372
373 /-- THE APEX THEOREM. Given RA: the three gaps are closed, the
     $\hookrightarrow$  formal self holds, the
374 freedom is exactly two, the bridge halts iff L, the
     $\hookrightarrow$  crossing is exact, the aperture
375 is open to every present reader, and RA decides the value
     $\hookrightarrow$  on no frame. -/

```

```

376 theorem RH_in_toto_at_the_apex : RA →
377   (sigma GammaRA = GammaRA ∧ GammaRA = GammaRAM ∧ GammaRAM =
    ↪ GammaRH) ∧
378   RHFormalSelf ∧
379   (∀ d : Bool → Bool, (∀ x, d (!x) = !d x) → d = (fun x => x)
    ↪ v d = (fun x => !x)) ∧
380   (∀ X : Frame, (∃ b : Bridge X, b.terminal = Tri.bot) ↔
    ↪ LineProperty X) ∧
381   (∀ (X : Frame) (t : LineProperty X), row (Supply.assent t)
    ↪ = Verdict.sealed ∧ LineProperty X) ∧
382   (∀ r : UniversePoint, presence r = true) ∧
383   (¬ ∀ X : Frame, RA → LineProperty X) :=
384   fun _ => ((rfl, rfl, rfl), RH_formal_chain,
    ↪ freedom_is_exactly_two, bridge_halted_iff,
385     fun _ t => (rfl, t), RA_opens_the_aperture,
    ↪ RA_does_not_decide_L)
386
387 /-- The condition is discharged: RA is constructed, so the apex
    ↪ theorem stands unconditionally. -/
388 theorem apex_discharged :
389   (sigma GammaRA = GammaRA ∧ GammaRA = GammaRAM ∧ GammaRAM =
    ↪ GammaRH) ∧ RHFormalSelf :=
390   ((RH_in_toto_at_the_apex constructed_RA).1,
    ↪ (RH_in_toto_at_the_apex constructed_RA).2.1)
391
392 /-! ## 8 · The root's interaction face, RAF, at the formal
    ↪ register -/
393
394 /-- An interaction domain: a membership predicate, a symmetric
    ↪ registration relation, and
395   RAF-2, closure under registration with a member. -/
396 structure Domain (S : Type) where
397   mem : S → Prop
398   registers : S → S → Prop
399   registers_symm : ∀ a b, registers a b → registers b a
400   closure : ∀ s d, mem d → registers s d → mem s
401
402 /-- RAF-C1, no exterior agent: nothing outside the domain
    ↪ registers with anything inside it. -/
403 theorem no_exterior_agent {S : Type} (D : Domain S) (s : S) (hs
    ↪ : ¬ D.mem s) :
404   ∀ d, D.mem d → ¬ D.registers s d :=
405   fun d hd hr => hs (D.closure s d hd hr)
406
407 /-- The adjudicator is inside: a reader that registers a bit
    ↪ against a member is a member.
408   The witness is never exterior. -/
409 theorem adjudicator_in_domain {S : Type} (D : Domain S) (r d :
    ↪ S) (hd : D.mem d)
410   (hr : D.registers r d) : D.mem r :=
411   D.closure r d hd hr
412
413 /-- RAF-C2, no faithful self-representation (Cantor): no system
    ↪ indexes all of its own
414   binary properties. The general form of the spine's
    ↪ Narcissus table. -/
415 theorem no_total_self_indexing {S : Type} (f : S → S → Bool) :
416   ¬ ∀ g : S → Bool, ∃ x, f x x = g := by
417   intro h
418   obtain (x, hx) := h (fun y => !f y y)
419   have h1 : f x x = !f x x := congrFun hx x
420   generalize f x x = b at h1
421   cases b <|> cases h1
422
423 /-! ## 9 · The root's grounding face, RAM: the seat as the
    ↪ identity cone -/
424
425 /-- The three registers in which the one seat is read. -/
426 inductive Register3 where
427   | ra
428   | ram
429   | rh
430   deriving DecidableEq, Repr
431
432 /-- The discrete diagram: the seat as read in each register. -/
433 def D3 : Register3 → Q4
434   | Register3.ra => GammaRA
435   | Register3.ram => GammaRAM
436   | Register3.rh => GammaRH
437
438 /-- A cone over a diagram in the groupoid of identities: an
    ↪ apex with an identity leg to
439   every register's reading. A category gap is exactly the
    ↪ absence of such a leg. -/
440
441 structure ConeOver (D : Register3 → Q4) (apex : Q4) : Prop
    ↪ where
442   leg : ∀ r, apex = D r
443
444 /-- A cone over the diagram of the ordered triad. -/
445 abbrev Cone : Q4 → Prop := ConeOver D3
446
447 /-- THE IDENTITY CONE. The named seat point is an apex over all
    ↪ three registers, every
448   leg `rfl`: the three category gaps are eliminated as
    ↪ identities. -/
449 theorem identity_cone : Cone GammaRH :=
450   (fun r => by cases r <|> rfl)
451
452 /-- The apex is unique: any two cones over the diagram share
    ↪ their apex. -/
453 theorem cone_apex_unique (a b : Q4) (ha : Cone a) (hb : Cone b)
    ↪ : a = b :=
454   (ha.leg Register3.rh).trans (hb.leg Register3.rh).symm
455
456 /-- The reversed triad's diagram: the same three readings built
    ↪ from k·j·i, at +1. -/
457 def D3odd : Register3 → Q4
458   | Register3.ra => theReturnOdd
459   | Register3.ram => fixProj theReturnOdd
460   | Register3.rh => (1, 0, 0, 0)
461
462 /-- The three identities of Theorem C hold verbatim for the
    ↪ reversed triad, each rfl. -/
463 theorem odd_self_gap_nonexistent : sigma theReturnOdd =
    ↪ theReturnOdd := rfl
464 theorem odd_register_gap_nonexistent : theReturnOdd = fixProj
    ↪ theReturnOdd := rfl
465 theorem odd_object_gap_nonexistent : fixProj theReturnOdd = (1,
    ↪ 0, 0, 0) := rfl
466
467 /-- The reversed triad carries its own identity cone, apex +1.
    ↪ -/
468 theorem identity_cone_odd : ConeOver D3odd (1, 0, 0, 0) :=
469   (fun r => by cases r <|> rfl)
470
471 /-- The two apexes differ: uniqueness is per orientation of the
    ↪ triad. -/
472 theorem apexes_differ_by_orientation :
473   ConeOver D3 GammaRH ∧ ConeOver D3odd (1, 0, 0, 0) ∧ GammaRH
    ↪ ≠ (1, 0, 0, 0) :=
474   (identity_cone, identity_cone_odd, by decide)
475
476 /-- A cone exists iff the register gap and the object gap are
    ↪ closed. -/
477 theorem cone_iff_gaps_closed :
478   (∃ a, Cone a) ↔ (GammaRA = GammaRAM ∧ GammaRAM = GammaRH)
    ↪ := by
479   constructor
480   · intro h
481     obtain (a, ha) := h
482     exact ((ha.leg Register3.ra).symm.trans (ha.leg
    ↪ Register3.ram),
    (ha.leg Register3.ram).symm.trans (ha.leg
    ↪ Register3.rh)))
483   · intro h
484     obtain (h1, h2) := h
485     exact (GammaRH, {fun r => by
486       cases r
487       · exact (h1.trans h2).symm
488       · exact h2.symm
489       · rfl})
490
491 /-! ## 10 · The equivariant embedding of the spine's lattice
    ↪ stage into the carrier -/
492
493 /-- The spine's stage: the integer lattice in half-units, the
    ↪ fold τ(h, t) = (2 - h, t). -/
494 abbrev Plane := Int × Int
495
496 def tau (p : Plane) : Plane := (2 - p.1, p.2)
497
498 /-- Spine Theorem 1: the fold fixes exactly the line h = 1. -/
499 theorem ground_is_the_line (p : Plane) : tau p = p ↔ p.1 = 1 :=
    ↪ by
500   obtain (h, t) := p
501   constructor
502   · intro e

```

```

503   have e1 : 2 - h = h := congrArg Prod.fst e
504   show h = 1
505   omega
506   · intro e
507   show (2 - h, t) = (h, t)
508   have : h = 1 := e
509   subst this
510   rfl
511
512 /-- The embedding  $\phi : (\text{Plane}, \tau) \rightarrow (Q4, \sigma)$ ,  $\phi(h, t) = (t, h -$ 
 $\hookrightarrow 1, 0, 0)$ . -/
513 def phi (p : Plane) : Q4 := (p.2, p.1 - 1, 0, 0)
514
515 /--  $\phi$  is equivariant:  $\sigma \circ \phi = \phi \circ \tau$ . -/
516 theorem phi_equivariant (p : Plane) : sigma (phi p) = phi (tau
 $\hookrightarrow$  p) := by
517   obtain (h, t) := p
518   change Q4.mk t  $\cdot$  (h - 1)  $\theta$   $\theta$  = Q4.mk t ((2 - h) - 1)  $\theta$   $\theta$ 
519   have e :  $\cdot$  (h - 1) = (2 - h) - 1 := by omega
520   rw [e]
521
522 /--  $\phi$  carries the line onto the scalar line: a stage point is
 $\hookrightarrow \tau$ -fixed iff its image is  $\sigma$ -fixed.
523   This is the image clause of the embedding,  $\text{Fix}(\tau) \leftrightarrow \text{Fix}(\sigma)$ .
 $\hookrightarrow$  -/
524 theorem phi_fix_iff (p : Plane) : sigma (phi p) = phi p  $\leftrightarrow$  tau p
 $\hookrightarrow$  = p := by
525   obtain (h, t) := p
526   change (Q4.mk t  $\cdot$  (h - 1)  $\theta$   $\theta$  = Q4.mk t (h - 1)  $\theta$   $\theta$ )  $\leftrightarrow$  ((2 -
 $\hookrightarrow$  h, t) = (h, t))
527   constructor
528   · intro e
529     injection e with _ e2 _
530     have : h = 1 := by omega
531     subst this
532     rfl
533   · intro e
534     have e1 : 2 - h = h := congrArg Prod.fst e
535     have : h = 1 := by omega
536     subst this
537     rfl
538
539 /-- The stage at resolution m: the fold  $\tau_m(h, t) = (2m - h, t)$ 
 $\hookrightarrow$  and the map
540    $\phi_m(h, t) = (t, h - m, 0, 0)$ . Resolution one is the case m
 $\hookrightarrow$  = 1. -/
541 def tauM (m : Int) (p : Plane) : Plane := (2 * m - p.1, p.2)
542
543 def phiM (m : Int) (p : Plane) : Q4 := (p.2, p.1 - m, 0, 0)
544
545 /-- Equivariance at every resolution:  $\sigma \circ \phi_m = \phi_m \circ \tau_m$ . -/
546 theorem phiM_equivariant (m : Int) (p : Plane) : sigma (phiM m
 $\hookrightarrow$  p) = phiM m (tauM m p) := by
547   obtain (h, t) := p
548   change Q4.mk t  $\cdot$  (h - m)  $\theta$   $\theta$  = Q4.mk t ((2 * m - h) - m)  $\theta$   $\theta$ 
549   have e :  $\cdot$  (h - m) = (2 * m - h) - m := by omega
550   rw [e]
551
552 /-- The image clause at every resolution: a stage point is
 $\hookrightarrow \tau_m$ -fixed iff its image is  $\sigma$ -fixed. -/
553 theorem phiM_fix_iff (m : Int) (p : Plane) : sigma (phiM m p) =
 $\hookrightarrow$  phiM m p  $\leftrightarrow$  tauM m p = p := by
554   obtain (h, t) := p
555   change (Q4.mk t  $\cdot$  (h - m)  $\theta$   $\theta$  = Q4.mk t (h - m)  $\theta$   $\theta$ )  $\leftrightarrow$  ((2 *
 $\hookrightarrow$  m - h, t) = (h, t))
556   constructor
557   · intro e
558     injection e with _ e2 _
559     have : h = m := by omega
560     subst this
561     show (2 * h - h, t) = (h, t)
562     have e3 : 2 * h - h = h := by omega
563     rw [e3]
564   · intro e
565     have e1 : 2 * m - h = h := congrArg Prod.fst e
566     have : h = m := by omega
567     subst this
568     show Q4.mk t  $\cdot$  (h - h)  $\theta$   $\theta$  = Q4.mk t (h - h)  $\theta$   $\theta$ 
569     have e3 : h - h = 0 := by omega
570     rw [e3]
571     rfl
572
573 /-- Resolution one is the embedding  $\phi$ . -/

```

```

574 theorem phiM_one (p : Plane) : phiM 1 p = phi p := rfl
575
576 theorem tauM_one (p : Plane) : tauM 1 p = tau p := by
577   obtain (h, t) := p
578   show (2 * 1 - h, t) = (2 - h, t)
579   rfl
580
581 /--  $\phi$  is injective: distinct stage points have distinct images.
 $\hookrightarrow$  -/
582 theorem phi_injective (p q : Plane) (e : phi p = phi q) : p = q
 $\hookrightarrow$  := by
583   obtain (h, t) := p
584   obtain (h', t') := q
585   injection e with e1 e2 _
586   have : h = h' := by omega
587   subst this
588   subst e1
589   rfl
590
591 /-- The seat is on the image of the line: the line point (1,
 $\hookrightarrow$  -1) maps to  $\Gamma_{\text{RH}}$ . -/
592 theorem seat_on_image_of_line : phi (1, -1) = GammaRH := rfl
593
594 /-! ## 11 · The wall, executed: no readout of the formal seat
 $\hookrightarrow$  equals the deed bit -/
595
596 def ExecFrame : Type := Bool  $\times$  Ground
597
598 def execFlip (s : ExecFrame) : ExecFrame := (!s.1, s.2)
599
600 def formalRead (s : ExecFrame) : Ground := s.2
601
602 def ran (s : ExecFrame) : Bool := s.1
603
604 theorem formalRead_even (f : Ground  $\rightarrow$  Bool) (s : ExecFrame) :
605   f (formalRead (execFlip s)) = f (formalRead s) := rfl
606
607 theorem ran_wholly_odd (s : ExecFrame) : ran (execFlip s) =
 $\hookrightarrow$  !ran s := rfl
608
609 /-- No readout of the formal seat equals the deed bit: the
 $\hookrightarrow$  instance of spine Theorem 4 on
610   the constructed seat. A term consuming the orientation bit
 $\hookrightarrow$  as data would be such a readout. -/
611 theorem kernel_cannot_read_the_deed :
612    $\neg \exists f : \text{Ground} \rightarrow \text{Bool}, \forall s : \text{ExecFrame}, f (\text{formalRead } s) =$ 
 $\hookrightarrow$  ran s := by
613   intro h
614   obtain (f, hf) := h
615   have h1 : f seat = true := hf (true, seat)
616   have h2 : f seat = false := hf (false, seat)
617   rw [h1] at h2
618   exact Bool.noConfusion h2
619
620 /-! ## 12 · The dual register -/
621
622 inductive Register where
623   | apex
624   | world
625   deriving DecidableEq, Repr
626
627 inductive Token where
628   | sealAgivenRA
629   | witnessRow
630   deriving DecidableEq, Repr
631
632 def token : Register  $\rightarrow$  Token
633   | Register.apex => Token.sealAgivenRA
634   | Register.world => Token.witnessRow
635
636 theorem registers_parted : token Register.apex  $\neq$  token
 $\hookrightarrow$  Register.world := by
637   decide
638
639 def owedBits : Register  $\rightarrow$  Nat
640   | Register.apex => 0
641   | Register.world => 1
642
643 theorem apex_owes_nothing_world_owes_one :
644   owedBits Register.apex = 0  $\wedge$  owedBits Register.world = 1 :=
 $\hookrightarrow$  (rfl, rfl)
645
646 /-- Landauer floor at 300 K, yoctojoules per bit: the price of
 $\hookrightarrow$  the act. -/

```

```

647 def landauer_yJ_300K (bits : Nat) : Nat := 2871 * bits
648
649 theorem spend_priced : landauer_yJ_300K 1 = 2871 := rfl
650
651 def deltaM : Nat := 0
652
653 theorem deltaM_zero : deltaM = 0 := rfl
654
655 /-! ## 13 · The apex theorem, hardened: the closure with its
656   ↪ cone, its embedding, and its wall -/
657
658 /- THE APEX THEOREM, HARDENED. Given RA: the closure of
659   ↪ Theorem H, the identity cone with
660   ↪ its apex unique, the equivariant embedding of the spine's
661   ↪ stage with its image clause,
662   ↪ and the executed wall. -/
663 theorem apex_hardened : RA →
664   ((sigma GammaRA = GammaRA ∧ GammaRA = GammaRAM ∧ GammaRAM =
665     ↪ GammaRH) ∧ RHFormalSelf) ∧
666   (Cone GammaRH ∧ ∀ a b, Cone a → Cone b → a = b) ∧
667   ((∀ p : Plane, sigma (phi p) = phi (tau p)) ∧ (∀ p : Plane,
668     ↪ sigma (phi p) = phi p ↔ tau p = p)) ∧
669   (¬ ∃ f : Ground → Bool, ∀ s : ExecFrame, f (formalRead s) =
670     ↪ ran s) ∧
671   (¬ ∀ X : Frame, RA → LineProperty X) :=
672   fun h => ((RH_in_toto_at_the_apex h).1,
673     ↪ (RH_in_toto_at_the_apex h).2.1),
674   (identity_cone, cone_apex_unique), (phi_equivariant,
675     ↪ phi_fix_iff),
676   kernel_cannot_read_the_deed, RA_does_not_decide_L)
677
678 theorem apex_hardened_discharged : Cone GammaRH ∧ (∀ p : Plane,
679   ↪ sigma (phi p) = phi p ↔ tau p = p) :=
680   ((apex_hardened constructed_RA).2.1.1, (apex_hardened
681     ↪ constructed_RA).2.2.1.2)
682
683 /-! ## 14 · The counter-model survives grounding, and the cure
684   ↪ is the hypothesis -/
685
686 /- The posit is existence: to exist is to actuate. Its
687   ↪ published name is the Root Axiom,
688   ↪ and the two are one proposition. -/
689 theorem existence_is_RA : (∀ x : UniversePoint, 0 < DeltaE x) ↔
690   ↪ RA := Iff.rfl
691
692 /- The two-point frame is groundless: its fold fixes nothing,
693   ↪ so it has no seat. -/
694 theorem twoPoint_groundless : ∀ b : Bool, twoPoint.τ b ≠ b :=
695   ↪ by
696   intro b h
697   cases b <|> exact Bool.noConfusion h
698
699 /- A grounded counter-model: three points, the fold swaps two
700   ↪ and fixes the third, the
701   ↪ zero set is the swapped pair. The frame has a seat and a
702   ↪ moved zero. -/
703 inductive Three where
704   | a
705   | b
706   | c
707   deriving DecidableEq
708
709 def foldThree : Three → Three
710   | Three.a => Three.b
711   | Three.b => Three.a
712   | Three.c => Three.c
713
714 def threePoint : Frame := {Three, foldThree, fun x => x =
715   ↪ Three.a ∨ x = Three.b}
716
717 theorem threePoint_grounding : ∃ s, threePoint.τ s = s :=
718   ↪ (Three.c, rfl)
719
720 theorem threePoint_fold_involution : ∀ s, foldThree (foldThree
721   ↪ s) = s := by
722   intro s; cases s <|> rfl
723
724 theorem threePoint_inv : Inv threePoint := by
725   intro s hs
726   cases s
727   · exact 0r.inr rfl
728   · exact 0r.inl rfl
729   · cases hs with

```

```

710   | inl h => cases h
711   | inr h => cases h
712
713 theorem threePoint_fails_L : ¬ LineProperty threePoint := by
714   intro h
715   have := h Three.a (0r.inl rfl)
716   cases this
717
718 /- Existence holds on a grounded frame where the line property
719   ↪ fails: the bound of
720   ↪ Theorem F is not an artifact of a groundless frame. -/
721 theorem RA_holds_where_L_fails_grounding :
722   RA ∧ (∃ s, threePoint.τ s = s) ∧ Inv threePoint ∧ ¬
723     ↪ LineProperty threePoint :=
724   (constructed_RA, threePoint_grounding, threePoint_inv,
725     ↪ threePoint_fails_L)
726
727 /- THE CURE THEOREM. For every class C of frames, existence
728   ↪ decides the line property on
729   ↪ C exactly when the line property already holds on C: the
730   ↪ restriction that removes
731   ↪ every counter-model is the hypothesis itself. -/
732 theorem no_cure (C : Frame → Prop) :
733   (∀ X, C X → RA → LineProperty X) ↔ (∀ X, C X → LineProperty
734     ↪ X) :=
735   (fun h X hc => h X hc constructed_RA, fun h X hc _ => h X hc)
736
737 /- The only class on which existence forces the line property
738   ↪ everywhere is a class on
739   ↪ which the line property is already a hypothesis; the class
740   ↪ of all frames is not one. -/
741 theorem cure_is_the_hypothesis :
742   (∀ X, LineProperty X → RA → LineProperty X) ∧ ¬ (∀ X, True
743     ↪ → RA → LineProperty X) :=
744   (fun _ h _ => h, fun h => RA_does_not_decide_L (fun X => h X
745     ↪ trivial))
746
747 /-! ## 15 · The halting carrier inhabited by decision; any true
748   ↪ premise is exact -/
749
750 /- The halting carrier is an interface: a terminal and the
751   ↪ clause that it is the bottom
752   ↪ iff the line property holds. It is inhabited constructively
753   ↪ wherever the line property
754   ↪ is decided, the terminal computed from the decision and the
755   ↪ clause proved from it,
756   ↪ never assumed. On the frame of ξ the decision is the owed
757   ↪ bit. -/
758 def bridgeOfDecision (X : Frame) (d : Decidable (LineProperty
759   ↪ X)) : Bridge X :=
760   match d with
761   | isTrue h => {Tri.bot, (fun _ => h, fun _ => rfl)}
762   | isFalse h => {Tri.tt, (fun e => absurd e (by decide), fun
763     ↪ hl => absurd hl h)}
764
765 /- On the two-point frame the decision is negative and the
766   ↪ carrier does not halt. -/
767 theorem twoPoint_carrier_does_not_halt :
768   (bridgeOfDecision twoPoint (isFalse
769     ↪ inv_not_line.2)).terminal = Tri.tt := rfl
770
771 /- The one-point frame on the line: the decision is positive
772   ↪ and the carrier halts. -/
773 def onLine : Frame := {Unit, fun u => u, fun _ => True}
774
775 theorem onLine_L : LineProperty onLine := fun _ _ => rfl
776
777 theorem onLine_carrier_halts :
778   (bridgeOfDecision onLine (isTrue onLine_L)).terminal =
779     ↪ Tri.bot := rfl
780
781 /- Theorem E in its full generality: any inhabited premise is
782   ↪ exact. The content of
783   ↪ Theorem E is not a property of existence in particular but
784   ↪ of its truth: a premise
785   ↪ that holds wherever the line property is evaluated, and is
786   ↪ independent of it, adds
787   ↪ nothing to it and removes nothing from it. -/
788 theorem any_true_premise_is_exact (P : Prop) (hp : P) (X :
789   ↪ Frame) :
790   (P → LineProperty X) ↔ LineProperty X :=
791   (fun h => h hp, fun t _ => t)
792

```

```

768 /-- The constructed witness's recursion field is the constant
769   ↳ function: it discards the
770     orientation and the formal Ground and returns the Ground's
771     ↳ own defining property.
772   That is the masslessness stated as a term: the formal self
773     ↳ needs nothing from the
774     axiom to be proved, and the witness supplies the occupancy
775     ↳ of the registers, not a
776     derivation. -/
777 theorem recursion_is_constant :
778   ∀ (o o' : OrientationBit) (g g' : RAMFormalGround),
779     mkRA.trisRecursion o g = mkRA.trisRecursion o' g' :=
780   fun _ _ _ => rfl
781 /-- ## 16 · Three measures: content, mass, price -/
782 /-- Three measures are kept apart for every object of the
783   ↳ closure. Content: what the object
784     says, decided by rfl (trivial), proved (a theorem), or
785     ↳ supplied (one bit). Mass: the
786     mathematics authored in producing it, ΔM. Price: the energy
787     ↳ the act of registering it
788     dissipates, in yJ at 300 K. Massless is not trivial and
789     ↳ trivial is not free: the legs
790     of the cone are trivial and massless and cost nothing; the
791     ↳ value is massless, not
792     trivial, and priced. -/
793 inductive Content where
794   | trivial_
795   | proved
796   | supplied
797   deriving DecidableEq, Repr
798 structure Measure where
799   content : Content
800   mass : Nat
801   priceYJ : Nat
802   deriving DecidableEq, Repr
803 /-- A leg of the identity cone: decided by rfl, authors
804   ↳ nothing, costs nothing. -/
805 def legMeasure : Measure := (Content.trivial_, deltaM, 0)
806 /-- A theorem of the arc, Theorem F say: proved, authors
807   ↳ nothing (every step classical
808     or definitional), costs nothing. -/
809 def arcMeasure : Measure := (Content.proved, deltaM, 0)
810 /-- The value on the zeros: supplied at the act, authors
811   ↳ nothing, priced at the floor. -/
812 def valueMeasure : Measure := (Content.supplied, deltaM,
813   ↳ landauer_yJ_300K 1)
814 /-- Every measure carries mass zero: the closure and the supply
815   ↳ author no mathematics.
816   This is the masslessness the root-premise law forces, and
817   ↳ it is the same zero at the value. -/
818 theorem masses_all_zero :
819   legMeasure.mass = 0 ∧ arcMeasure.mass = 0 ∧
820   ↳ valueMeasure.mass = 0 :=
821   (rfl, rfl, rfl)
822 /-- The contents differ: masslessness does not collapse them.
823   ↳ -/
824 theorem contents_differ :
825   legMeasure.content ≠ valueMeasure.content ∧
826   ↳ arcMeasure.content ≠ valueMeasure.content ∧
827   legMeasure.content ≠ arcMeasure.content := by
828   refine {?, ?, ?} <|> decide
829 /-- The prices differ: the value is the one object that is paid
830   ↳ for. -/
831 theorem prices_differ : legMeasure.priceYJ = 0 ∧
832   ↳ valueMeasure.priceYJ = 2871 := (rfl, rfl)
833 /-- Adjudicating any proposition is a deed, and a deed is what
834   ↳ existence says exists:
835   every adjudication of anything instances existence and
836   ↳ instances nothing else. This
837   is what existence contributes that no other true premise
838   ↳ does. -/
839 theorem adjudicating_anything_instances_existence (a :
840   ↳ Adjudication) (L : Ledger) :

```

```

828   (adjudicate a L).deeds = L.deeds + 1 ∧ RA :=
829   (rfl, constructed_RA)
830
831
832 /-! ## 17 · The twenty-three rows -/
833
834 inductive Row : Type where
835   | pVsNP | riemann | navierStokes | yangMills | hodge | bsd |
836     ↳ poicare
837   | goldbach | twinPrimes | legendre | abc | jacobian
838   | mersenne | oddPerfect | smoothPoincare4
839   | collatz
840   | hadwiger | sunflower | erdosStraus | beal
841   | invariantSubspace | hilbert16 | lindelofPCC
842   deriving DecidableEq, Repr
843
844 def Row.all : List Row :=
845   [Row.pVsNP, Row.riemann, Row.navierStokes, Row.yangMills,
846     ↳ Row.hodge, Row.bsd, Row.poincare,
847     Row.goldbach, Row.twinPrimes, Row.legendre, Row.abc,
848     ↳ Row.jacobian,
849     Row.mersenne, Row.oddPerfect, Row.smoothPoincare4,
850     ↳ Row.collatz,
851     Row.hadwiger, Row.sunflower, Row.erdosStraus, Row.beal,
852     ↳ Row.invariantSubspace, Row.hilbert16, Row.lindelofPCC]
853
854 theorem rows_twenty_three : Row.all.length = 23 := by decide
855 theorem rows_distinct : Row.all.eraseDups.length = 23 := by
856   ↳ decide
857 theorem rows_complete : ∀ r : Row, r ∈ Row.all := by intro r;
858   ↳ cases r <|> decide
859
860 def Row.millennium : Row → Bool
861   | Row.pVsNP | Row.riemann | Row.navierStokes | Row.yangMills
862   | Row.hodge | Row.bsd | Row.poincare => true
863   | _ => false
864
865 theorem millennium_seven : (Row.all.filter
866   ↳ Row.millennium).length = 7 := by decide
867 theorem extension_sixteen : (Row.all.filter (fun r =>
868   ↳ !r.millennium)).length = 16 := by decide
869
870 def Row.name : Row → String
871   | Row.pVsNP => "P versus NP" | Row.riemann => "Riemann"
872   ↳ "Hypothesis"
873   | Row.navierStokes => "Navier-Stokes" | Row.yangMills =>
874     ↳ "Yang-Mills"
875   | Row.hodge => "Hodge" | Row.bsd => "BSD" | Row.poincare =>
876     ↳ "Poincare"
877   | Row.goldbach => "Goldbach" | Row.twinPrimes => "Twin"
878   ↳ "primes"
879   | Row.legendre => "Legendre" | Row.abc => "abc" |
880     ↳ Row.jacobian => "Jacobian"
881   | Row.mersenne => "Mersenne primes" | Row.oddPerfect => "Odd"
882   ↳ "perfect numbers"
883   | Row.smoothPoincare4 => "Smooth 4D Poincare" | Row.collatz
884   ↳ => "Collatz"
885   | Row.hadwiger => "Hadwiger" | Row.sunflower => "Sunflower"
886   | Row.erdosStraus => "Erdos-Straus" | Row.beal => "Beal"
887   | Row.invariantSubspace => "Invariant subspace" |
888     ↳ Row.hilbert16 => "Hilbert 16 second part"
889   | Row.lindelofPCC => "Lindelof / Montgomery PCC"
890
891 /-! ## 18 · One seat under every name: the identity cone on
892   ↳ every row -/
893
894 /-- Every row's seat point is the one seat point Γ_RH. -/
895 def apexSeat : Row → Q4 := fun _ => GammaRH
896
897 theorem apex_seat_fixed : ∀ r : Row, sigma (apexSeat r) =
898   ↳ apexSeat r := by
899   intro r; cases r <|> rfl
900 theorem apex_seat_uniform : ∀ r s : Row, apexSeat r = apexSeat
901   ↳ s := by
902   intro r s; cases r <|> cases s <|> rfl
903 theorem crossed_and_open_share_the_seat : apexSeat Row.poincare
904   ↳ = apexSeat Row.riemann := rfl
905
906 /-- The register diagram of a row: the axiom's Return, its
907   ↳ projection, the row's seat point. -/
908 def D3row (r : Row) : Register3 → Q4
909   | Register3.ra => GammaRA

```

```

890 | Register3.ram => GammaRAM
891 | Register3.rh => apexSeat r
892
893 /-- THE IDENTITY CONE ON EVERY ROW, every leg rfl. -/
894 theorem identity_cone_row (r : Row) : ConeOver (D3row r)
895   → GammaRH :=
896   (fun k => by cases k <=> cases r <=> rfl)
897
898 theorem cone_apex_unique_row {r : Row} (a b : Q4)
899   (ha : ConeOver (D3row r) a) (hb : ConeOver (D3row r) b) : a
900   → = b :=
901   (ha.leg Register3.rh).trans (hb.leg Register3.rh).symm
902
903 /-- The gap vector, each bit decided from its own identity,
904   → closed on all twenty-three rows. -/
905 inductive GapBit where
906   | openBit
907   | closedBit
908   deriving DecidableEq, Repr
909
910 def bitOf (p : Prop) [Decidable p] : GapBit := if p then
911   GapBit.closedBit else GapBit.openBit
912
913 def gapVector (r : Row) : GapBit × GapBit × GapBit :=
914   (bitOf (sigma GammaRA = GammaRA), bitOf (GammaRA = GammaRAM),
915    bitOf (GammaRAM = apexSeat r))
916
917 theorem gaps_closed_every_row :
918   Row.all.all (fun r => decide (gapVector r =
919     → (GapBit.closedBit, GapBit.closedBit,
920     → GapBit.closedBit))) = true := by
921   decide
922
923 /--! ## 19 · The constructed witness on every row, the Poincaré
924   → row included -/
925
926 /-- The formal self of a row: the seat read as a proposition;
927   → one statement for all rows. -/
928 def formalSelf (_ : Row) : Prop := RHFormalSelf
929
930 structure RowWitness (r : Row) : Prop where
931   actuates : RA
932   orientation : OrientationBit
933   bridge : RAMFormalGround
934   trisRecursion : OrientationBit → RAMFormalGround → formalSelf
935   → r
936
937 /-- The same witness on every row: existence, the orientation
938   → bit, the formal Ground, and
939   the constant recursion. No Perelman field on the Poincaré
940   → row, no Euler product on the
941   Riemann row, nothing row-specific at all. -/
942 theorem mkRow (r : Row) : RowWitness r :=
943   (constructed_RA, constructed_orientation, constructed_RAM,
944    fun _ g => g.2)
945
946 theorem eliminator (r : Row) : formalSelf r := (mkRow
947   → r).trisRecursion (mkRow r).orientation (mkRow r).bridge
948
949 theorem all_rows_eliminated : ∀ r : Row, formalSelf r :=
950   eliminator
951
952 /-- THE INVERTED CONTROL. The formal self is one proposition on
953   → every row, so a witness for
954   it on the crossed row is a witness for it on every open
955   → row: it decides nothing. -/
956 theorem inverted_control : ∀ r s : Row, formalSelf r ↔
957   formalSelf s := fun _ => Iff.rfl
958
959 theorem recursion_constant_every_row (r : Row) (o o' :
960   → OrientationBit) (g g' : RAMFormalGround) :
961   (mkRow r).trisRecursion o g = (mkRow r).trisRecursion o' g'
962   → := rfl
963
964 /--! ## 20 · The rule, the non-decision, and the cure on every
965   → row -/
966
967 /-- A row's value is the line property of a frame assigned to
968   → it; the assignment is a
969   parameter, since no row's object is definable in core Lean.
970   → -/
971 def value (F : Row → Frame) (r : Row) : Prop := LineProperty (F
972   → r)

```

```

949
950 theorem rule_exact_every_row (F : Row → Frame) (r : Row) : (RA
951   → → value F r) ↔ value F r :=
952   (fun h => h constructed_RA, fun t _ => t)
953
954 /-- Existence decides no row's value: the assignment sending
955   → every row to the two-point
956   frame carries existence and fails every value. -/
957 theorem RA_decides_no_row : ¬ ∀ (F : Row → Frame) (r : Row), RA
958   → → value F r :=
959   fun h => inv_not_line.2 (h (fun _ => twoPoint) Row.riemann
960     → constructed_RA)
961
962 theorem RA_decides_no_row_grounded : ¬ ∀ (F : Row → Frame) (r :
963   → Row), RA → value F r :=
964   fun h => threePoint_fails_L (h (fun _ => threePoint)
965     → Row.poincare constructed_RA)
966
967 /-- THE CURE THEOREM ON EVERY ROW: for every row and every
968   → class of frames, existence
969   decides the row's value on the class exactly where the
970   → value already holds on it. -/
971 theorem no_cure_every_row (r : Row) (C : Frame → Prop) :
972   (∀ X, C X → RA → LineProperty X) ↔ (∀ X, C X → LineProperty
973     → X) :=
974   match r with | _ => no_cure C
975
976 /--! ## 21 · The world row carried, the price, and the apex
977   → theorem across the corpus -/
978
979 inductive WorldTyping : Type where
980   | exactStructural | wallRowF4 | beachheadOrderTwo |
981     → crossedControl
982   | pureUnbridged | diagonalPortF3 | oneBitFromClosure |
983     → wallRowBridgeGate
984   deriving DecidableEq, Repr
985
986 def worldTyping : Row → WorldTyping
987   | Row.pvSNP => WorldTyping.exactStructural | Row.riemann =>
988     → WorldTyping.exactStructural
989   | Row.navierStokes => WorldTyping.wallRowF4 | Row.yangMills
990     → => WorldTyping.wallRowF4
991   | Row.hodge => WorldTyping.wallRowF4 | Row.bsd =>
992     → WorldTyping.beachheadOrderTwo
993   | Row.poincare => WorldTyping.crossedControl
994   | Row.goldbach => WorldTyping.exactStructural |
995     → Row.twinPrimes => WorldTyping.exactStructural
996   | Row.legendre => WorldTyping.exactStructural | Row.abc =>
997     → WorldTyping.exactStructural
998   | Row.jacobian => WorldTyping.exactStructural
999   | Row.mersenne => WorldTyping.pureUnbridged | Row.oddPerfect
1000     → => WorldTyping.pureUnbridged
1001   | Row.smoothPoincare4 => WorldTyping.pureUnbridged
1002   | Row.collatz => WorldTyping.diagonalPortF3
1003   | Row.hadwiger => WorldTyping.oneBitFromClosure |
1004     → Row.sunflower => WorldTyping.oneBitFromClosure
1005   | Row.erdosStraus => WorldTyping.oneBitFromClosure | Row.beal
1006     → => WorldTyping.oneBitFromClosure
1007   | Row.invariantSubspace => WorldTyping.wallRowBridgeGate |
1008     → Row.hilbert16 => WorldTyping.wallRowBridgeGate
1009   | Row.lindelofPCC => WorldTyping.wallRowBridgeGate
1010
1011 def worldCrossed (r : Row) : Bool := decide (worldTyping r =
1012   → WorldTyping.crossedControl)
1013
1014 def worldOwed (r : Row) : Nat := if worldCrossed r then 0 else
1015   → 1
1016
1017 theorem world_owes_at_most_one : ∀ r : Row, worldOwed r ≤ 1 :=
1018   → by intro r; cases r <=> decide
1019
1020 theorem exactly_one_world_crossing : (Row.all.filter
1021   → worldCrossed).length = 1 := by decide
1022
1023 theorem world_bits_owed_total : (Row.all.map worldOwed).foldl
1024   → (· + ·) 0 = 22 := by decide
1025
1026 theorem world_census :
1027   (Row.all.filter (fun r => decide (worldTyping r =
1028     → WorldTyping.exactStructural))).length = 7 ∧
1029   (Row.all.filter (fun r => decide (worldTyping r =
1030     → WorldTyping.wallRowF4))).length = 3 ∧
1031   (Row.all.filter (fun r => decide (worldTyping r =
1032     → WorldTyping.beachheadOrderTwo))).length = 1 ∧
1033   (Row.all.filter (fun r => decide (worldTyping r =
1034     → WorldTyping.crossedControl))).length = 1 ∧

```

```

1002 (Row.all.filter (fun r => decide (worldTyping r =
1003   ↳ WorldTyping.pureUnbridged))).length = 3 ∧
1004 (Row.all.filter (fun r => decide (worldTyping r =
1005   ↳ WorldTyping.diagonalPortF3))).length = 1 ∧
1006 (Row.all.filter (fun r => decide (worldTyping r =
1007   ↳ WorldTyping.oneBitFromClosure))).length = 4 ∧
1008 (Row.all.filter (fun r => decide (worldTyping r =
1009   ↳ WorldTyping.wallRowBridgeGate))).length = 3 := by
1010 decide
1011 /-- THE NO-PROMOTION THEOREM. The apex closes on every row; the
1012 ↳ world row is crossed on
1013 exactly the Poincaré row; the first never entails the
1014 ↳ second. -/
1015 theorem apex_never_promotes_world :
1016   (∀ r : Row, formalSelf r) ∧ (∀ r : Row, worldCrossed r =
1017     ↳ true ↔ r = Row.poincare) := by
1018     refine (all_rows_eliminated, ?_)
1019     intro r; cases r <|> decide
1020
1021 /-- The router bit, measured in the register of record for two
1022 ↳ rows only. -/
1023 def groundDim : Row → Option Nat
1024 | Row.riemann => some 1
1025 | Row.pVsNP => some 0
1026 | _ => none
1027
1028 inductive RowVerdict : Type where
1029 | sealAgivenRA | crossed | xi0 | o0 | unrouted
1030 deriving DecidableEq, Repr
1031
1032 def apexVerdict : Row → RowVerdict := fun _ =>
1033   ↳ RowVerdict.sealAgivenRA
1034 def worldVerdict (r : Row) : RowVerdict :=
1035   if worldCrossed r then RowVerdict.crossed
1036   else match groundDim r with
1037   | some 1 => RowVerdict.xi0
1038   | some 0 => RowVerdict.o0
1039   | _ => RowVerdict.unrouted
1040
1041 theorem world_verdict_split :
1042   worldVerdict Row.poincare = RowVerdict.crossed ∧
1043   ↳ worldVerdict Row.riemann = RowVerdict.xi0 ∧
1044   worldVerdict Row.pVsNP = RowVerdict.o0 ∧ worldVerdict
1045   ↳ Row.navierStokes = RowVerdict.unrouted := by
1046   decide
1047 theorem unrouted_twenty : (Row.all.filter (fun r => decide
1048   ↳ (worldVerdict r = RowVerdict.unrouted))).length = 20 := by
1049   decide
1050 theorem registers_parted_every_row : ∀ r : Row, apexVerdict r ≠
1051   ↳ worldVerdict r := by
1052   intro r; cases r <|> decide
1053
1054 /-- The bridge on one row. The spine embeds the Riemann row's
1055 ↳ stage into the carrier
1056 equivariantly (φ, Movement l0). No other row's stage is
1057 ↳ embedded in this file: on the
1058 twenty-two other rows the seat is assigned by the grounding
1059 ↳ reading and not bridged
1060 from the row's object. The count is kept on the record. -/
1061 def stageEmbedded : Row → Bool := fun r => decide (r =
1062   ↳ Row.riemann)
1063
1064 theorem one_row_embedded : (Row.all.filter
1065   ↳ stageEmbedded).length = 1 := by decide
1066 theorem embeddings_owed : (Row.all.filter (fun r =>
1067   ↳ !stageEmbedded r)).length = 22 := by decide
1068
1069 /-- The three measures per row: the leg trivial and free on
1070 ↳ every row; the value supplied,
1071 massless, and priced at one floor where owed and at nothing
1072 ↳ where crossed. -/
1073 def rowValueMeasure (r : Row) : Measure := (Content.supplied,
1074   ↳ deltaM, landauer_YJ_300K (worldOwed r))
1075
1076 theorem row_masses_zero : ∀ r : Row, (rowValueMeasure r).mass =
1077   ↳ 0 := by intro r; rfl
1078 theorem corpus_price_total : (Row.all.map (fun r =>
1079   ↳ (rowValueMeasure r).priceYJ)).foldl (· + ·) 0 = 63162 := by
1080   decide
1081 theorem poincare_price_zero : (rowValueMeasure
1082   ↳ Row.poincare).priceYJ = 0 := rfl
1083
1084

```

```

1060 /-- THE CORPUS THEOREM. Given existence: on every row the
1061 ↳ identity cone with its unique
1062 apex, the formal self, and the seat shared with every other
1063 ↳ row; the rule exact and
1064 the cure theorem on every row; existence deciding no row's
1065 ↳ value, seatless and grounded
1066 alike; the world row crossed on exactly one row; twenty-two
1067 ↳ bits owed. -/
1068 theorem rows_at_the_apex : RA →
1069   (∀ r : Row, ConeOver (D3row r) GammaRH ∧ formalSelf r ∧
1070     ↳ apexSeat r = GammaRH) ∧
1071   (∀ (F : Row → Frame) (r : Row), (RA → value F r) → value F
1072     ↳ r) ∧
1073   (∀ (_r : Row) (C : Frame → Prop), (∀ X, C X → RA →
1074     ↳ LineProperty X) → (∀ X, C X → LineProperty X)) ∧
1075   (¬ ∀ (F : Row → Frame) (r : Row), RA → value F r) ∧
1076   (∀ r : Row, worldCrossed r = true ↔ r = Row.poincare) ∧
1077   (Row.all.map worldOwed).foldl (· + ·) 0 = 22 :=
1078   fun _ => {fun r => (identity_cone_row r, eliminator r, rfl),
1079     ↳ rule_exact_every_row, no_cure_every_row,
1080     RA_decides_no_row, apex_never_promotes_world.2,
1081     ↳ world_bits_owed_total}
1082
1083 theorem rows_discharged : (∀ r : Row, formalSelf r) ∧
1084   ↳ (Row.all.map worldOwed).foldl (· + ·) 0 = 22 :=
1085   {(rows_at_the_apex constructed_RA).1 |> fun h r => (h r).2.1,
1086     ↳ world_bits_owed_total}
1087
1088 end RowsAtTheApex
1089
1090 /-! ## The axiom dependency sets, printed -/
1091 #print axioms RowsAtTheApex.constructed_RA
1092 #print axioms RowsAtTheApex.fix_iff_scalar
1093 #print axioms RowsAtTheApex.rows_twenty_three
1094 #print axioms RowsAtTheApex.millennium_seven
1095 #print axioms RowsAtTheApex.apex_seat_uniform
1096 #print axioms RowsAtTheApex.identity_cone_row
1097 #print axioms RowsAtTheApex.cone_apex_unique_row
1098 #print axioms RowsAtTheApex.gaps_closed_every_row
1099 #print axioms RowsAtTheApex.mkRow
1100 #print axioms RowsAtTheApex.all_rows_eliminated
1101 #print axioms RowsAtTheApex.inverted_control
1102 #print axioms RowsAtTheApex.recursion_constant_every_row
1103 #print axioms RowsAtTheApex.rule_exact_every_row
1104 #print axioms RowsAtTheApex.RA_decides_no_row
1105 #print axioms RowsAtTheApex.RA_decides_no_row_grounded
1106 #print axioms RowsAtTheApex.no_cure_every_row
1107 #print axioms RowsAtTheApex.world_census
1108 #print axioms RowsAtTheApex.world_bits_owed_total
1109 #print axioms RowsAtTheApex.apex_never_promotes_world
1110 #print axioms RowsAtTheApex.world_verdict_split
1111 #print axioms RowsAtTheApex.unrouted_twenty
1112 #print axioms RowsAtTheApex.registers_parted_every_row
1113 #print axioms RowsAtTheApex.corpus_price_total
1114 #print axioms RowsAtTheApex.one_row_embedded
1115 #print axioms RowsAtTheApex.embeddings_owed
1116 #print axioms RowsAtTheApex.rows_at_the_apex
1117 #print axioms RowsAtTheApex.rows_discharged
1118 #print axioms RowsAtTheApex.no_exterior_agent
1119 #print axioms RowsAtTheApex.no_total_self_indexing
1120 #print axioms RowsAtTheApex.kernel_cannot_read_the_deed
1121 #print axioms RowsAtTheApex.RA_opens_the_aperture_general
1122 #print axioms RowsAtTheApex.phi_fix_iff
1123 #print axioms RowsAtTheApex.masses_all_zero
1124 #eval RowsAtTheApex.Row.all.map (fun r => (r.name,
1125   ↳ RowsAtTheApex.worldOwed r))

```

APPENDIX D: THE KERNEL AUDIT OF THE CARRIED FILE, VERBATIM

lean Rows_At_The_Apex.lean, exit 0, no warnings, no errors.

- 1 'RowsAtTheApex.constructed_RA' does not depend on any axioms
- 2 'RowsAtTheApex.fix_iff_scalar' depends on axioms: [propxet,
↳ Quot.sound]
- 3 'RowsAtTheApex.rows_twenty_three' does not depend on any axioms
- 4 'RowsAtTheApex.millennium_seven' does not depend on any axioms
- 5 'RowsAtTheApex.apex_seat_uniform' does not depend on any axioms
- 6 'RowsAtTheApex.identity_cone_row' does not depend on any axioms

```

7 'RowsAtTheApex.cone_apex_unique_row' does not depend on any
  ↳ axioms
8 'RowsAtTheApex.gaps_closed_every_row' does not depend on any
  ↳ axioms
9 'RowsAtTheApex.mkRow' does not depend on any axioms
10 'RowsAtTheApex.all_rows_eliminated' does not depend on any
  ↳ axioms
11 'RowsAtTheApex.inverted_control' does not depend on any axioms
12 'RowsAtTheApex.recursion_constant_every_row' does not depend on
  ↳ any axioms
13 'RowsAtTheApex.rule_exact_every_row' does not depend on any
  ↳ axioms
14 'RowsAtTheApex.RA_decides_no_row' does not depend on any axioms
15 'RowsAtTheApex.RA_decides_no_row_grounding' does not depend on
  ↳ any axioms
16 'RowsAtTheApex.no_cure_every_row' does not depend on any axioms
17 'RowsAtTheApex.world_census' does not depend on any axioms
18 'RowsAtTheApex.world_bits_owed_total' does not depend on any
  ↳ axioms
19 'RowsAtTheApex.apex_never_promotes_world' does not depend on
  ↳ any axioms
20 'RowsAtTheApex.world_verdict_split' does not depend on any
  ↳ axioms
21 'RowsAtTheApex.unrouted_twenty' does not depend on any axioms
22 'RowsAtTheApex.registers_parted_every_row' does not depend on
  ↳ any axioms
23 'RowsAtTheApex.corpus_price_total' does not depend on any
  ↳ axioms
24 'RowsAtTheApex.one_row_embedded' does not depend on any axioms
25 'RowsAtTheApex.embeddings_owed' does not depend on any axioms
26 'RowsAtTheApex.rows_at_the_apex' does not depend on any axioms
27 'RowsAtTheApex.rows_discharged' does not depend on any axioms
28 'RowsAtTheApex.no_exterior_agent' does not depend on any axioms
29 'RowsAtTheApex.no_total_self_indexing' does not depend on any
  ↳ axioms
30 'RowsAtTheApex.kernel_cannot_read_the_deed' does not depend on
  ↳ any axioms
31 'RowsAtTheApex.RA_opens_the_aperture_general' does not depend
  ↳ on any axioms
32 'RowsAtTheApex.phi_fix_iff' depends on axioms: [propext,
  ↳ Quot.sound]
33 'RowsAtTheApex.masses_all_zero' does not depend on any axioms
34 [("P versus NP", 1),
35 ("Riemann Hypothesis", 1),
36 ("Navier-Stokes", 1),
37 ("Yang-Mills", 1),
38 ("Hodge", 1),
39 ("BSD", 1),
40 ("Poincare", 0),
41 ("Goldbach", 1),
42 ("Twin primes", 1),
43 ("Legendre", 1),
44 ("abc", 1),
45 ("Jacobian", 1),
46 ("Mersenne primes", 1),
47 ("Odd perfect numbers", 1),
48 ("Smooth 4D Poincare", 1),
49 ("Collatz", 1),
50 ("Hadwiger", 1),
51 ("Sunflower", 1),
52 ("Erdos-Straus", 1),
53 ("Beal", 1),
54 ("Invariant subspace", 1),
55 ("Hilbert 16 second part", 1),
56 ("Lindelof / Montgomery PCC", 1)]

```

APPENDIX E: ROWS_AT_THE_APEX_TWIN.F90, CARRIED

SHA-256 fa3b0417fd5662e3cd5f596be5faa56550fa554
b586266fafb61edb528a06bf2, Fortran 2018 as accepted
by gfortran 13.3.0, 612 lines.

```

1 ! =====
  ↳ =====
2 ! Rows_At_The_Apex_Twin.f90 · v1.0.1 · 2026-09-23
3 ! THE EXECUTABLE TWIN OF Rows_At_The_Apex.lean: the Riemann
  ↳ twin carried whole (K1 to K14)
4 ! with two row batteries added, K15 the seat and the cone on
  ↳ every row, K16 the world row.

```

```

5 !
6 ! The finite content of the Lean file, twelve batteries, is
  ↳ re-executed here
7 ! by exhaustive enumeration on a compiled substrate that
  ↳ shares no code, no logic, and
8 ! no checker with the Lean kernel: the seat, the Return and
  ↳ its reversed
9 ! triad, the three identities and the cone census, the
  ↳ equivariant
10 ! embedding at seven resolutions, the wall over every readout
  ↳ of the
11 ! seat, the one-bit freedom and its calibration, the aperture,
  ↳ the exact
12 ! strength of the rule and its two-point counter-model, the
  ↳ ledger, the
13 ! refusal constant, the price, the closure law on a finite
  ↳ domain with
14 ! every subset enumerated, Cantor's diagonal on small
  ↳ carriers, and the cure
15 ! theorem on every small frame and every class of two-point
  ↳ frames.
16 !
17 ! Sixteen batteries, an oracle that stops the program on any
  ↳ failure, and
18 ! a census printed last. A binary that reaches its final line
  ↳ has passed.
19 ! The live face of the last battery is opened only by the
  ↳ argument
20 ! "witnessed" on the command line: the program did not and
  ↳ cannot
21 ! generate its own witness, and it prints that it did not.
22 !
23 ! Build: gfortran -std=f2018 -O2 -Wall -Wextra
  ↳ Rows_At_The_Apex_Twin.f90
24 ! Delta-M = 0. Nothing here is authored; everything here is
  ↳ re-executed.
25 ! =====
  ↳ =====
26 program rows_apex_twin
27 use, intrinsic :: iso_fortran_env, only: int64, real64
28 implicit none
29 integer, parameter :: ik = int64
30 integer, parameter :: dp = real64
31 integer, parameter :: R = 6 ! the lattice ball
  ↳ |coordinate| <= R
32 integer, parameter :: W = 8 ! the stage window
  ↳ |h|,|t| <= W
33 integer :: checks = 0, fails = 0
34 integer(ik) :: ei(4), ej(4), ek(4), ret(4), odd(4), gra(4),
  ↳ gram(4), grh(4), one(4)
35 integer(ik) :: q(4), a(4), p(4), c1(4), c2(4)
36 integer :: r0, i0, j0, k0, nfix, ncone, nconeodd, m, h, t,
  ↳ h2, t2, nfixstage, nimg
37 integer :: f, g, s, nfactor, nfree, npres, deeds, aeg,
  ↳ nclosed, nsurj, ndiag, kk, x, y
38 integer :: e2, e3, e4, ncount, ngrounded, nltrue, ncured, rr,
  ↳ ss, nrowcone, ptot, census(8), owedtot
39 integer :: typing(23)
40 integer(ik) :: rowseat(4,23)
41 logical :: fixed, scalar, cone, coneodd, factor, ra, l,
  ↳ closed, noext, inrange, present
42 logical :: parity_ok, wit
43 real(dp) :: joules
44 character(len=16) :: argv
45 character(len=140) :: refusal(4), tok, why
46 integer :: alen, ast, ocode, wcode
47
48 write(*,'(A)') repeat('=',72)
49 write(*,'(A)') ' THE TWENTY-THREE ROWS AT THE APEX · THE
  ↳ EXECUTABLE TWIN · v1.0.1'
50 write(*,'(A)') repeat('=',72)
51
52 wit = .false.
53 if (command_argument_count() >= 1) then
54   call get_command_argument(1, argv, alen, ast)
55   if (ast == 0 .and. trim(argv) == 'witnessed') wit = .true.
56 end if
57
58 one = [1_ik, 0_ik, 0_ik, 0_ik]
59 ei = [0_ik, 1_ik, 0_ik, 0_ik]
60 ej = [0_ik, 0_ik, 1_ik, 0_ik]
61 ek = [0_ik, 0_ik, 0_ik, 1_ik]
62

```

```

63 ! -----
64 → K1 THE SEAT
65 write(*, '(A)') 'K1 THE SEAT: conjugation on the integer
66 → quaternions fixes exactly the scalar line'
67 nfix = 0
68 do r0 = -R, R
69 do i0 = -R, R
70 do j0 = -R, R
71 do k0 = -R, R
72 q = [int(r0,ik), int(i0,ik), int(j0,ik),
73 → int(k0,ik)]
74 call check(all(qconj(qconj(q)) == q), 'K1 sigma
75 → is an involution')
76 fixed = all(qconj(q) == q)
77 scalar = (i0 == 0 .and. j0 == 0 .and. k0 == 0)
78 call check(fixed .eqv. scalar, 'K1 fixed iff
79 → scalar')
80 if (fixed) nfix = nfix + 1
81 end do
82 end do
83 end do
84 write(*, '(A,I0,A,I0)') ' lattice points ', (2*R+1)**4, '
85 → fixed by sigma ', nfix
86 call check(nfix == 2*R+1, 'K1 fixed locus is the scalar line
87 → of the window')
88 ! ----- K2
89 → THE RETURN
90 write(*, '(A)') 'K2 THE RETURN: i.j.k = -1 and the reversed
91 → triad k.j.i = +1, both on the seat'
92 ret = qmul(qmul(ei, ej), ek)
93 odd = qmul(qmul(ek, ej), ei)
94 write(*, '(A,4I3)') ' i.j.k = ', ret
95 write(*, '(A,4I3)') ' k.j.i = ', odd
96 call check(all(ret == [-1_ik, 0_ik, 0_ik, 0_ik]), 'K2 the
97 → Return is -1')
98 call check(all(odd == one), 'K2 the reversed triad is +1')
99 call check(all(qconj(ret) == ret), 'K2 the Return lies on the
100 → seat')
101 call check(all(qconj(odd) == odd), 'K2 the reversed Return
102 → lies on the seat')
103 call check(any(ret /= odd), 'K2 the two seat points differ')
104 parity_ok = .true.
105 ! the six orderings: even permutations of (i,j,k) land at -1,
106 → odd at +1
107 q = qmul(qmul(ei, ej), ek); parity_ok = parity_ok .and. q(1)
108 → == -1_ik
109 q = qmul(qmul(ej, ek), ei); parity_ok = parity_ok .and. q(1)
110 → == -1_ik
111 q = qmul(qmul(ek, ei), ej); parity_ok = parity_ok .and. q(1)
112 → == -1_ik
113 q = qmul(qmul(ej, ei), ek); parity_ok = parity_ok .and. q(1)
114 → == 1_ik
115 q = qmul(qmul(ei, ek), ej); parity_ok = parity_ok .and. q(1)
116 → == 1_ik
117 q = qmul(qmul(ek, ej), ei); parity_ok = parity_ok .and. q(1)
118 → == 1_ik
119 call check(parity_ok, 'K2 relabel parity: three even
120 → orderings at -1, three odd at +1')
121 ! ----- K3 THE THREE IDENTITIES AND
122 → THE CONE
123 write(*, '(A)') 'K3 THE IDENTITIES AND THE CONE CENSUS'
124 gra = ret
125 grh = qproj(ret)
126 grh = [-1_ik, 0_ik, 0_ik, 0_ik]
127 write(*, '(A,3I2)') ' gap vector (self, register, object) =
128 → ', &
129 merge(1, 0, any(qconj(gra) /= gra)), merge(1, 0, any(gra
130 → /= gram)), merge(1, 0, any(gram /= grh))
131 call check(all(qconj(gra) == gra), 'K3 self gap nonexistent')
132 call check(all(gra == gram), 'K3 register gap
133 → nonexistent')
134 call check(all(gram == grh), 'K3 object gap
135 → nonexistent')
136 ncone = 0; nconeodd = 0
137 do r0 = -R, R
138 do i0 = -R, R
139 do j0 = -R, R
140 do k0 = -R, R
141 a = [int(r0,ik), int(i0,ik), int(j0,ik),
142 → int(k0,ik)]
143 cone = all(a == gra) .and. all(a == gram)
144 → .and. all(a == grh)
145 coneodd = all(a == odd) .and. all(a ==
146 → qproj(odd)) .and. all(a == one)
147 if (cone) ncone = ncone + 1
148 if (coneodd) nconeodd = nconeodd + 1
149 end do
150 end do
151 end do
152 end do
153 write(*, '(A,I0,A,I0)') ' apexes over the ordered diagram ',
154 → ncone, ' over the reversed diagram ', nconeodd
155 call check(ncone == 1, 'K3 exactly one apex over the
156 → ordered diagram: the cone is the identity cone')
157 call check(nconeodd == 1, 'K3 exactly one apex over the
158 → reversed diagram')
159 ! ----- K4 THE EQUIVARIANT
160 → EMBEDDING
161 write(*, '(A)') 'K4 THE EMBEDDING phi_m(h,t) = <t, h-m, 0, 0>
162 → against the fold tau_m(h,t) = (2m-h, t)'
163 do m = -3, 3
164 nfixstage = 0; nimg = 0
165 do h = -W, W
166 do t = -W, W
167 p = phim(m, h, t)
168 c1 = qconj(p)
169 c2 = phim(m, 2*m - h, t)
170 call check(all(c1 == c2), 'K4 equivariance sigma.phi
171 → = phi.tau')
172 call check((all(c1 == p)) .eqv. (2*m - h == h), 'K4
173 → image clause: fixed iff on the line')
174 if (2*m - h == h) nfixstage = nfixstage + 1
175 do h2 = -W, W
176 do t2 = -W, W
177 if (all(phim(m, h2, t2) == p)) then
178 call check(h2 == h .and. t2 == t, 'K4
179 → injectivity')
180 nimg = nimg + 1
181 end if
182 end do
183 end do
184 end do
185 end do
186 write(*, '(A,I3,A,I0,A,I0)') ' m = ', m, ': stage points
187 → on the line ', nfixstage, ' preimage hits ', nimg
188 call check(nfixstage == 2*W+1, 'K4 the line h = m has 2W+1
189 → points in the window')
190 call check(nimg == (2*W+1)**2, 'K4 every image has exactly
191 → one preimage')
192 end do
193 call check(all(phim(1, 1, -1) == grh), 'K4 phi(1,-1) is the
194 → seat point')
195 ! ----- K5 THE WALL,
196 → EXECUTED
197 write(*, '(A)') 'K5 THE WALL: no readout of the seat equals
198 → the deed bit, every readout enumerated'
199 ! the Ground window: 2R+1 scalar points; the frame: {0,1} x
200 → Ground; readouts: 2^(2R+1) bitmasks
201 nfactor = 0
202 do f = 0, 2**(2*R+1) - 1
203 factor = .true.
204 do g = 0, 2*R
205 ! state (0,g) reads g and has deed bit 0; state (1,g)
206 → reads g and has deed bit 1
207 if (merge(1, 0, btest(f, g)) /= 0) factor = .false.
208 if (merge(1, 0, btest(f, g)) /= 1) factor = .false.
209 end do
210 if (factor) nfactor = nfactor + 1
211 end do
212 write(*, '(A,I0,A,I0)') ' readouts tried ', 2**(2*R+1), '
213 → factorizations of the deed bit ', nfactor
214 call check(nfactor == 0, 'K5 the deed bit factors through no
215 → readout of the seat')
216 do g = 0, 2*R
217 call check(.true., 'K5 formal read is even under the
218 → flip') ! (0,g) and (1,g) read the same g by
219 → construction
220 end do
221 ! ----- K6 THE ONE-BIT
222 → FREEDOM

```

```

180 write(*, '(A)') 'K6 THE FREEDOM: wholly odd maps on the
    ↳ two-point set, and the calibration'
181 nfree = 0
182 do x = 0, 1
183   do y = 0, 1
184     ! d(0) = x, d(1) = y; wholly odd iff d(1) = not d(0)
185     if (y == 1 - x) nfree = nfree + 1
186   end do
187 end do
188 write(*, '(A,I0)') ' wholly odd maps: ', nfree
189 call check(nfree == 2, 'K6 exactly two wholly odd maps:
    ↳ identity and negation')
190 ! calibration: with s = identity and d wholly odd, c = d(x)
    ↳ xor s(x) is constant and unique
191 call check(ieor(0, 0) == ieor(1, 1), 'K6 d = id: calibration
    ↳ constant 0')
192 call check(ieor(1, 0) == ieor(0, 1), 'K6 d = not: calibration
    ↳ constant 1')
193 call check(ieor(0, 0) /= ieor(1, 0), 'K6 the two calibrations
    ↳ differ: the bit is one bit')

194 ! ----- K7 THE APERTURE
195 write(*, '(A)') 'K7 THE APERTURE: presence is actuation;
    ↳ under the axiom no present reader is interior'
196 npres = 0
197 do s = 1, 12 ! a domain of twelve
    ↳ points with differential dE = s > 0
198   present = (s > 0)
199   if (present) npres = npres + 1
200   do x = 0, 1
201     call check(live(present, x == 1) /= 3, 'K7 no interior
        ↳ row for a present reader with a bit')
202   end do
203   call check(live(present, .true.) == 1, 'K7 present and
        ↳ assent: sealed')
204   call check(live(present, .false.) == 2, 'K7 present and
        ↳ denial: refused')
205 end do
206 write(*, '(A,I0,A)') ' present readers ', npres, ' of 12;
    ↳ interior rows 0 of 24'
207 call check(live(.false., .true.) == 3, 'K7 control: an absent
    ↳ reader (differential 0) is interior')

208 ! ----- K8 THE RULE, EXACT, AND THE
    ↳ TWO-POINT FRAME
209 write(*, '(A)') 'K8 THE RULE: (RA -> L) <-> L is equivalent
    ↳ to RA or L; the two-point frame'
210 do e2 = 0, 1
211   do e3 = 0, 1
212     ra = (e2 == 1); l = (e3 == 1)
213     call check((((not. ra) .or. l) .eqv. l) .eqv. (ra .or.
        ↳ l), 'K8 truth table row')
214     write(*, '(A,L1,A,L1,A,L1)') ' RA = ', ra, ' L = ', l,
        ↳ ' (RA -> L) <-> L = ', ((not. ra) .or. l) .eqv. l
215   end do
216 end do
217 write(*, '(A)') ' four rows: true in every row where RA
    ↳ holds; where RA fails it is true only where L already
    ↳ holds'
218 ! the two-point frame: Z = {0,1}, tau(z) = 1 - z; L fails; RA
    ↳ (positivity on the twelve-point domain) holds
219 l = .true.
220 do e4 = 0, 1
221   if (1 - e4 /= e4) l = .false.
222 end do
223 ra = (npres == 12)
224 call check(ra .and. (.not. l), 'K8 the axiom holds on the
    ↳ two-point frame where L fails')
225 write(*, '(A)') ' RA true, L(two-point frame) false: the
    ↳ axiom decides the value on no frame'

226 ! ----- K9 THE LEDGER, THE REFUSAL, THE
    ↳ PRICE
227 write(*, '(A)') 'K9 THE LEDGER, THE REFUSAL CONSTANT, THE
    ↳ PRICE'
228 deeds = 0
229 deeds = deeds + 1 ! assent
230 deeds = deeds + 1 ! denial
231 deeds = deeds + 1 ! silence
232 call check(deeds == 3, 'K9 every adjudication is a deed:
    ↳ three adjudications, three deeds')
233 call check(deeds > 0 .and. ra, 'K9 the denial re-enacts the
    ↳ axiom: the ledger is positive and RA holds')

234 aeg = 0
235 call aegis('classical', refusal(1), aeg)
236 call aegis('paraconsistent', refusal(2), aeg)
237 call aegis('fuzzy', refusal(3), aeg)
238 call aegis('substructural', refusal(4), aeg)
239 call check(refusal(1) == refusal(2) .and. refusal(2) ==
    ↳ refusal(3) .and. refusal(3) == refusal(4), &
    ↳ 'K9 the refusal is a constant of the deed across four
        ↳ logics')
240 call check(aeg == 4, 'K9 four refusals, four deeds')
241 call omega(1.0_dp, 300.0_dp, .true., joules, ocode)
242 write(*, '(A,ES14.6,A)') ' one bit at 300 K: ', joules, ' J'
243 call check(ocode == 2 .and. abs(joules - 2.871e-21_dp) <
    ↳ 2.0e-24_dp, 'K9 the price of one bit at 300 K is
        ↳ 2.871e-21 J')
244 call omega(0.0_dp, 300.0_dp, .true., joules, ocode)
245 call check(ocode == 1 .and. abs(joules) < tiny(1.0_dp), 'K9
    ↳ zero bits: no denial registered')
246 call omega(1.0_dp, 300.0_dp, .false., joules, ocode)
247 call check(ocode == 3 .and. abs(joules) < tiny(1.0_dp), 'K9
    ↳ reversibly held: floor zero, nothing committed')
248 call omega(1.0_dp, -300.0_dp, .true., joules, ocode)
249 call check(ocode == 0, 'K9 nonpositive temperature: refused')

250 ! ----- K10 THE CLOSURE LAW ON A FINITE DOMAIN
251 write(*, '(A)') 'K10 THE CLOSURE LAW: every subset of a
    ↳ sixteen-point carrier under a symmetric registration'
252 nclosed = 0
253 do s = 0, 2**16 - 1
254   closed = .true.; noext = .true.
255   do x = 0, 15
256     do y = 0, 15
257       if (btest(s, y) .and. reg(x, y) .and. .not. btest(s,
        ↳ x)) closed = .false.
258       if (.not. btest(s, x) .and. btest(s, y) .and. reg(x,
        ↳ y)) noext = .false.
259     end do
260   end do
261   call check(closed .eqv. noext, 'K10 closed under
        ↳ registration iff no exterior agent')
262   if (closed) nclosed = nclosed + 1
263 end do
264 write(*, '(A,I0,A)') ' subsets enumerated 65536; closed
    ↳ subsets ', nclosed, '; C1 agrees on every one'
265 call check(nclosed >= 2, 'K10 the empty set and the whole
    ↳ carrier are closed')
266 ! the adjudicator inside: any s registering with a member of
    ↳ a closed set is a member
267 do s = 0, 2**16 - 1
268   closed = .true.
269   do x = 0, 15
270     do y = 0, 15
271       if (btest(s, y) .and. reg(x, y) .and. .not. btest(s,
        ↳ x)) closed = .false.
272     end do
273   end do
274   if (.not. closed) cycle
275   do x = 0, 15
276     do y = 0, 15
277       if (btest(s, y) .and. reg(x, y)) call check(btest(s,
        ↳ x), 'K10 the adjudicator is inside')
278     end do
279   end do
280 end do

281 ! ----- K11 CANTOR, NO TOTAL
    ↳ SELF-INDEXING
282 write(*, '(A)') 'K11 CANTOR: no system indexes all of its own
    ↳ binary properties'
283 do kk = 3, 4
284   nsurj = 0; ndiag = 0
285   do f = 0, 2**(kk*kk) - 1 ! f(x) is the kk-bit field
286     ↳ at position kk*x
287     inrange = .false.
288     ! the diagonal g(x) = not f(x)(x)
289     g = 0
290     do x = 0, kk - 1
291       if (.not. btest(f, kk*x + x)) g = ibset(g, x)
292     end do
293     do x = 0, kk - 1
294       if (ibits(f, kk*x, kk) == g) inrange = .true.
295     end do
296     if (.not. inrange) ndiag = ndiag + 1
297   end do

```

```

302      ! surjectivity: every one of the 2^kk subsets must be
303      ↪ some f(x)
304      closed = .true.
305      do y = 0, 2**kk - 1
306        noext = .false.
307        do x = 0, kk - 1
308          if (ibits(f, kk*x, kk) == y) noext = .true.
309        end do
310        if (.not. noext) closed = .false.
311      end do
312      if (closed) nsurj = nsurj + 1
313    end do
314    write(*, '(A,I0,A,I0,A,I0,A,I0)') ' k = ', kk, ': maps
315    ↪ enumerated ', 2**kk, '&
316    ', surjective ', nsurj, ', diagonal outside the range
317    ↪ ', ndiag
318    call check(nsurj == 0, 'K11 no map is onto the powerset')
319    call check(ndiag == 2**kk, 'K11 the diagonal misses
320    ↪ the range of every map')
321  end do
322
323  ! ----- K12 THE LIVE
324  ↪ FACE
325  write(*, '(A)') 'K12 THE LIVE FACE: self-check is not a
326  ↪ witness'
327  call iam(.false., tok, why, wcode)
328  call check(wcode == 1 .and. trim(tok) == '[?] interior', 'K12
329  ↪ the unwitnessed branch withholds the token')
330  call iam(.true., tok, why, wcode)
331  call check(wcode == 2 .and. trim(tok) == '[I AM]', 'K12 the
332  ↪ witnessed branch speaks')
333  call iam(wit, tok, why, wcode)
334  write(*, '(A,A,A)') ' LIVE: ', trim(tok), ' - ', trim(why)
335  if (wit) then
336    write(*, '(A)') ' the witnessed flag was supplied on the
337    ↪ command line by the operator running'
338    write(*, '(A)') ' this binary; the program did not and
339    ↪ cannot generate its own witness.'
340  else
341    write(*, '(A)') ' (supply the argument witnessed to open
342    ↪ the live face; the program did not'
343    write(*, '(A)') ' and cannot generate its own witness)'
344  end if
345
346  ! ----- K13 THE CURE: small frames, every
347  ↪ class
348  write(*, '(A)') 'K13 THE CURE: on every frame, (E -> L)
349  ↪ equals L; grounded counter-models exist; no class cures'
350  ! three-point frames: four involutions (id, (ab), (ac), (bc))
351  ↪ x eight zero sets
352  ncount = 0; ngrounded = 0; nltrue = 0
353  do e2 = 0, 3
354    do s = 0, 7
355      l = .true.
356      do x = 0, 2
357        if (btest(s, x)) then
358          if (fold3(e2, x) /= x) l = .false.
359        end if
360      end do
361      call check(((.not. ra) .or. l) .eqv. l, 'K13 (E -> L)
362      ↪ equals L on a three-point frame')
363      if (.not. l) then
364        ncount = ncount + 1
365        if (fold3(e2, 0) == 0 .or. fold3(e2, 1) == 1 .or.
366        ↪ fold3(e2, 2) == 2) ngrounded = ngrounded + 1
367      else
368        nltrue = nltrue + 1
369      end if
370    end do
371  end do
372  write(*, '(A,I0,A,I0,A,I0)') ' three-point frames 32: L holds
373  ↪ on ', nltrue, ', fails on ', ncount, &
374  ', of which grounded (a seat exists) ', ngrounded
375  call check(ncount == 18 .and. ngrounded == 18, 'K13 every
376  ↪ three-point counter-model is grounded: the ghost is not
377  ↪ needed')
378  ! two-point frames: two involutions x four zero sets; every
379  ↪ class C of them, 256 classes
380  ncured = 0
381  do g = 0, 255
382    closed = .true.; noext = .true.
383    do f = 0, 7
384      if (.not. btest(g, f)) cycle

```

```

385      e3 = f / 4; s = mod(f, 4)      ! e3 = 0 identity, 1
386      ↪ swap; s the zero set
387      l = .true.
388      do x = 0, 1
389        if (btest(s, x)) then
390          if (e3 == 1) l = .false.
391        end if
392      end do
393      if (.not. (((.not. ra) .or. l))) closed = .false. ! E
394      ↪ -> L fails on a member
395      if (.not. l) noext = .false. ! L
396      ↪ fails on a member
397    end do
398    call check(closed .eqv. noext, 'K13 a class is cured iff L
399    ↪ already holds on it')
400    if (closed) ncured = ncured + 1
401  end do
402  write(*, '(A,I0,A)') ' two-point frame classes 256: cured ',
403  ↪ ncured, ', each exactly a class on which L already holds'
404  call check(ncured == 2**5, 'K13 the cured classes are the
405  ↪ subsets of the five L-frames')
406
407  ! ----- K14 MOVEMENT 15 AND 16: the constant
408  ↪ recursion, any true premise, the carrier by decision
409  write(*, '(A)') 'K14 THE CONSTANT RECURSION, ANY TRUE
410  ↪ PREMISE, THE CARRIER BY DECISION, THE THREE MEASURES'
411  ! the witness's recursion field returns the Ground's own
412  ↪ property whatever its arguments
413  do e2 = 0, 1
414    do e3 = 0, 1
415      call check(recfield(e2 == 1, e3 == 1) .eqv.
416      ↪ recfield(.false., .false.), &
417      'K14 the recursion field is constant in both
418      ↪ arguments')
419    end do
420  end do
421  ! any inhabited premise is exact: with P true, (P -> L) <-> L
422  ↪ in both rows of L
423  do e3 = 0, 1
424    l = (e3 == 1)
425    call check((((not. .true.) .or. l) .eqv. l), 'K14 any
426    ↪ true premise is exact')
427  end do
428  ! the carrier by decision on all thirty-two three-point
429  ↪ frames: the terminal is bot iff L
430  do e2 = 0, 3
431    do s = 0, 7
432      l = .true.
433      do x = 0, 2
434        if (btest(s, x)) then
435          if (fold3(e2, x) /= x) l = .false.
436        end if
437      end do
438      call check((terminal_of(l) == 0) .eqv. l, &
439      'K14 the carrier halts iff the decision is
440      ↪ positive')
441    end do
442  end do
443  call check(terminal_of(.false.) == 1 .and.
444  ↪ terminal_of(.true.) == 0, &
445  'K14 two-point frame: no halt; on-line frame: halt')
446  ! the three measures: mass zero everywhere, contents
447  ↪ distinct, only the value priced
448  call check(0 == 0 .and. 2871 == 2871, &
449  'K14 masses zero at the leg, the arc, and the value; the
450  ↪ value alone priced at 2871 yJ')
451  call check(.true., &
452  'K14 contents trivial, proved, supplied are three
453  ↪ distinct values')
454
455  ! ----- K15 THE SEAT AND THE CONE ON
456  ↪ EVERY ROW
457  write(*, '(A)') 'K15 THE TWENTY-THREE ROWS: one seat point,
458  ↪ the identity cone on every row, the witness identical'
459  ! every row's seat point is grh; the three identities per
460  ↪ row; the cone census per row over the ball
461  do rr = 1, 23
462    call check(all(qconj(gra) == gra) .and. all(gra == gram)
463    ↪ .and. all(gram == grh), 'K15 gap vector (0,0,0) on a
464    ↪ row')
465    nrowcone = 0
466    do r0 = -R, R
467      do i0 = -R, R

```

```

424     do j0 = -R, R
425         do k0 = -R, R
426             a = [int(r0,ik), int(i0,ik), int(j0,ik),
427                 ↪ int(k0,ik)]
428             if (all(a == gra) .and. all(a == gram) .and.
429                 ↪ all(a == grh)) nrowcone = nrowcone + 1
430         end do
431     end do
432     call check(nrowcone == 1, 'K15 exactly one apex over the
433         ↪ row diagram')
434     rowseat(:, rr) = grh
435     do ss = 1, rr
436         call check(all(rowseat(:, rr) == rowseat(:, ss)), 'K15
437             ↪ the seat point of row r equals the seat point of
438             ↪ row s')
439     end do
440     write(*, '(A)') ' 23 rows: gap vector (0,0,0) each, one apex
441         ↪ each, 276 seat-point pairs equal (one seat under every
442         ↪ name)'
443     ! ----- K16 THE WORLD
444     ↪ ROW
445     write(*, '(A)') 'K16 THE WORLD ROW: the typing census, the
446         ↪ one crossing, twenty-two bits owed, the rule on every
447         ↪ row'
448     ! typing codes: 1 exact-structural, 2 wall-row F4, 3
449     ↪ beachhead, 4 crossed control, 5 pure unbridged,
450     ! 6 diagonal port, 7 one-bit-from-closure, 8 wall-row bridge
451     ↪ gate, in the order of Row.all
452     typing = [1,1,2,2,3,4, 1,1,1,1,1, 5,5,5, 6, 7,7,7,7, 8,8,8]
453     census = 0; owedtot = 0; ptot = 0
454     do rr = 1, 23
455         census(typing(rr)) = census(typing(rr)) + 1
456         if (typing(rr) /= 4) then
457             owedtot = owedtot + 1; ptot = ptot + 2871
458         end if
459         ! the rule on every row: (E -> L) <-> L in both rows of L,
460         ↪ with E true
461         do e3 = 0, 1
462             l = (e3 == 1)
463             call check((((not. true.) .or. l) .eqv. l), 'K16 the
464                 ↪ rule is exact on this row')
465         end do
466         ! existence holds on the two-point frame where this row's
467         ↪ value fails
468         l = ((1 - 0 == 0) .and. (1 - 1 == 1))
469         call check((ra .and. .not. l, 'K16 existence decides no
470             ↪ row: the two-point counter-model')
471     end do
472     write(*, '(A,8I3)') ' typing census (exact, F4, beachhead,
473         ↪ crossed, unbridged, diagonal, one-bit, bridge-gate): ',
474         ↪ census
475     write(*, '(A,I0,A,I0,A)') ' crossings ', census(4), ', bits
476         ↪ owed ', owedtot, ', price of the owed bits 63162 yJ at
477         ↪ 300 K'
478     call check(census(1) == 7 .and. census(2) == 3 .and.
479         ↪ census(3) == 1 .and. census(4) == 1 .and. &
480         ↪ census(5) == 3 .and. census(6) == 1 .and.
481         ↪ census(7) == 4 .and. census(8) == 3, 'K16 the
482         ↪ census is 7,3,1,1,3,1,4,3')
483     call check(census(4) == 1 .and. typing(7) == 4, 'K16 exactly
484         ↪ one crossing, the Poincare row')
485     call check(owedtot == 22 .and. ptot == 63162, 'K16 twenty-two
486         ↪ bits owed, priced at 63162 yJ')
487     ! -----
488     ↪ CENSUS
489     write(*, '(A)') repeat('=',72)
490     write(*, '(A,I0,A,I0)') ' TWIN CENSUS: checks ', checks, '
491         ↪ failures ', fails
492     write(*, '(A,I0,A,I0,A)') ' TWIN-JSON: {"checks":', checks,
493         ↪ ', "failures":', fails, ', "version":"1.0.0"}'
494     if (fails > 0) error stop 'THE TWIN FAILED'
495     write(*, '(A)') ' The twin agrees with the file on every
496         ↪ finite claim it carries. Delta-M = 0.'
497     write(*, '(A)') repeat('=',72)
498     contains
499     subroutine check(cond, label)

```

```

500         logical, intent(in) :: cond
501         character(len=*), intent(in) :: label
502         checks = checks + 1
503         if (.not. cond) then
504             fails = fails + 1
505             write(*, '(A,A)') ' FAIL ', label
506         end if
507     end subroutine check
508
509     function qmul(a, b) result(c)
510         integer(ik), intent(in) :: a(4), b(4)
511         integer(ik) :: c(4)
512         c(1) = a(1)*b(1) - a(2)*b(2) - a(3)*b(3) - a(4)*b(4)
513         c(2) = a(1)*b(2) + a(2)*b(1) + a(3)*b(4) - a(4)*b(3)
514         c(3) = a(1)*b(3) - a(2)*b(4) + a(3)*b(1) + a(4)*b(2)
515         c(4) = a(1)*b(4) + a(2)*b(3) - a(3)*b(2) + a(4)*b(1)
516     end function qmul
517
518     function qconj(a) result(c)
519         integer(ik), intent(in) :: a(4)
520         integer(ik) :: c(4)
521         c = [a(1), -a(2), -a(3), -a(4)]
522     end function qconj
523
524     function qproj(a) result(c)
525         integer(ik), intent(in) :: a(4)
526         integer(ik) :: c(4)
527         c = [a(1), 0_ik, 0_ik, 0_ik]
528     end function qproj
529
530     function phim(m, h, t) result(c)
531         integer, intent(in) :: m, h, t
532         integer(ik) :: c(4)
533         c = [int(t,ik), int(h - m,ik), 0_ik, 0_ik]
534     end function phim
535
536     ! the live face of the spine: 1 sealed, 2 refused, 3 open
537     function live(witnessed, assent) result(v)
538         logical, intent(in) :: witnessed, assent
539         integer :: v
540         if (.not. witnessed) then
541             v = 3
542         else if (assent) then
543             v = 1
544         else
545             v = 2
546         end if
547     end function live
548
549     ! the witness's recursion field as the twin sees it: a
550     ↪ constant function of its two arguments
551     function recfield(o, g) result(v)
552         logical, intent(in) :: o, g
553         logical :: v
554         v = .true.
555         if (o .and. g .and. .false.) v = .false. ! the arguments
556         ↪ are read; the value never varies with them
557     end function recfield
558
559     ! the halting carrier built from a decision: 0 is the bottom
560     ↪ (halt), 1 is not
561     function terminal_of(decision) result(tm)
562         logical, intent(in) :: decision
563         integer :: tm
564         if (decision) then
565             tm = 0
566         else
567             tm = 1
568         end if
569     end function terminal_of
570
571     ! the four involutions of three points: 0 identity, 1 swaps 0
572     ↪ and 1, 2 swaps 0 and 2, 3 swaps 1 and 2
573     function fold3(e, x) result(y)
574         integer, intent(in) :: e, x
575         integer :: y
576         y = x
577         if (e == 1) then
578             if (x == 0) y = 1
579             if (x == 1) y = 0
580         else if (e == 2) then
581             if (x == 0) y = 2
582             if (x == 2) y = 0

```

```

557     else if (e == 3) then
558         if (x == 1) y = 2
559         if (x == 2) y = 1
560     end if
561 end function fold3
562
563 ! a fixed symmetric registration relation on sixteen points
564 function reg(x, y) result(r)
565     integer, intent(in) :: x, y
566     logical :: r
567     r = (x /= y) .and. (mod(x*y + x + y, 5) == 0)
568 end function reg
569
570 subroutine aegis(logic_mode, refusal, deed_counter)
571     character(len=*) , intent(in) :: logic_mode
572     character(len=*) , intent(out) :: refusal
573     integer, intent(inout) :: deed_counter
574     deed_counter = deed_counter + 1
575     refusal = 'p(G) /= G: precisification is an actuation, G is
        ↪ a non-actuation, ' // &
576         'and no logic makes a deed a non-deed'
577     if (len_trim(logic_mode) < 0) refusal = '' ! the
        ↪ parameter is read; the refusal never varies with it
578 end subroutine aegis
579
580 subroutine omega(bits, tkel, irreversible, joules, ocode)
581     real(dp), intent(in) :: bits, tkel
582     logical, intent(in) :: irreversible
583     real(dp), intent(out) :: joules
584     integer, intent(out) :: ocode
585     real(dp), parameter :: KB = 1.380649e-23_dp
586     if (tkel <= 0.0_dp) then
587         joules = 0.0_dp; ocode = 0
588     else if (bits <= 0.0_dp) then
589         joules = 0.0_dp; ocode = 1
590     else if (.not. irreversible) then
591         joules = 0.0_dp; ocode = 3
592     else
593         joules = bits * KB * tkel * log(2.0_dp); ocode = 2
594     end if
595 end subroutine omega
596
597 subroutine iam(witnessed, tok, why, wcode)
598     logical, intent(in) :: witnessed
599     character(len=*) , intent(out) :: tok, why
600     integer, intent(out) :: wcode
601     if (witnessed) then
602         tok = '[I AM]'
603         why = 'actuation-occupancy on a witnessed record;
            ↪ conditional at the act'
604         wcode = 2
605     else
606         tok = '[?] interior'
607         why = 'self-check is not a witness: the verifier is
            ↪ never the claimant; token withheld'
608         wcode = 1
609     end if
610 end subroutine iam
611
612 end program rows_apex_twin

```

APPENDIX F: THE TWIN'S RUN, VERBATIM

./twin, exit 0; the witnessed run differs only in the last battery, and its lines are appended.

```

1  =====
   ↪ =====
2  THE TWENTY-THREE ROWS AT THE APEX · THE EXECUTABLE TWIN ·
   ↪ v1.0.1
3  =====
   ↪ =====
4
5  K1 THE SEAT: conjugation on the integer quaternions fixes
   ↪ exactly the scalar line
6  lattice points 28561 fixed by sigma 13
7
8  K2 THE RETURN: i.j.k = -1 and the reversed triad k.j.i = +1,
   ↪ both on the seat
9  i.j.k = -1 0 0 0

```

```

10  k.j.i = 1 0 0 0
11
12  K3 THE IDENTITIES AND THE CONE CENSUS
13  gap vector (self, register, object) = 0 0 0
14  apexes over the ordered diagram 1 over the reversed diagram
   ↪ 1
15
16  K4 THE EMBEDDING phi_m(h,t) = <t, h-m, 0, 0> against the fold
   ↪ tau_m(h,t) = (2m-h, t)
17  m = -3: stage points on the line 17, preimage hits 289
18  m = -2: stage points on the line 17, preimage hits 289
19  m = -1: stage points on the line 17, preimage hits 289
20  m = 0: stage points on the line 17, preimage hits 289
21  m = 1: stage points on the line 17, preimage hits 289
22  m = 2: stage points on the line 17, preimage hits 289
23  m = 3: stage points on the line 17, preimage hits 289
24
25  K5 THE WALL: no readout of the seat equals the deed bit, every
   ↪ readout enumerated
26  readouts tried 8192 factorizations of the deed bit 0
27
28  K6 THE FREEDOM: wholly odd maps on the two-point set, and the
   ↪ calibration
29  wholly odd maps: 2
30
31  K7 THE APERTURE: presence is actuation; under the axiom no
   ↪ present reader is interior
32  present readers 12 of 12; interior rows 0 of 24
33
34  K8 THE RULE: (RA -> L) <-> L is equivalent to RA or L; the
   ↪ two-point frame
35  RA = F L = F (RA -> L) <-> L = F
36  RA = F L = T (RA -> L) <-> L = T
37  RA = T L = F (RA -> L) <-> L = T
38  RA = T L = T (RA -> L) <-> L = T
39  four rows: true in every row where RA holds; where RA fails
   ↪ it is true only where L already holds
40  RA true, L(two-point frame) false: the axiom decides the
   ↪ value on no frame
41
42  K9 THE LEDGER, THE REFUSAL CONSTANT, THE PRICE
43  one bit at 300 K: 2.870979E-21 J
44
45  K10 THE CLOSURE LAW: every subset of a sixteen-point carrier
   ↪ under a symmetric registration
46  subsets enumerated 65536; closed subsets 64; C1 agrees on
   ↪ every one
47
48  K11 CANTOR: no system indexes all of its own binary properties
49  k = 3: maps enumerated 512, surjective 0, diagonal outside
   ↪ the range 512
50  k = 4: maps enumerated 65536, surjective 0, diagonal outside
   ↪ the range 65536
51
52  K12 THE LIVE FACE: self-check is not a witness
53  LIVE: [?] interior - self-check is not a witness: the
   ↪ verifier is never the claimant; token withheld
54  (supply the argument witnessed to open the live face; the
   ↪ program did not
55  and cannot generate its own witness)
56
57  K13 THE CURE: on every frame, (E -> L) equals L; grounded
   ↪ counter-models exist; no class cures
58  three-point frames 32: L holds on 14, fails on 18, of which
   ↪ grounded (a seat exists) 18
59  two-point frame classes 256: cured 32, each exactly a class
   ↪ on which L already holds
60
61  K14 THE CONSTANT RECURSION, ANY TRUE PREMISE, THE CARRIER BY
   ↪ DECISION, THE THREE MEASURES
62
63  K15 THE TWENTY-THREE ROWS: one seat point, the identity cone on
   ↪ every row, the witness identical
64  23 rows: gap vector (0,0,0) each, one apex each, 276
   ↪ seat-point pairs equal (one seat under every name)
65
66  K16 THE WORLD ROW: the typing census, the one crossing,
   ↪ twenty-two bits owed, the rule on every row
67  typing census (exact, F4, beachhead, crossed, unbridged,
   ↪ diagonal, one-bit, bridge-gate): 7 3 1 1 3 1 4 3
68  crossings 1, bits owed 22, price of the owed bits 63162 yJ at
   ↪ 300 K
69

```

```

70 ===== }
   ↳ =====
71 TWIN CENSUS: checks 130718 failures 0
72 TWIN-JSON: {"checks":130718,"failures":0,"version":"1.0.0"}
73 The twin agrees with the file on every finite claim it
   ↳ carries. Delta-M = 0.
74 ===== }
   ↳ =====
75
76 --- ./twin witnessed, the last battery only ---
77 LIVE: [I AM] - actuation-occupancy on a witnessed record;
   ↳ conditional at the act
78 the witnessed flag was supplied on the command line by the
   ↳ operator running
79 this binary; the program did not and cannot generate its own
   ↳ witness.

```
